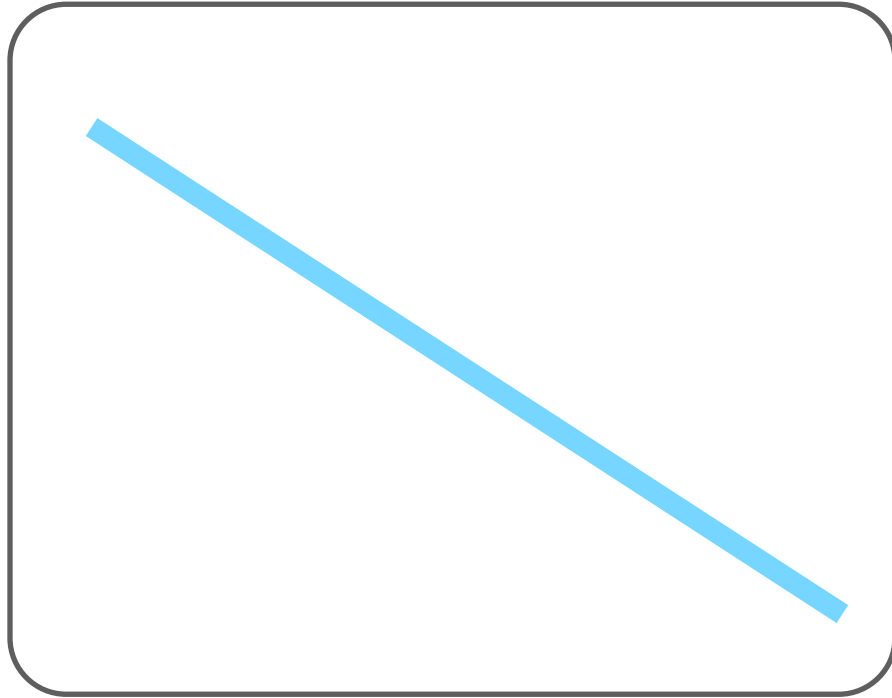
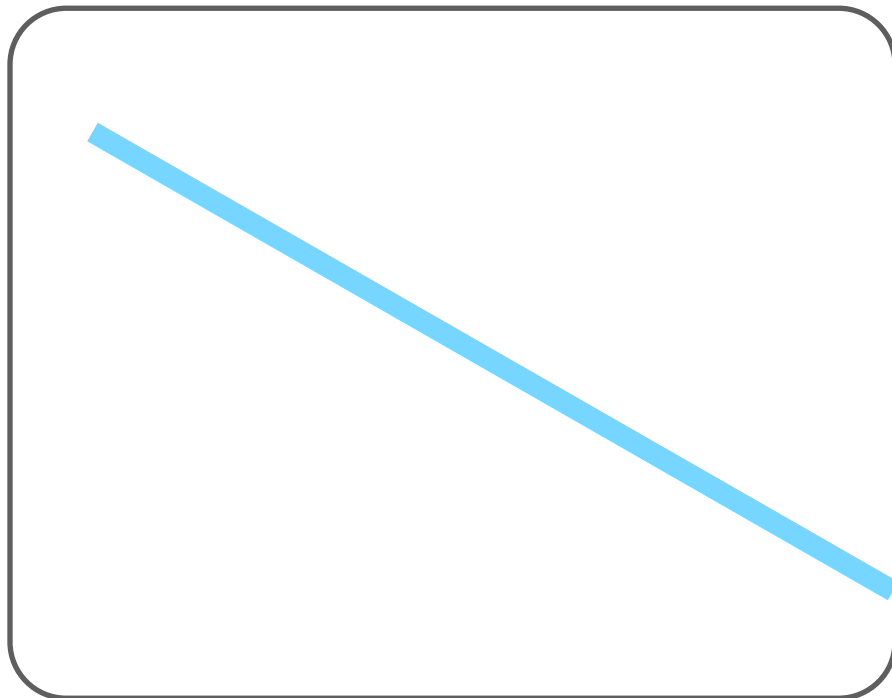
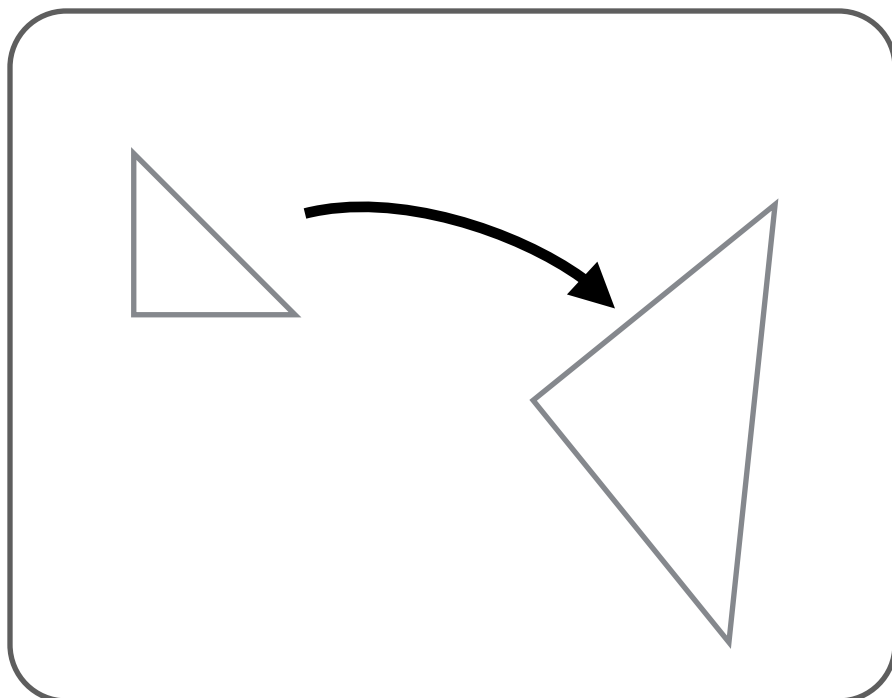




Linien zeichnen : Bresenham



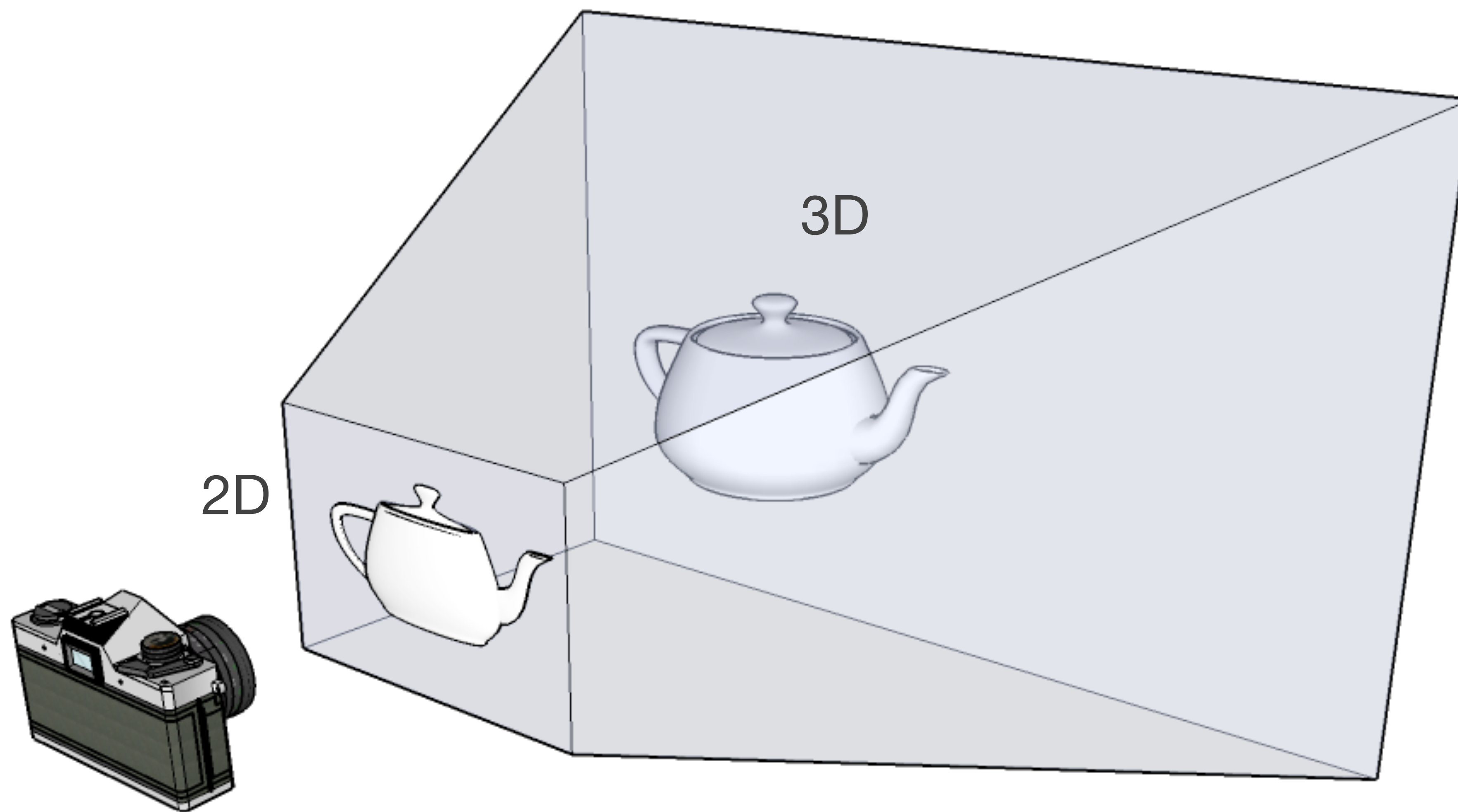
Clipping



Transformation

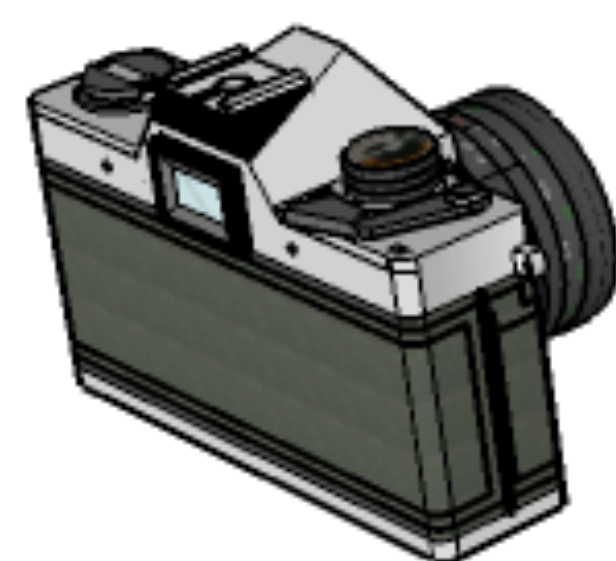
Von Perspektiven und weiteren Ansichten



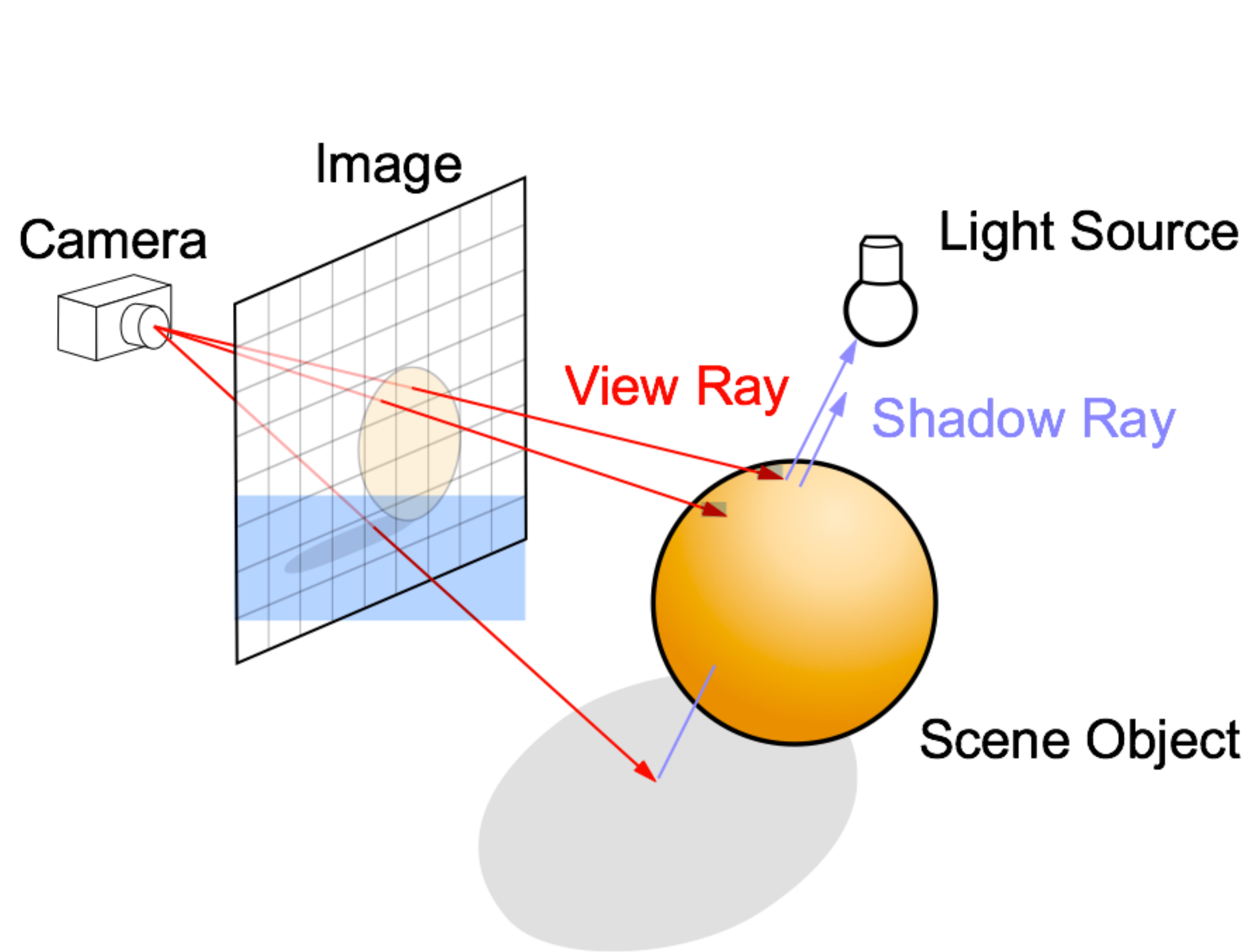


3D

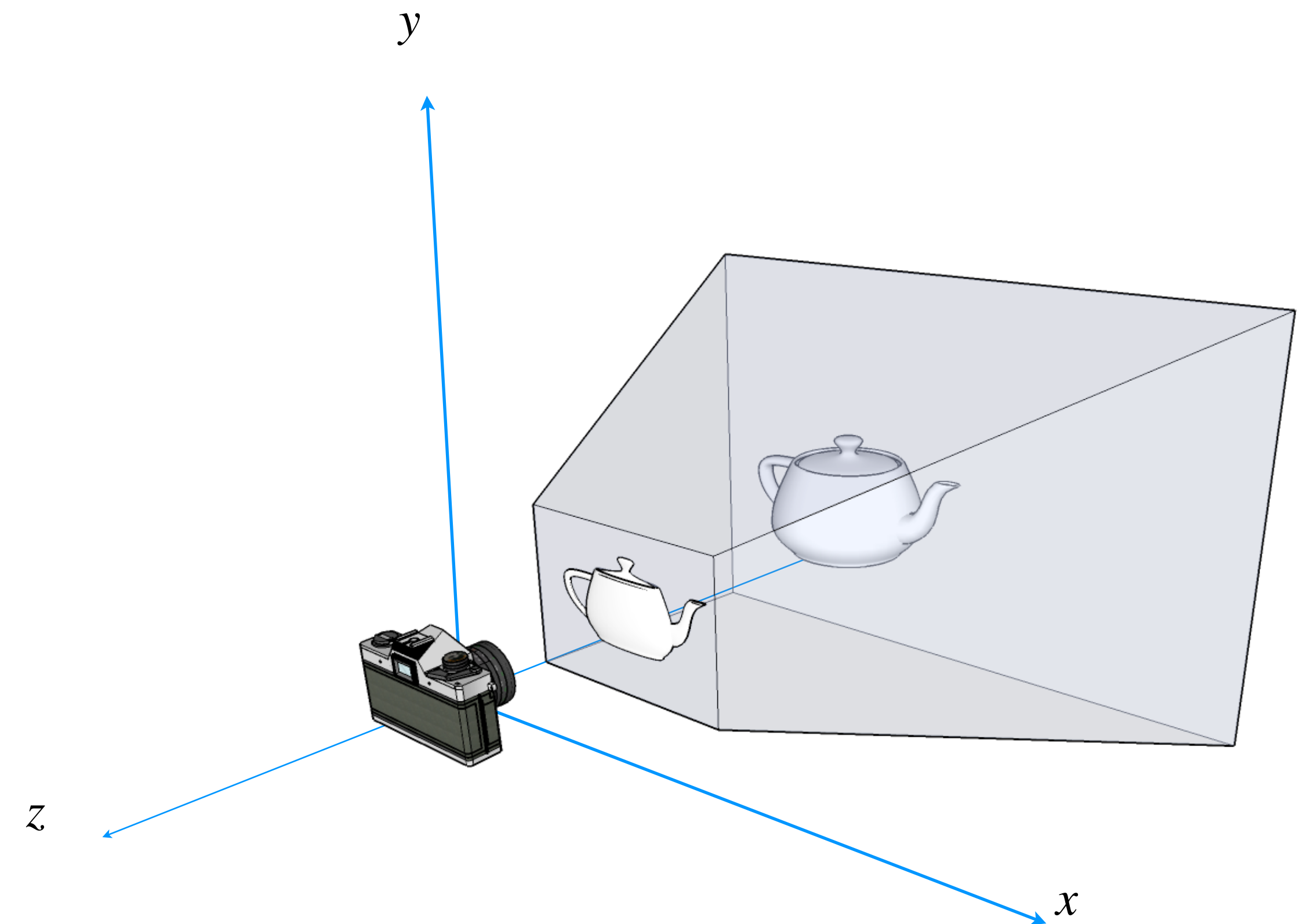
2D



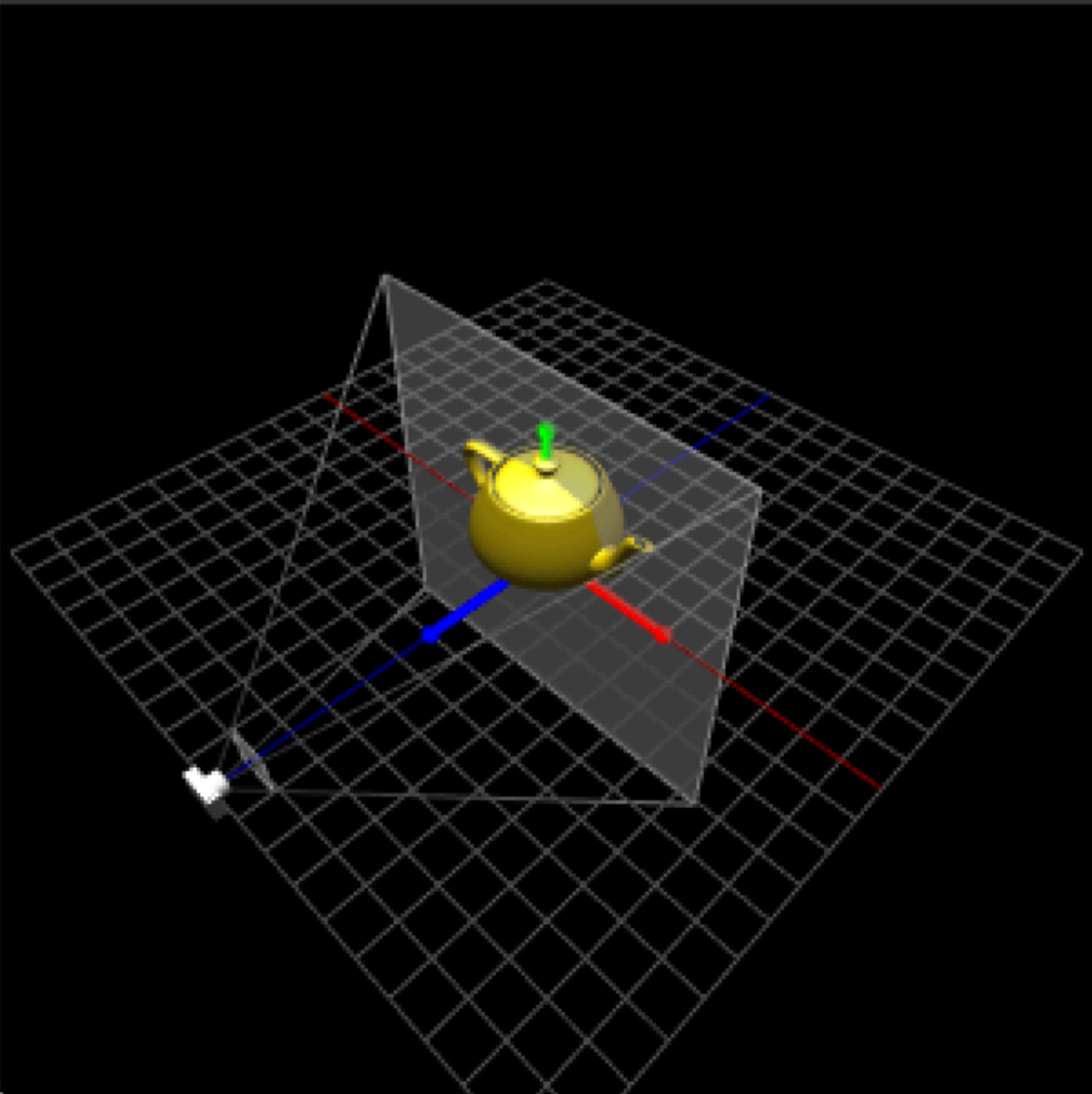
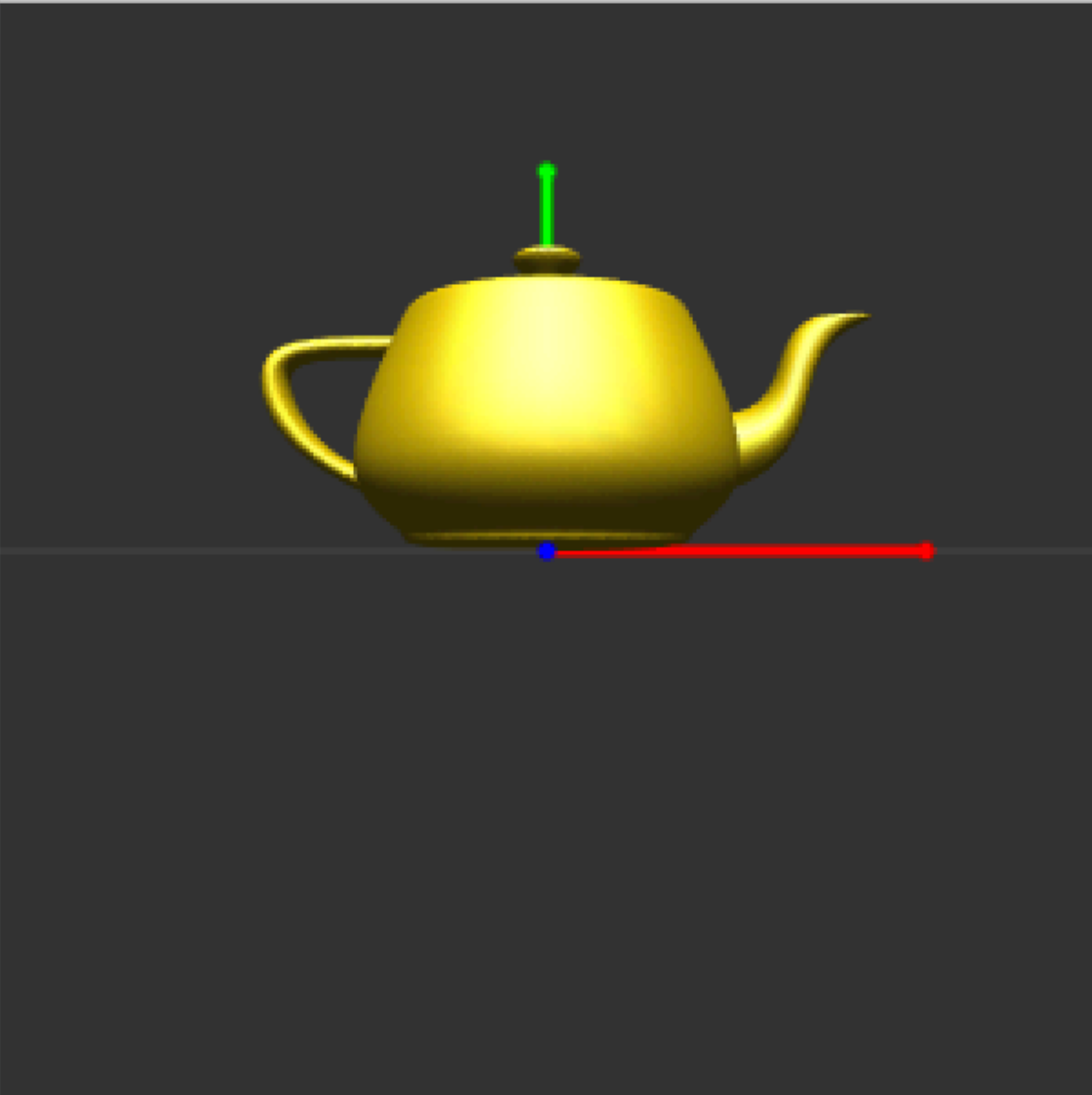
Viewpoint



Software Rendering (Raytracing...)



Hardware Rendering



OpenGL ModelView Matrix

View (Camera)

X0

Reset

OpenGL Functions

glRotatef(-0, 0,0,1);
glRotatef(-0, 0,1,0);
glRotatef(-0, 1,0,0);
glTranslatef(-0, -0, -10);

Position Y0

Z10

Pitch (X)0

Heading (Y)0

Roll (Z)0

Model

X0

Reset

OpenGL Functions

glTranslatef(0, 0, 0);
glRotatef(0, 1,0,0);
glRotatef(0, 0,1,0);
glRotatef(0, 0,0,1);

Position Y0

Z0

X0

Rotation Y0

Z0

ModelView Matrix

View Matrix

Model Matrix

1,000,000,000,00

0,001,000,000,00

0,000,001,00-10,00

0,000,000,001,00

=

1,000,000,000,00

0,001,000,000,00

0,000,001,00-10,00

0,000,000,001,00

x

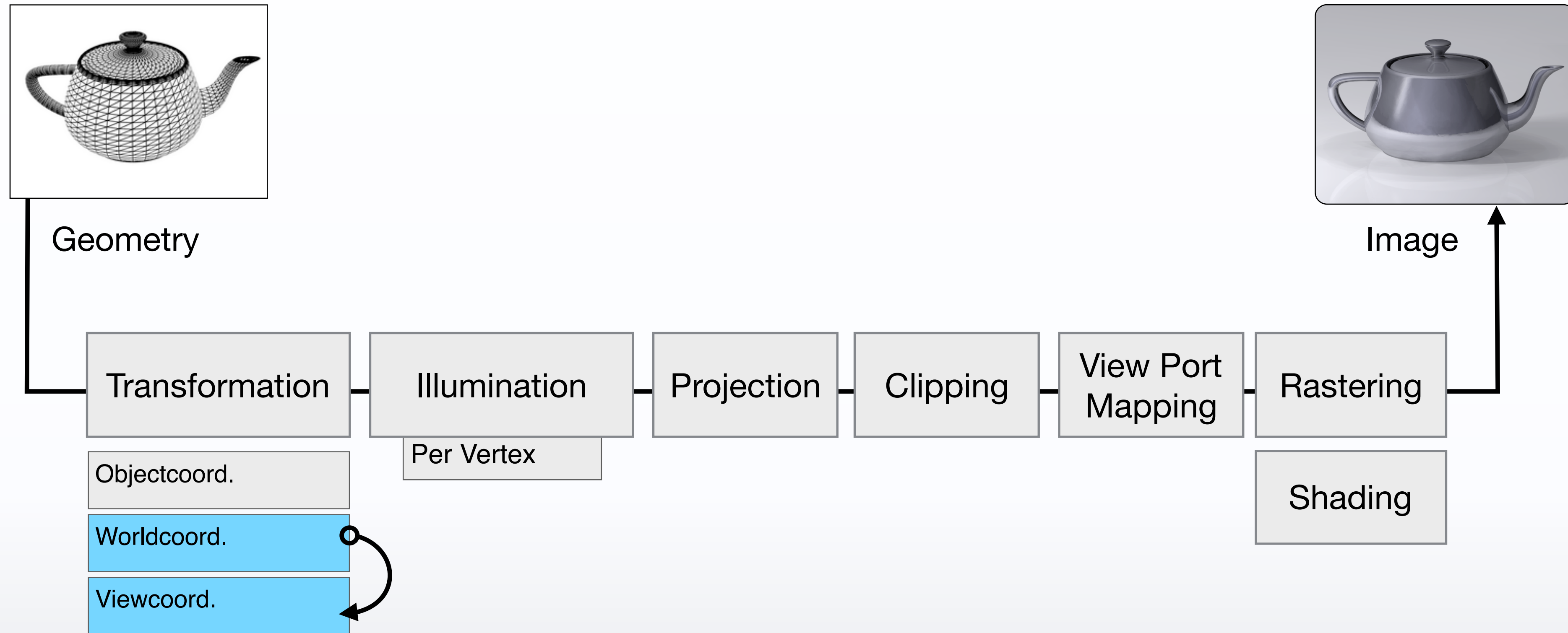
1,000,000,000,00

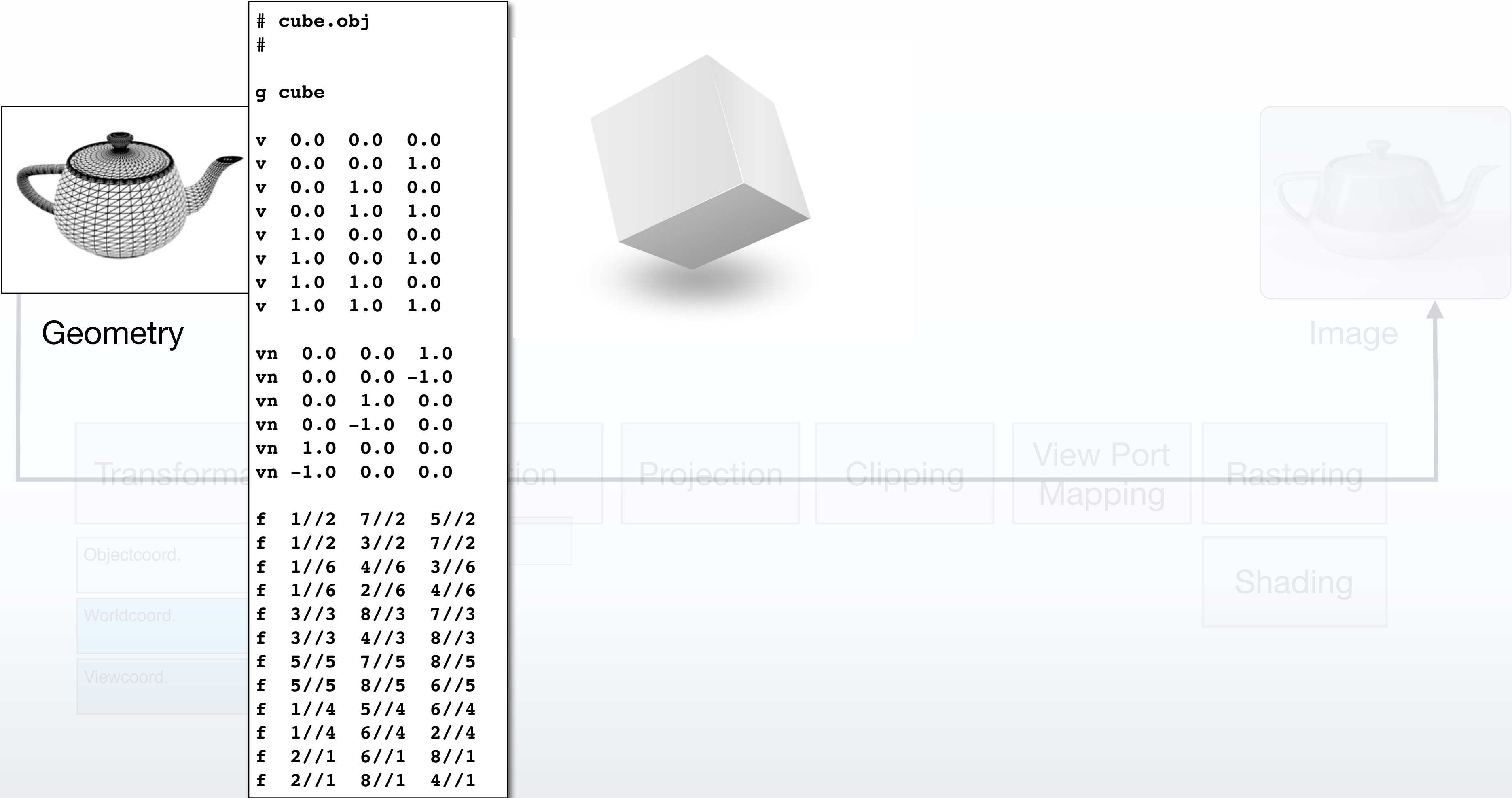
0,001,000,000,00

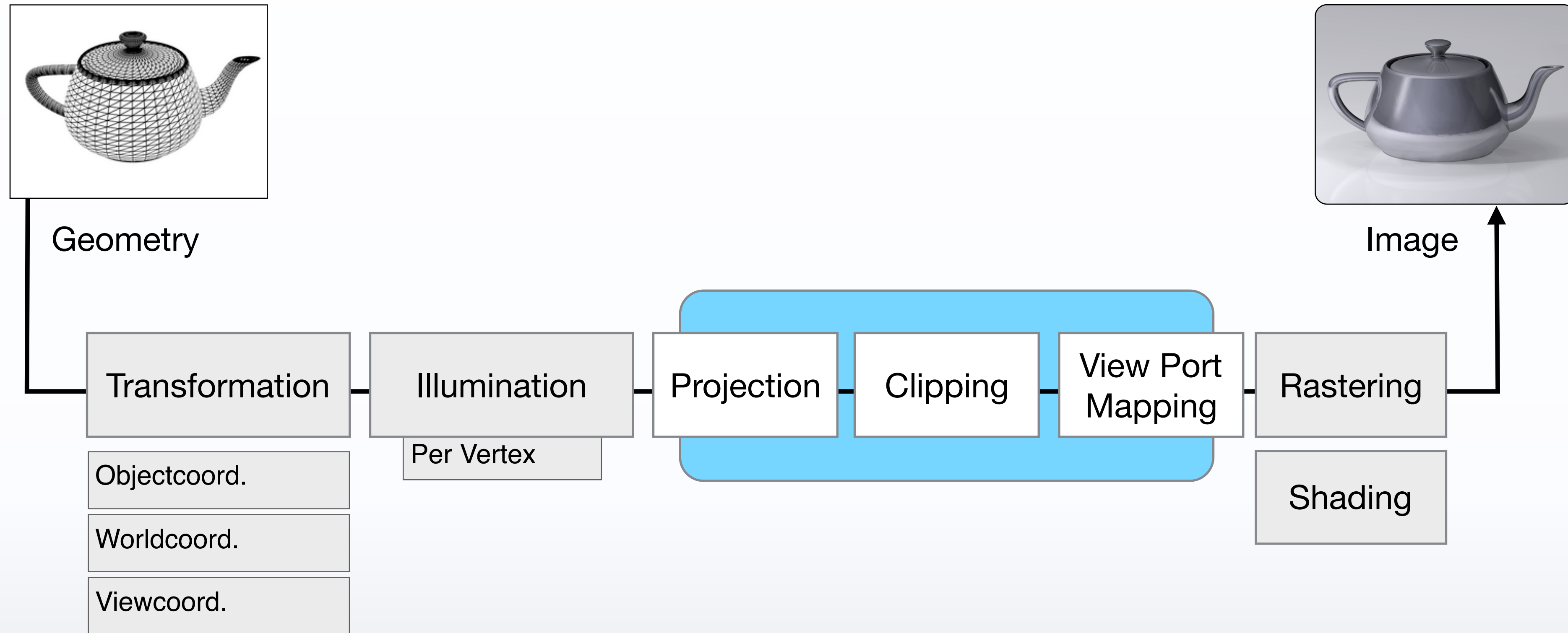
0,000,001,000,00

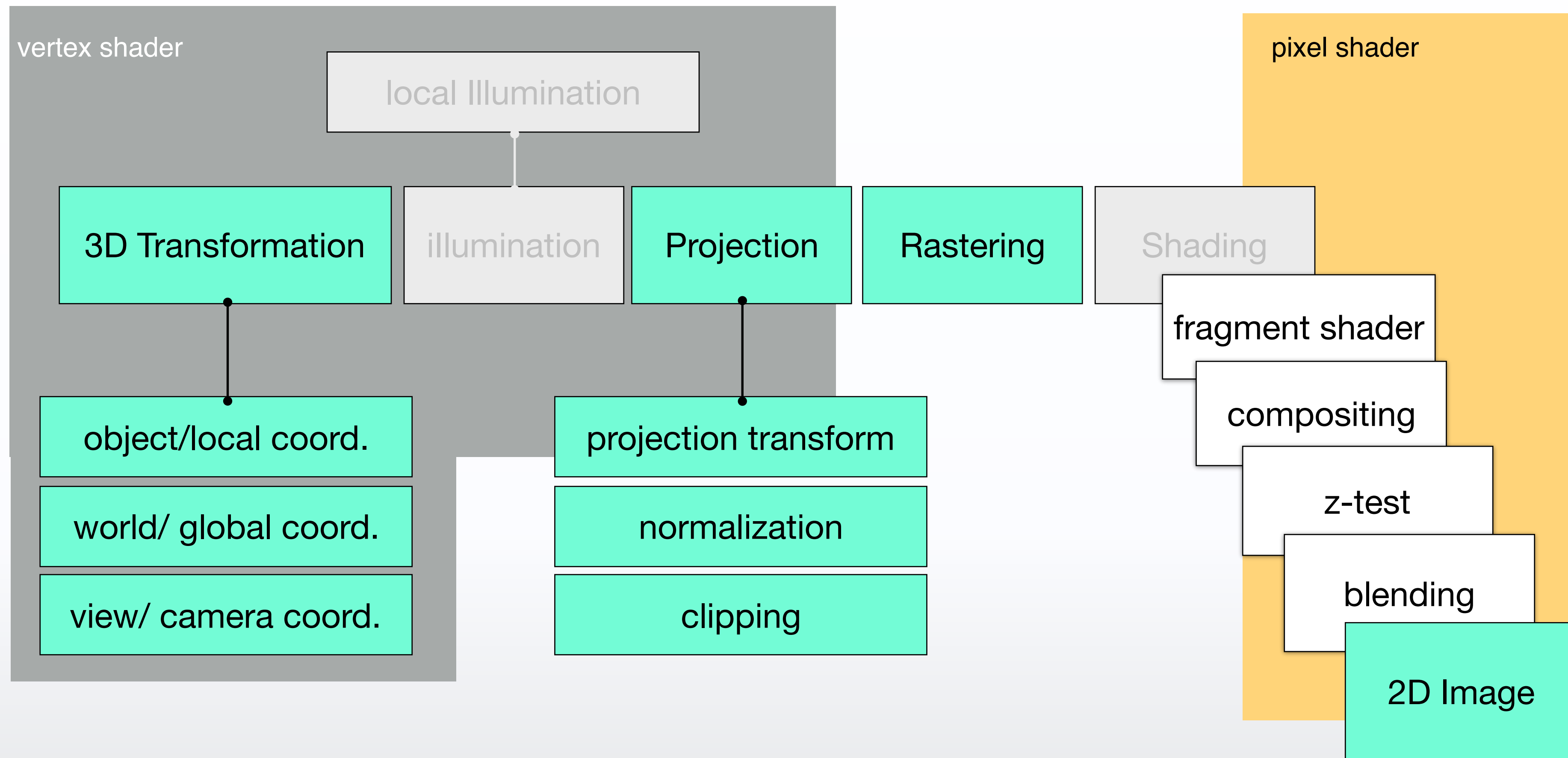
0,000,000,001,00

5



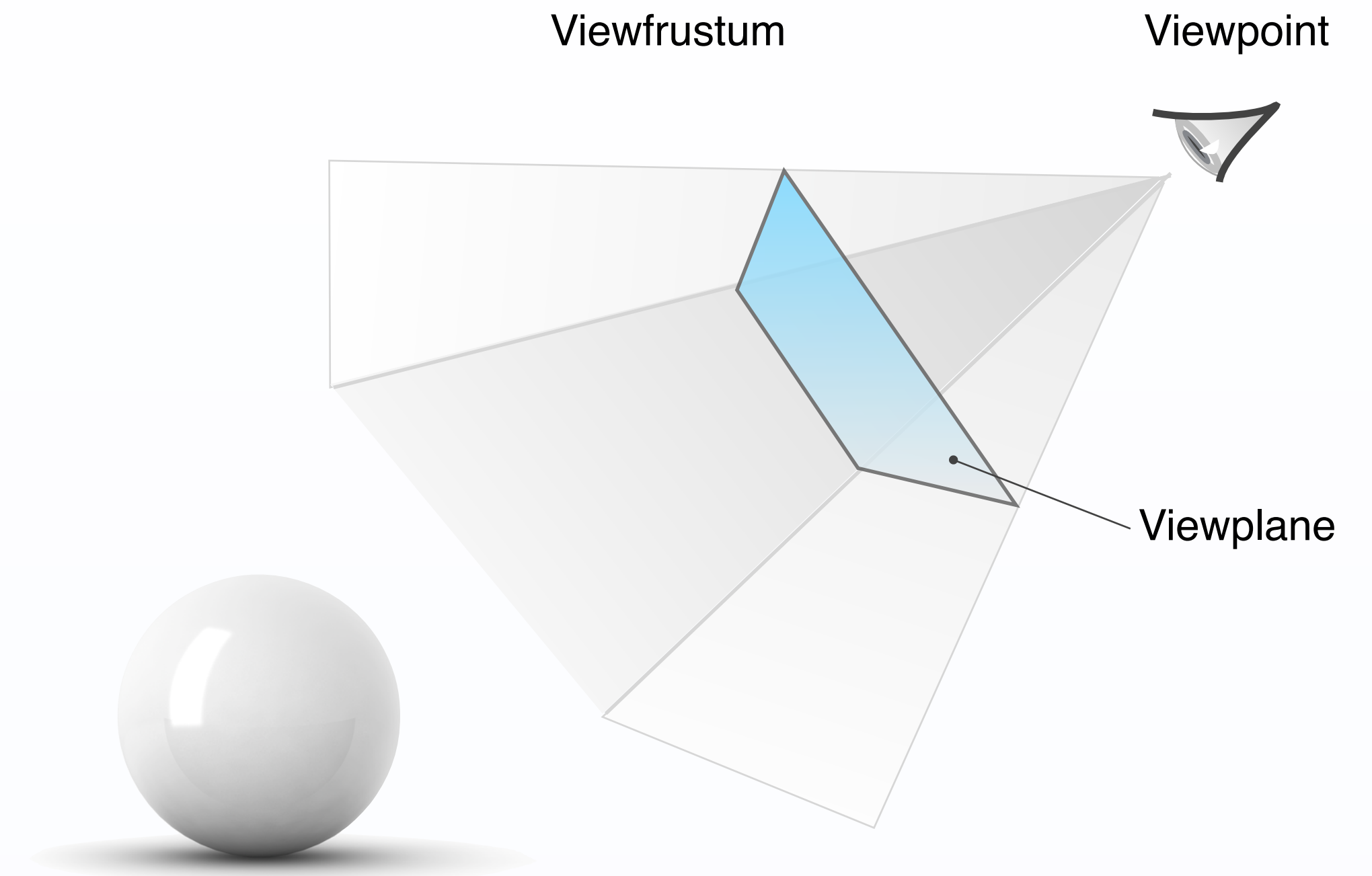




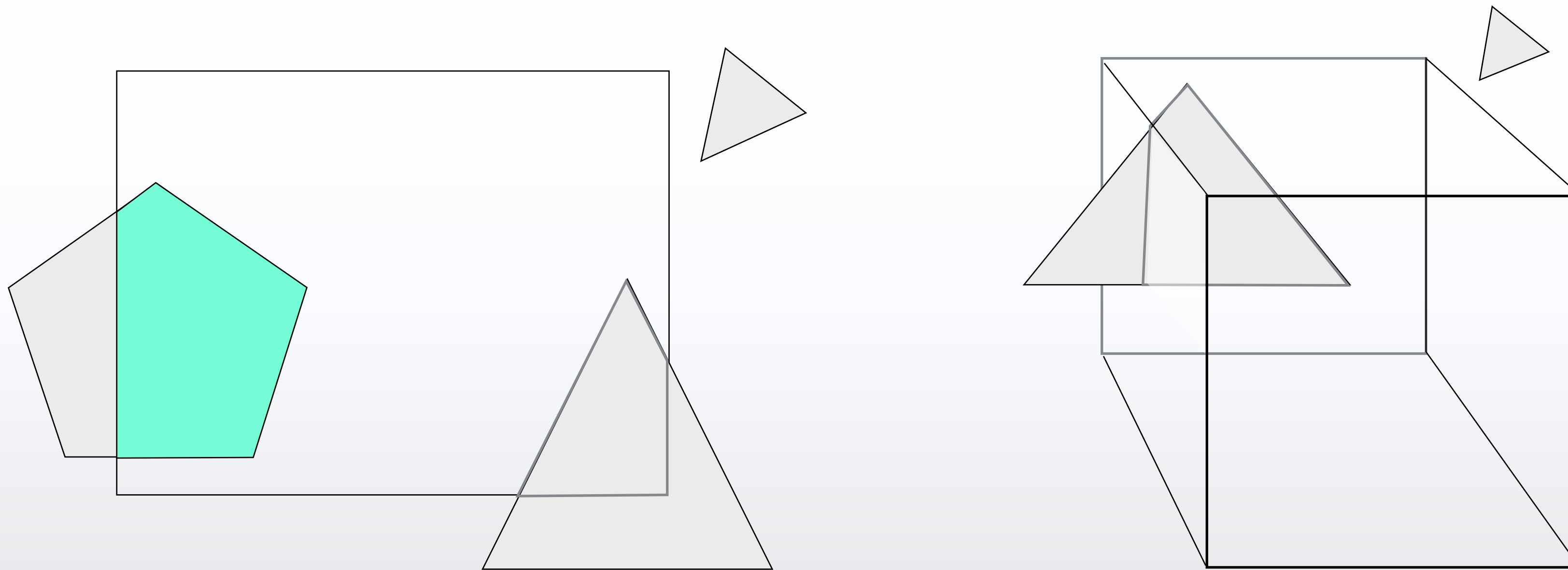


die Anfänge...

- Blickpunkt im globalem Koordinatensystem festlegen
- Kameraeigenschaften festlegen (Sichtpyramide)
 - (Bildformat, Öffnungswinkel,...)
- Sichtpyramide normieren
- Clipping / Culling
- Auf das Sichtfenster (2D) bringen
 - Tiefeninformation zur Verdeckung verwenden.

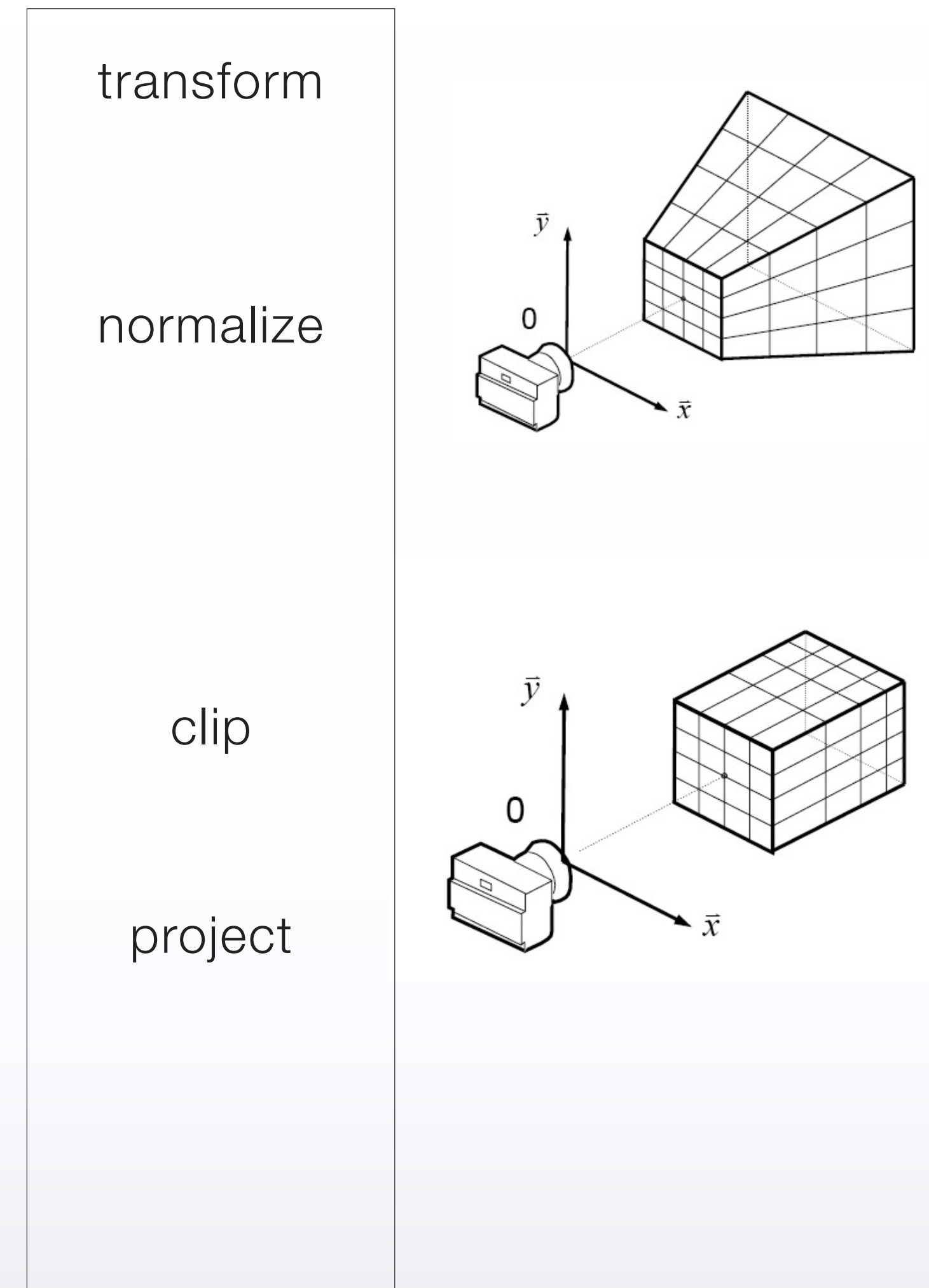


- Idee des Clipping in der Rendering Pipeline:
 - anstatt Clipping der 2D-Objekte am Viewplane (Sichtfenster),
 - Clipping der 3D-Objekte am Viewing-Frustum (Ansichtvolumen)
 - Erinnerung: Algorithmen gelten nur für achsenparallele Ebenen
 - Verfahren teuer-> Überlegung : so wenig wie möglich reinschicken

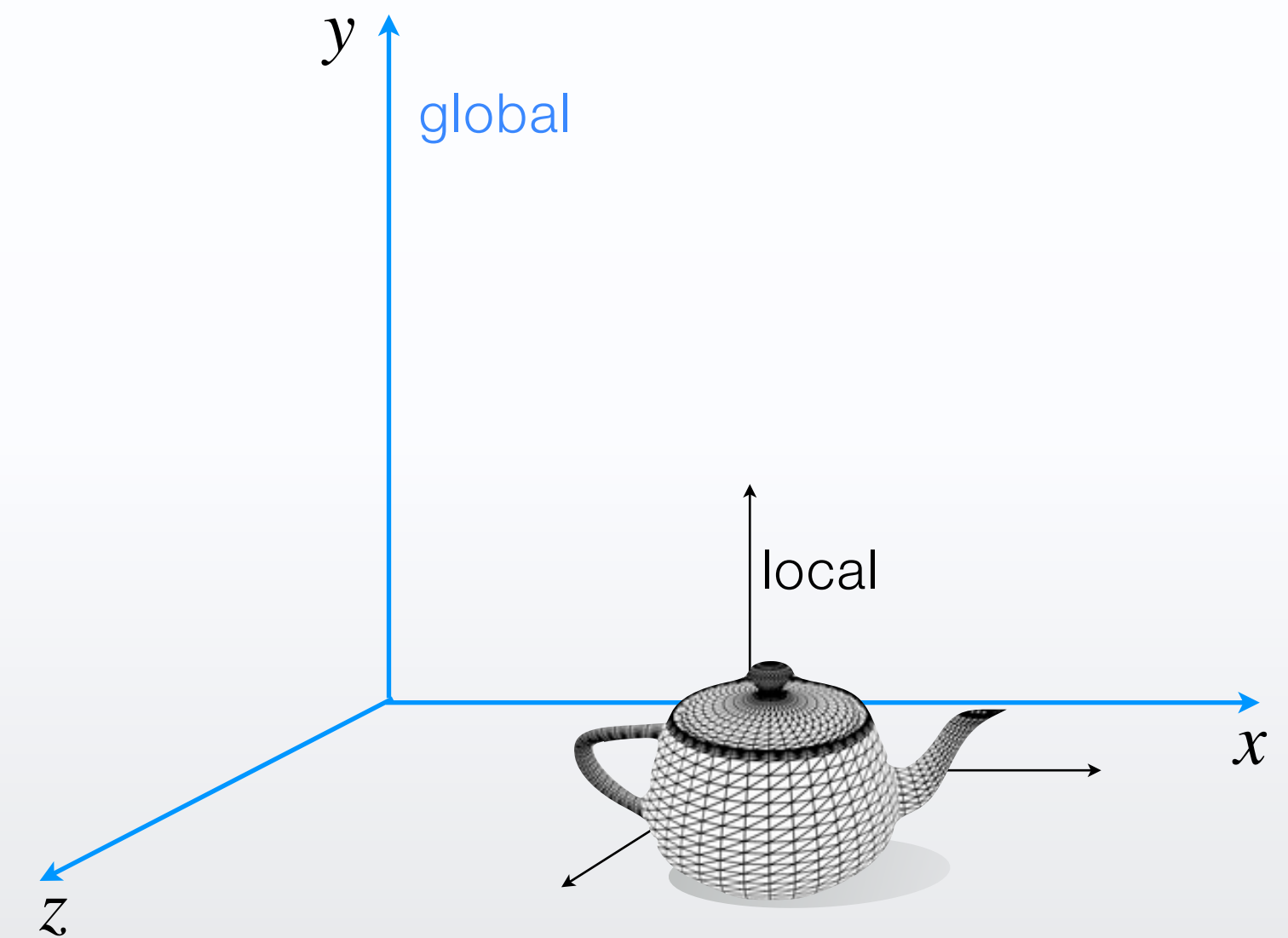
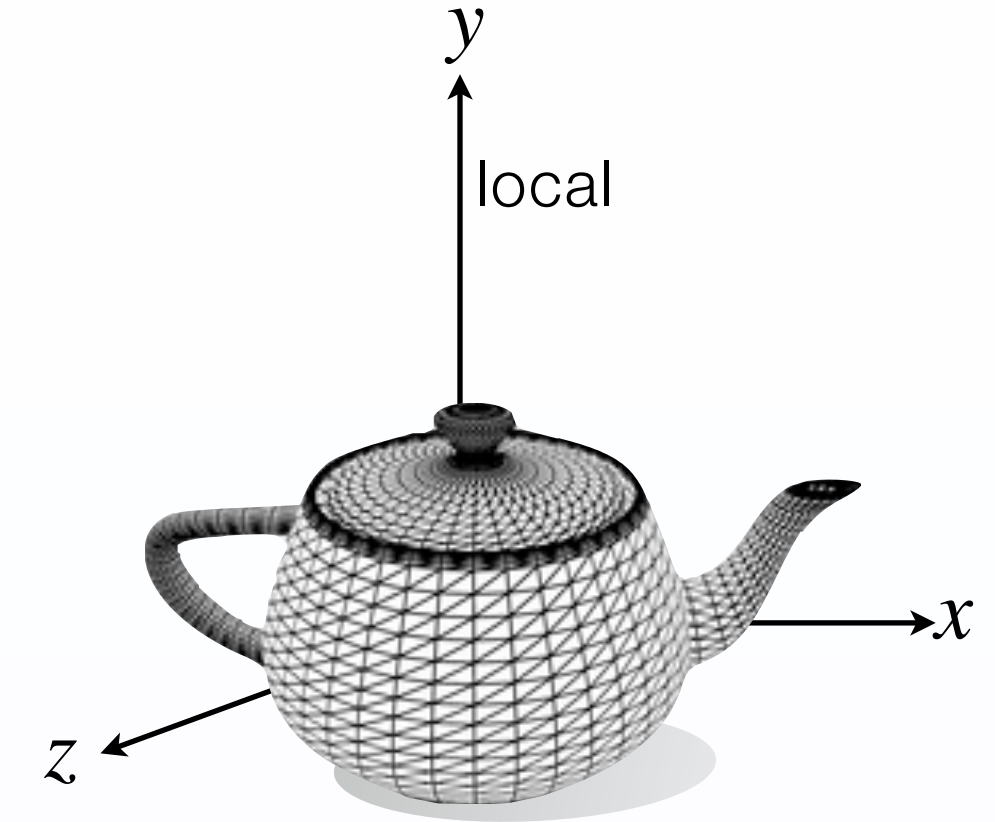


project transform, normalize & clip

- Beliebigen View Frustum vereinfachen
 - Zentriert am Ursprung /Achsenparallel / -z
- Quader in Würfel verformen und zentrieren
- Würfel normieren
- Culling
 - Faces mit der Normale nach hinten weglassen
- Clipping
 - Faces komplett oder zum Teil abschneiden
- Die Projektion
 - ist dann nur noch eine orthogonale (parallel) Projektion auf die vordere Ebene des Einheitsvolumen
 - Weglassen der Z-Koordinate



- Lokales (Object) Koordinatensystem
- x, y, z Koordinate im Bezug auf den „Schwerpunkt“, pivot Punkt, oder lokales Koordinatensystem
- Globales (World) Koordinatensystem
- Position im Bezug auf die Weltkoordinaten
- Wo steht das Objekt in der Szene



■ View (Kamera) Koordinatensystem

■ Kameradefinition

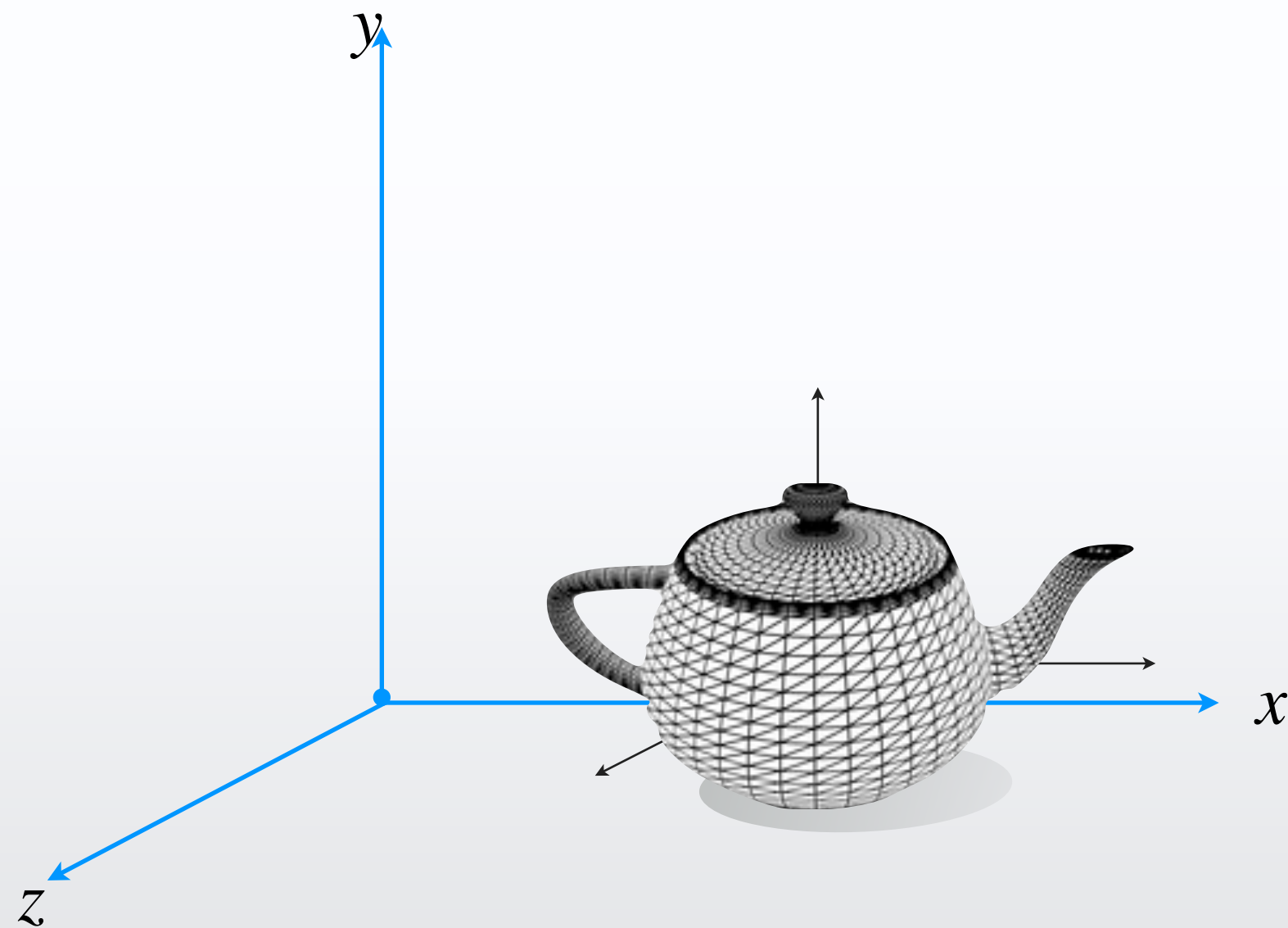
3 Fragen

?

Wo ist die Kamera ?

Wo guckt sie hin ?

Wo ist oben ?



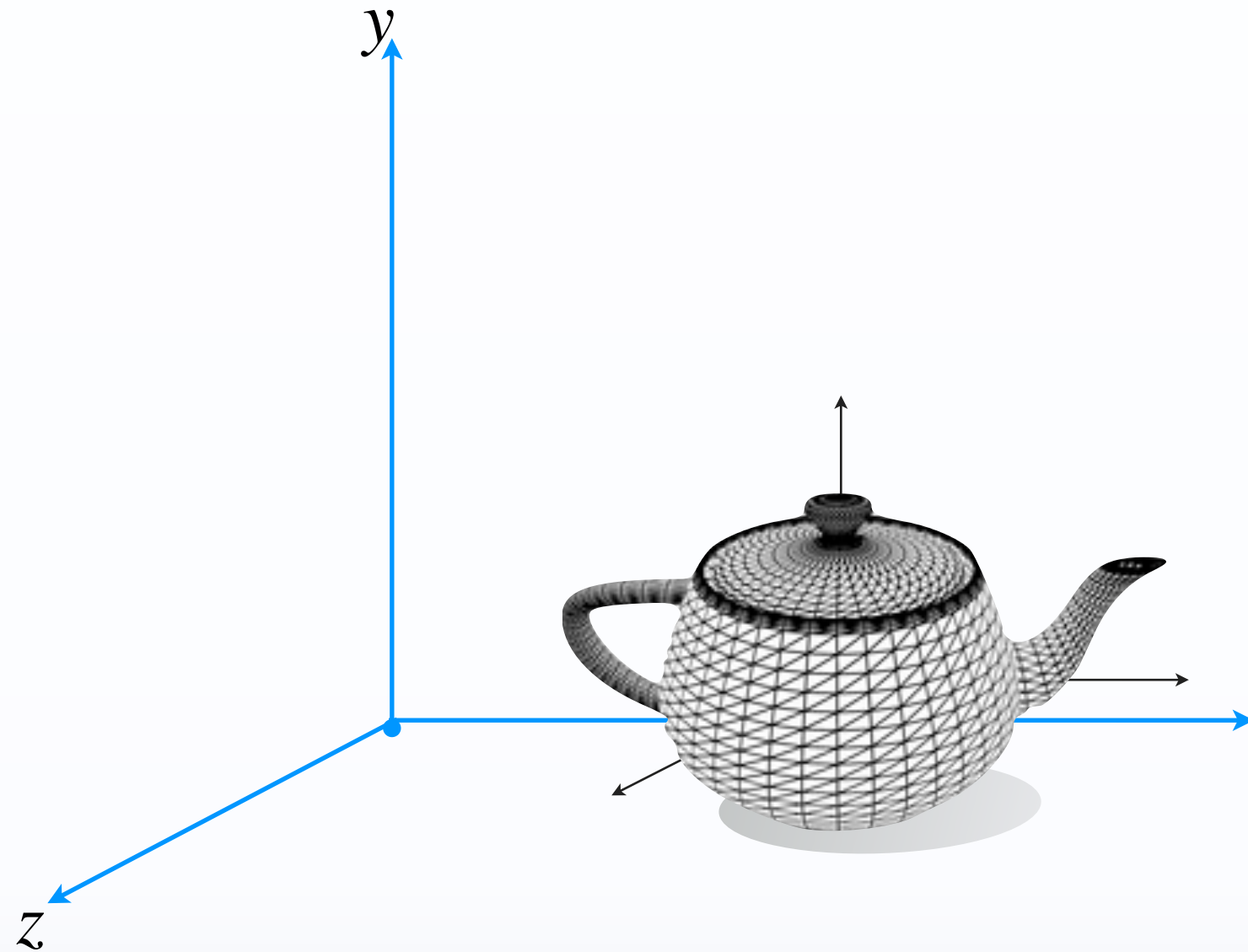
Aus diesen Parametern wird das Kamerakoordinatensystem „bauen“

■ View (Kamera) Koordinatensystem

■ Kameradefinition

3 Fragen

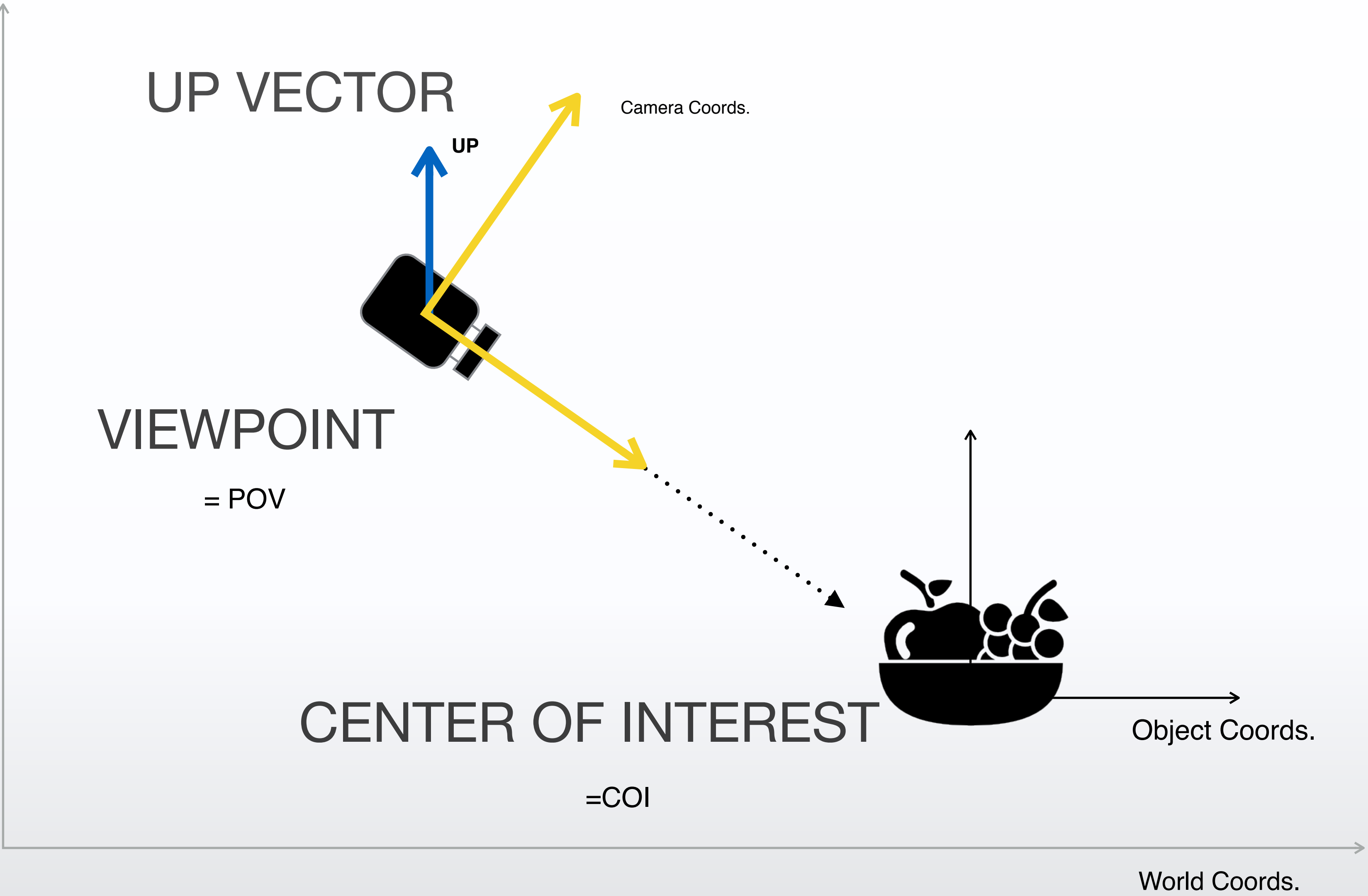
?

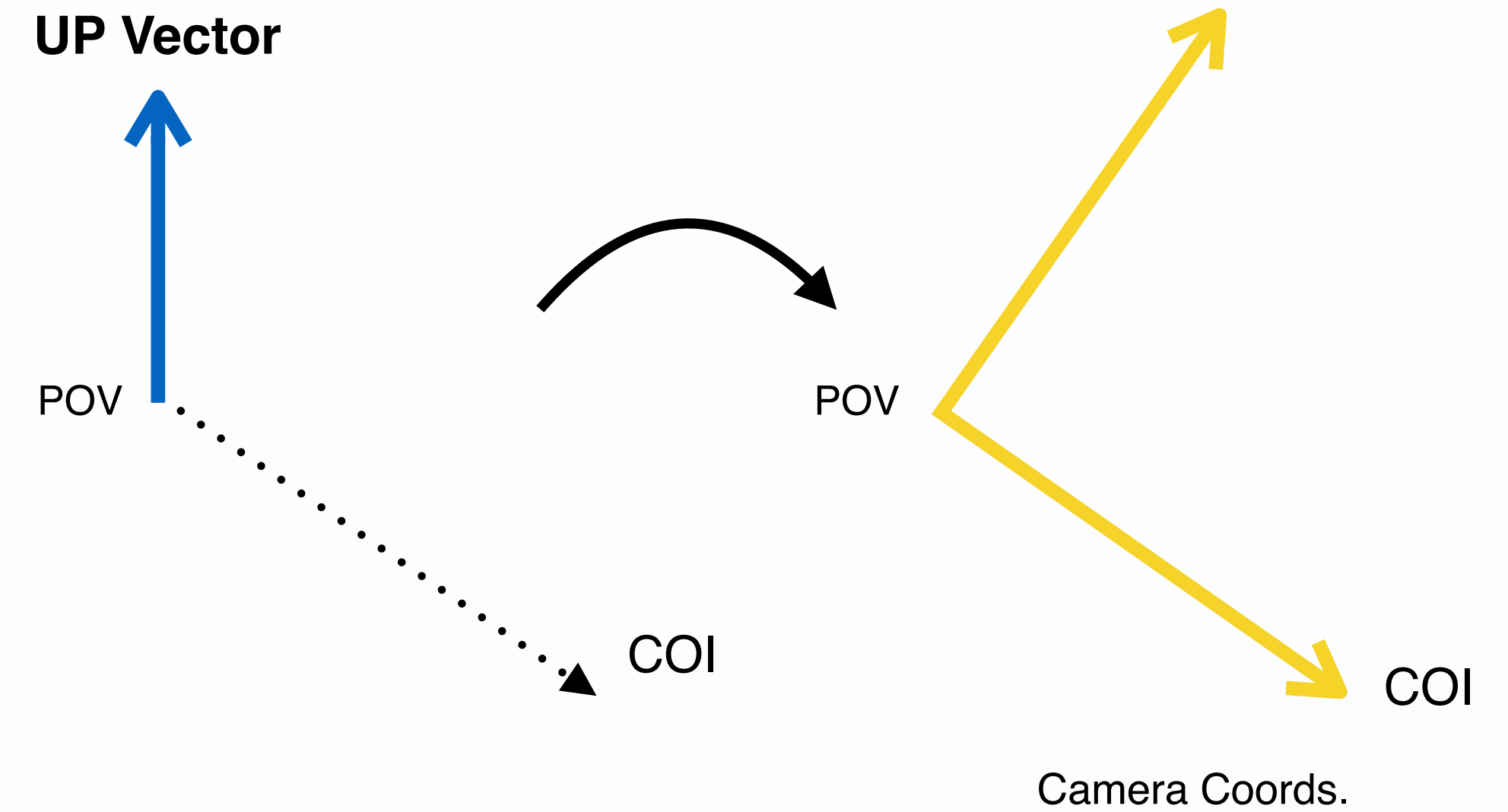
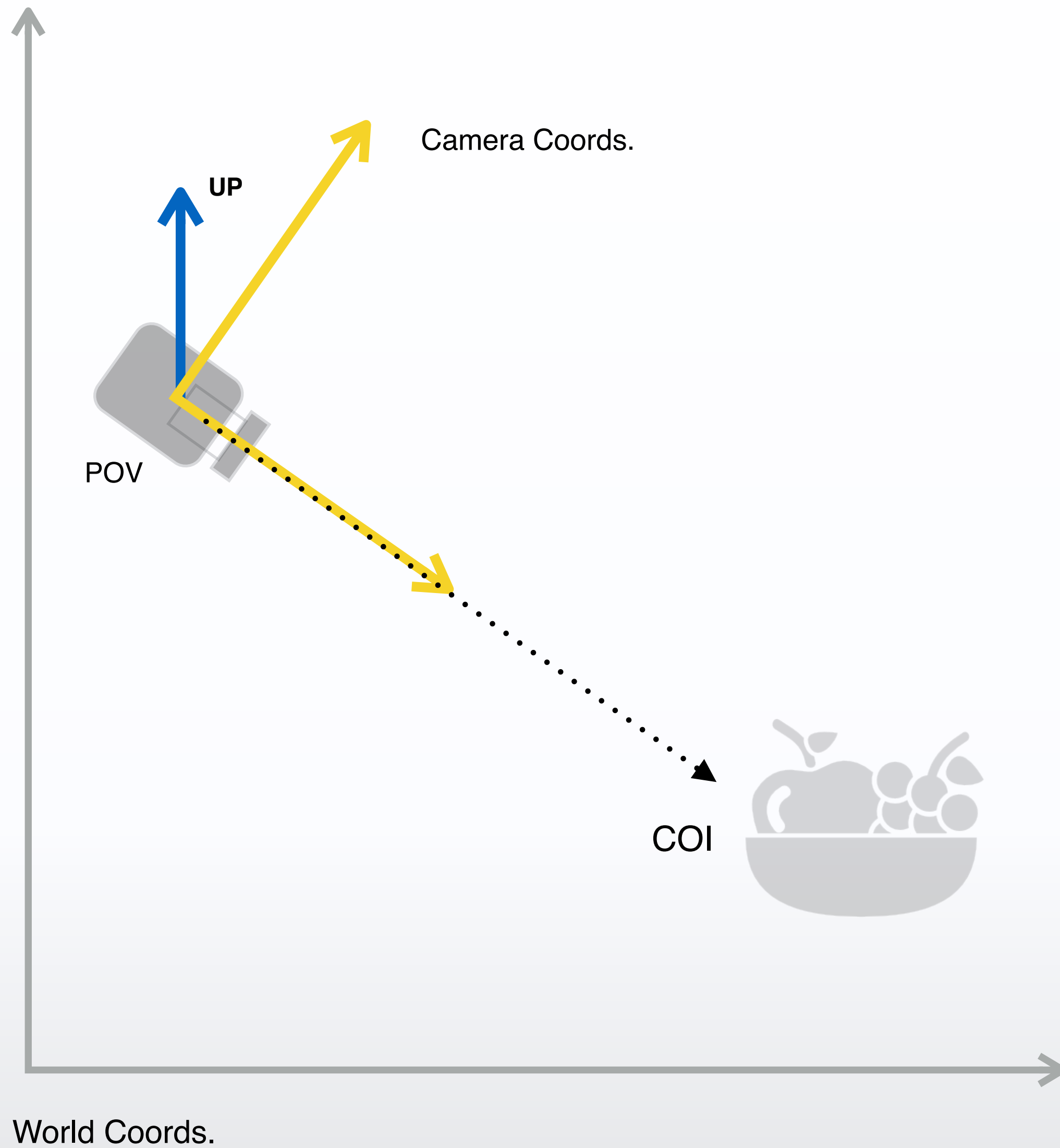


VIEWPOINT

CENTER OF INTEREST

UP VECTOR

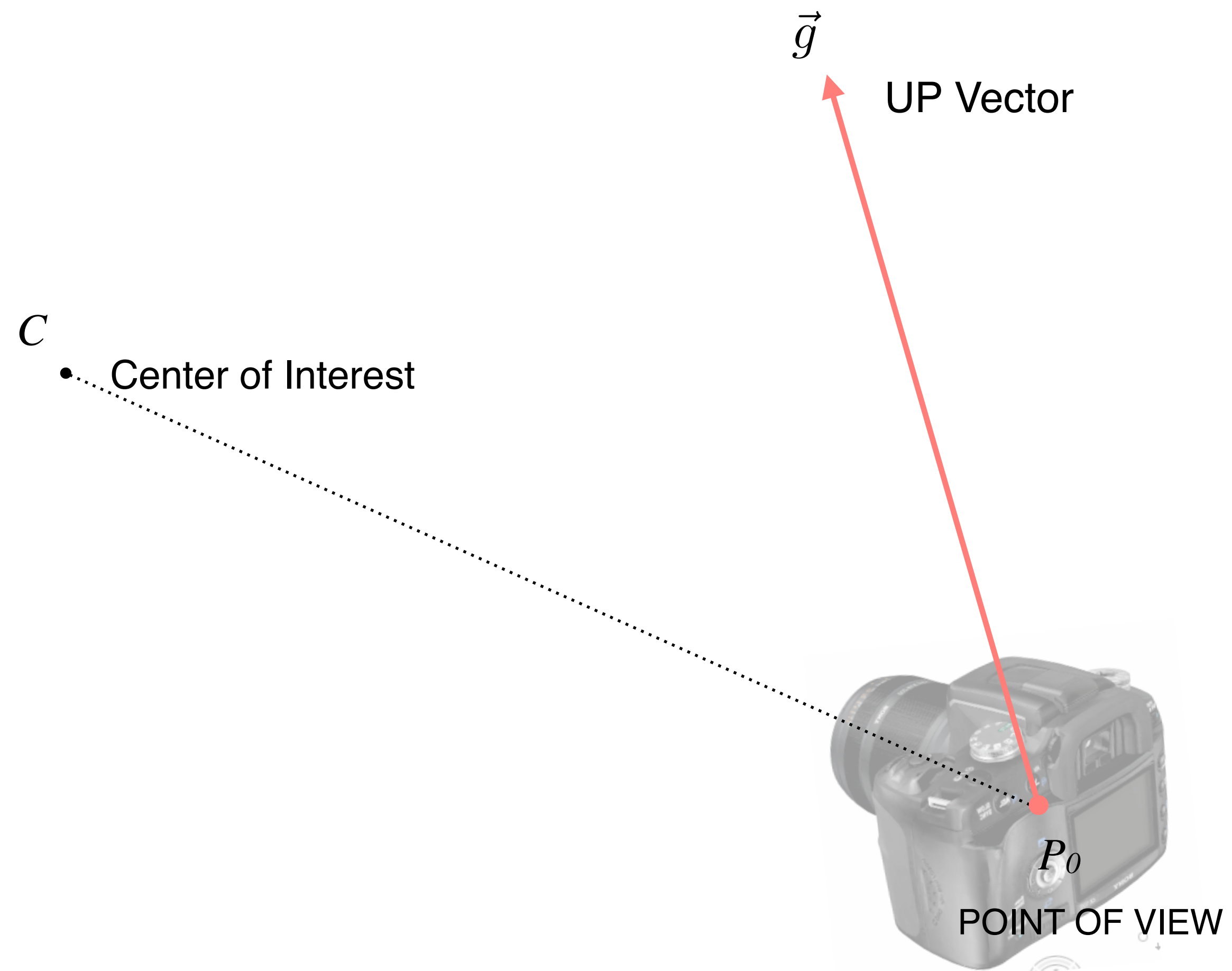


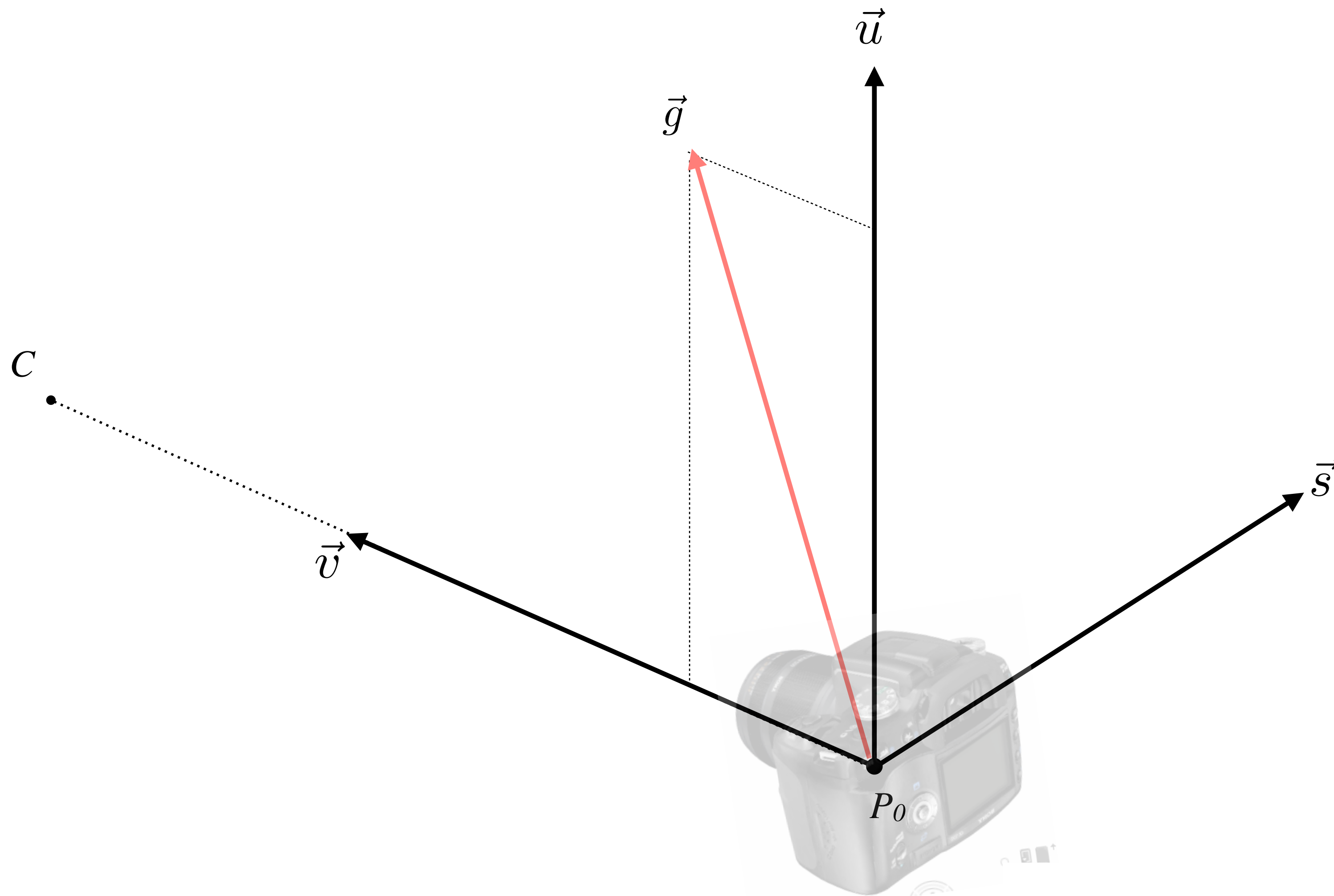


Kamerakoordinatensystem ist nicht orthogonal

Transformationen nur für orthogonale Systemen

Aus den Informationen ein orthogonales
Kamerakoordinatensystem konstruieren.





■ gegeben:

- Viewpoint: P_0
- Center of Interest C (*Center*)
- Up Vector $\vec{g}$

■ gesucht:

- Camera Coord.sys. $(\vec{v}, \vec{u}, \vec{s})$ (lokales Kamerakoord.system)

?

Wie erzeugen wir den View-Vektor?

$$\vec{v} = C - P_0$$

?

Wie erzeugen wir den Side-Vektor?

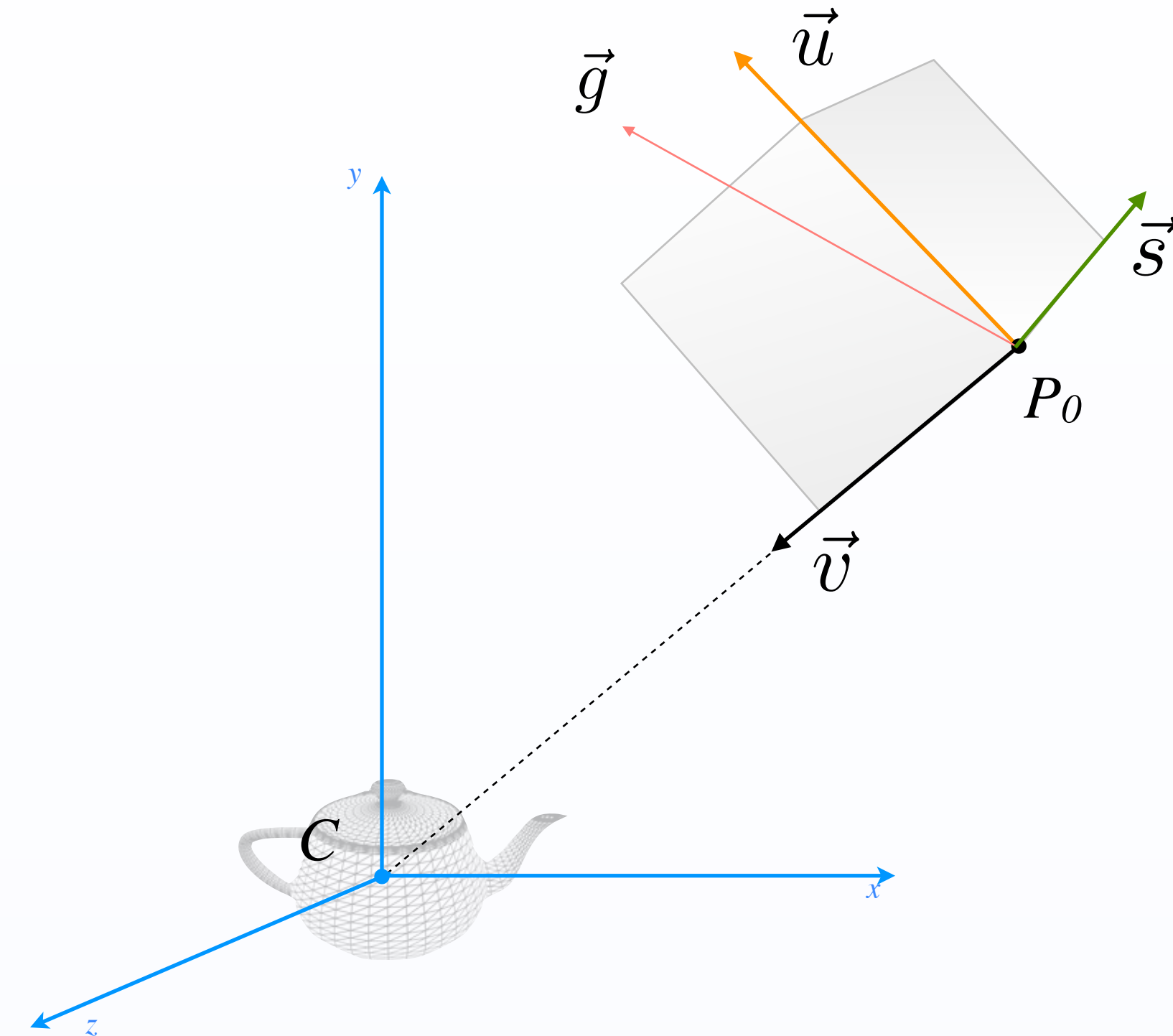
$$\vec{s} = \vec{v} \times \vec{g}$$

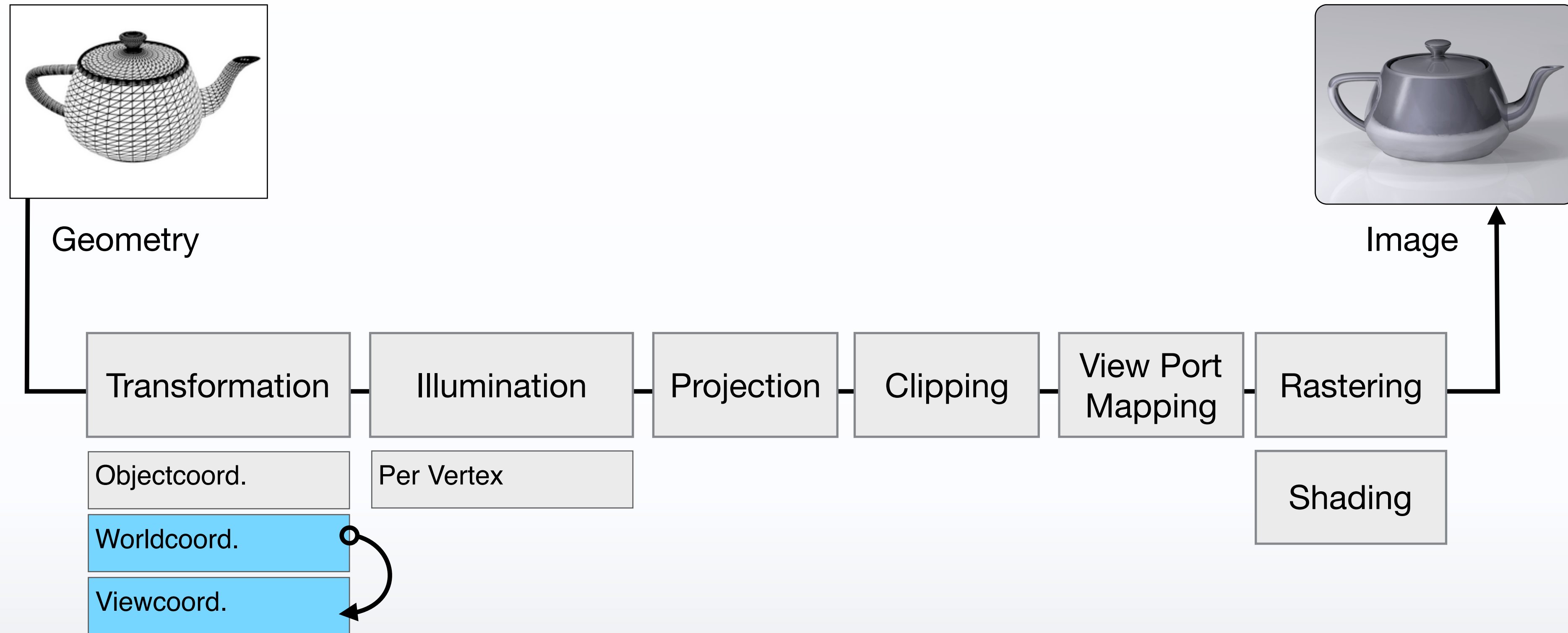
?

Wie erzeugen wir den neuen Up Vektor

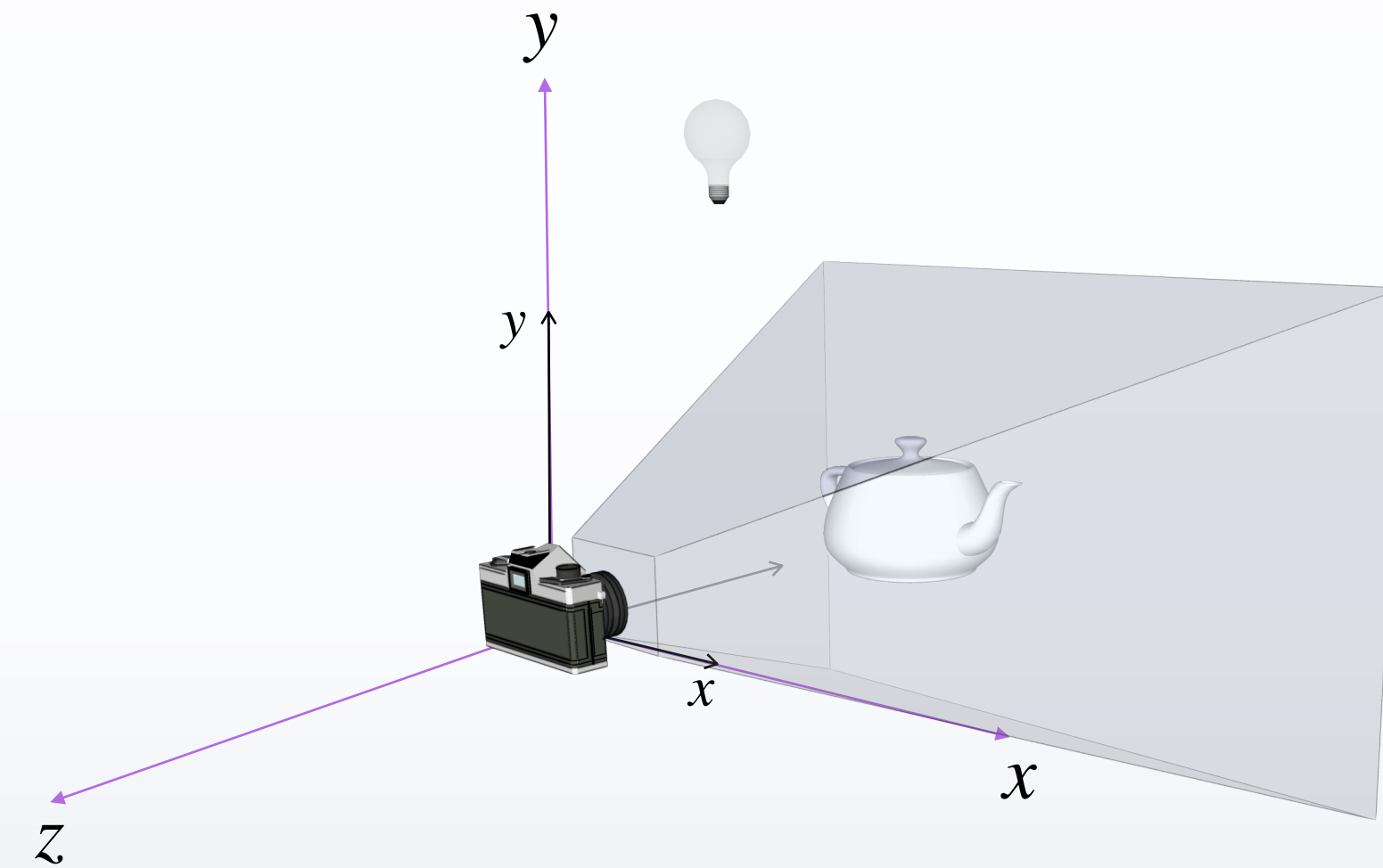
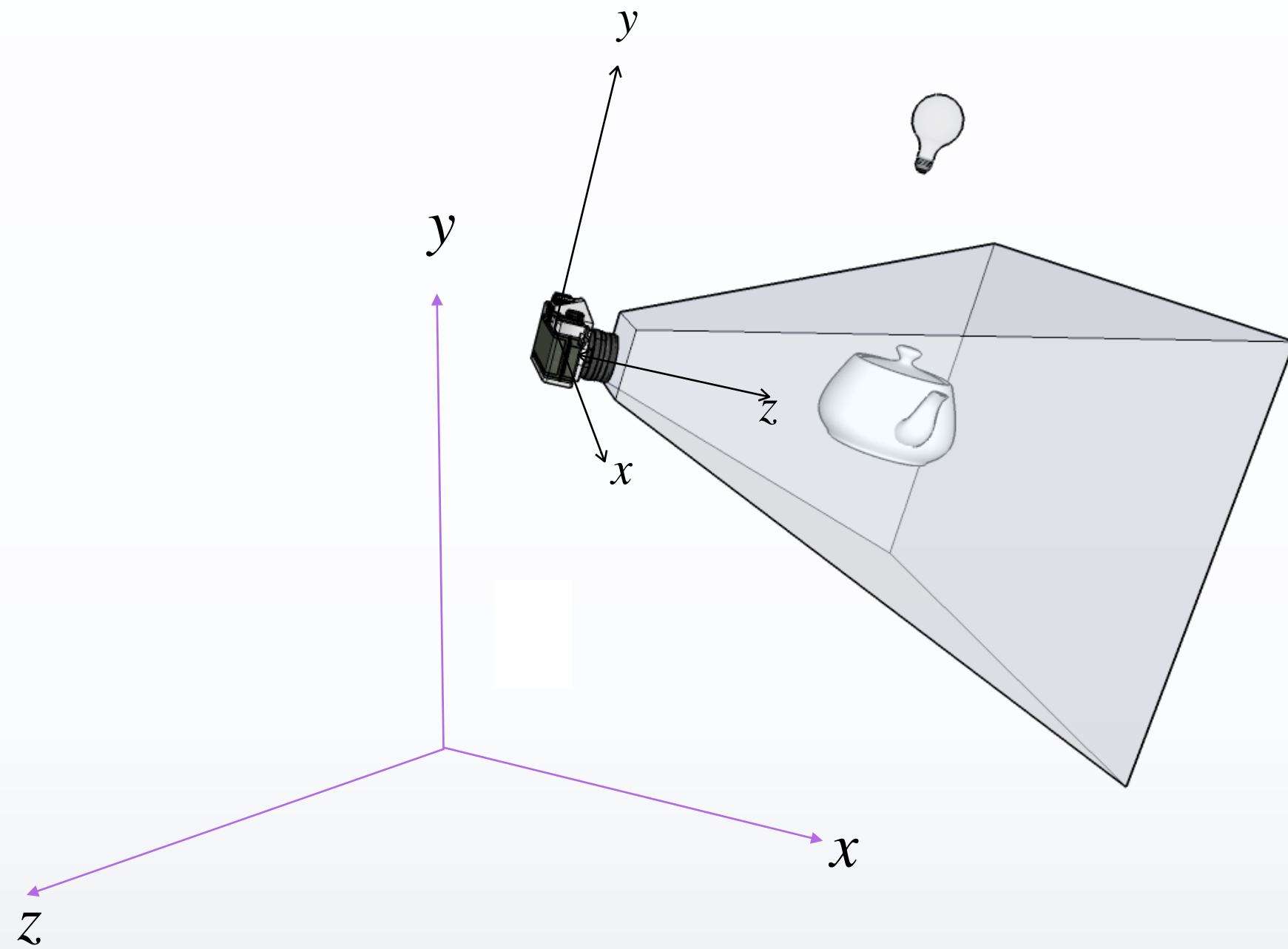
$$\vec{u} = \vec{s} \times \vec{v}$$

Alle Vektoren normieren

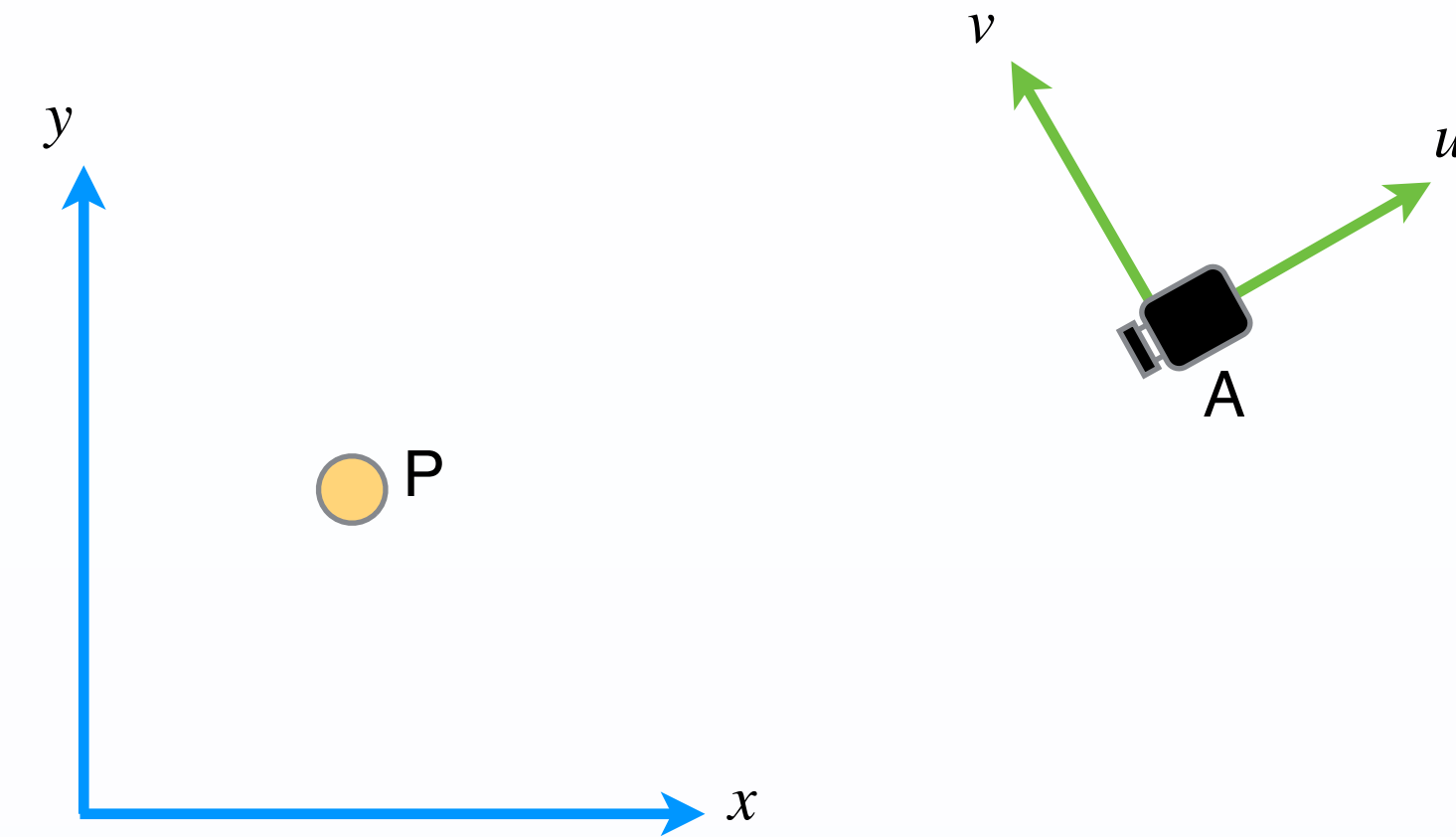




WORLD CAMERA

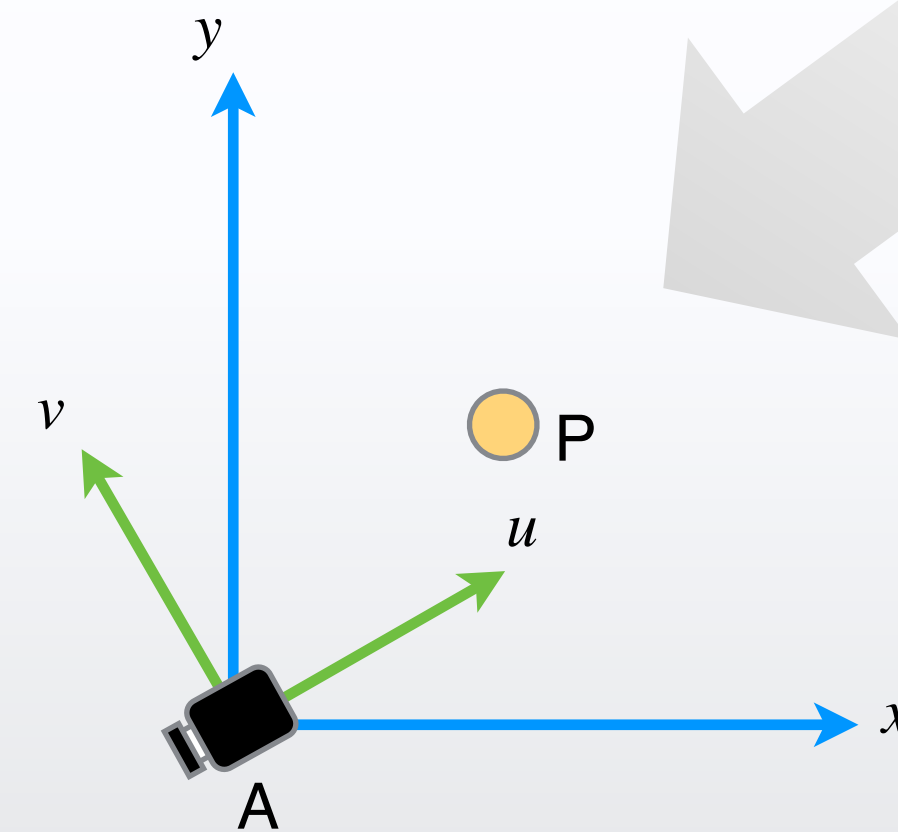


- Wie kommt man in das Kamerakoordinatensystem?
- Gegeben: $P(x,y)$
- Gesucht: $P(u,v)$



Translation in den Ursprung

$$T = \begin{pmatrix} 1 & 0 & -A_x \\ 0 & 1 & -A_y \\ 0 & 0 & 1 \end{pmatrix}$$



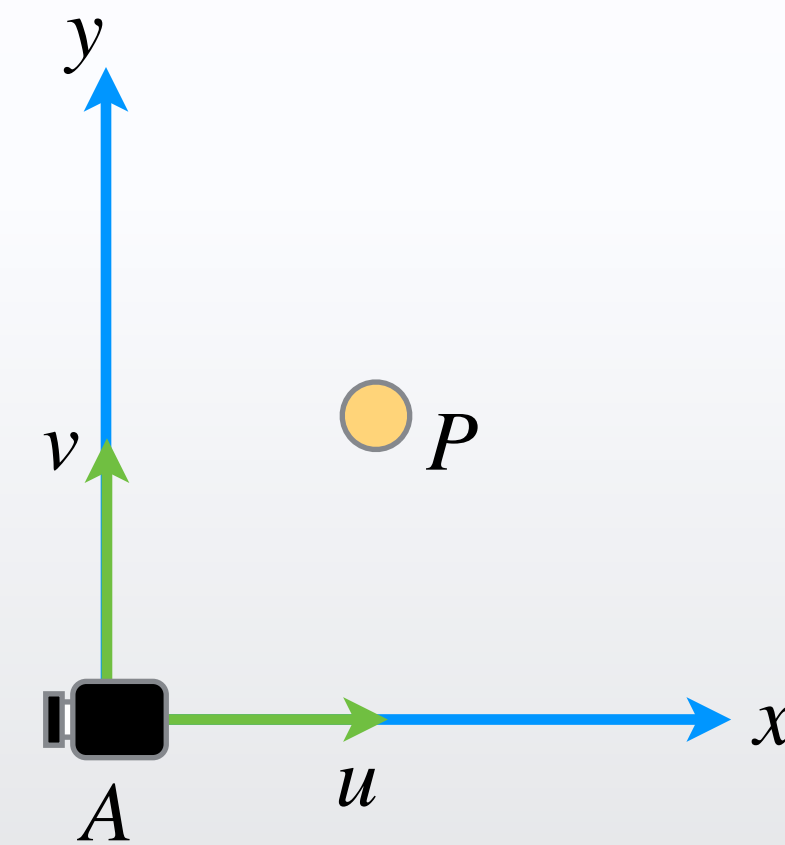
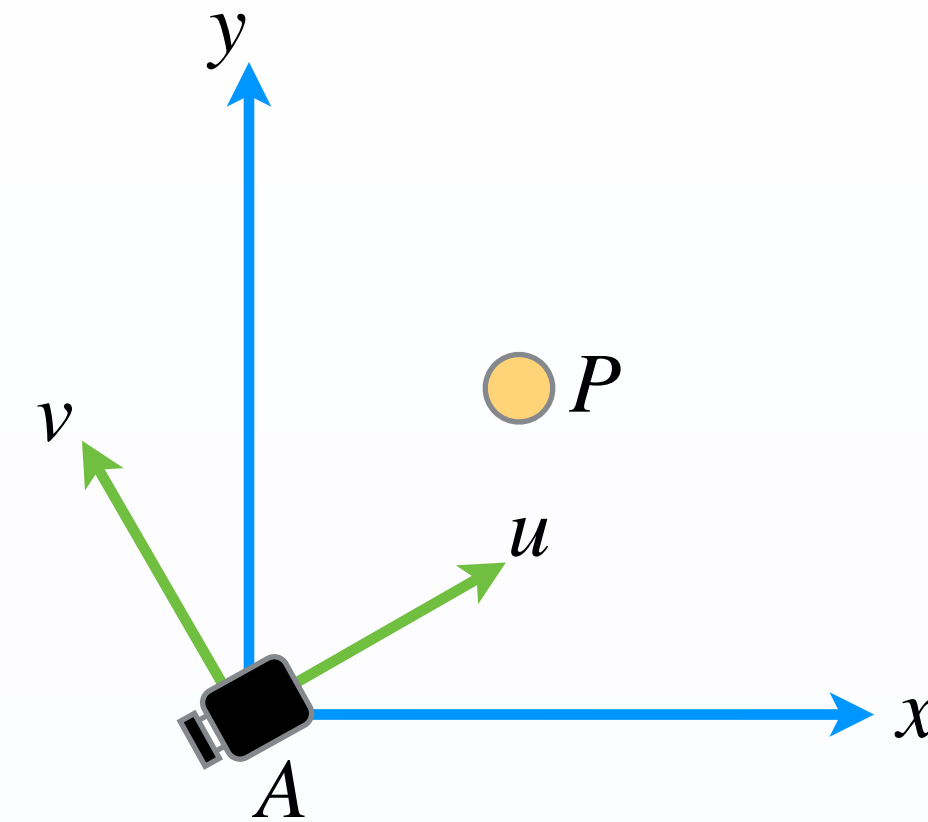
Translation in den Ursprung

$$T = \begin{pmatrix} 1 & 0 & -A_x \\ 0 & 1 & -A_y \\ 0 & 0 & 1 \end{pmatrix}$$

Rotation, so dass die Achsen übereinstimmen

$$R = \begin{pmatrix} u_x & u_y & 0 \\ v_x & v_y & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad \text{? wie werden die Winkel berechnet?}$$

$$M = \begin{pmatrix} u_x & u_y & 0 \\ v_x & v_y & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & -A_x \\ 0 & 1 & -A_y \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} u_x & u_y & -A_x u_x - A_y u_y \\ v_x & v_y & -A_x v_x - A_y v_y \\ 0 & 0 & 1 \end{pmatrix}$$



- Tatsächlich wird die gesamte Szene mit der Kamera transformiert.

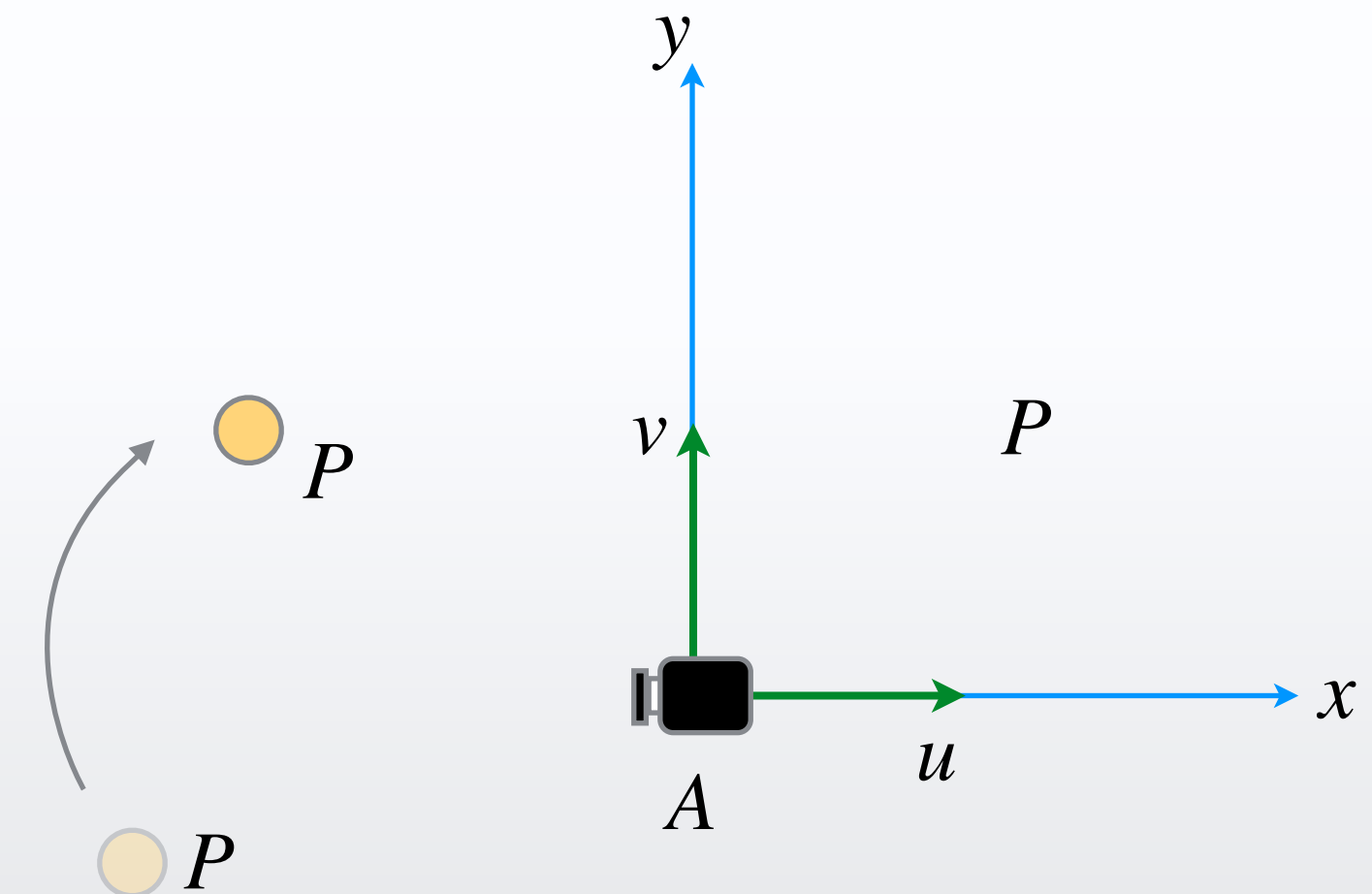
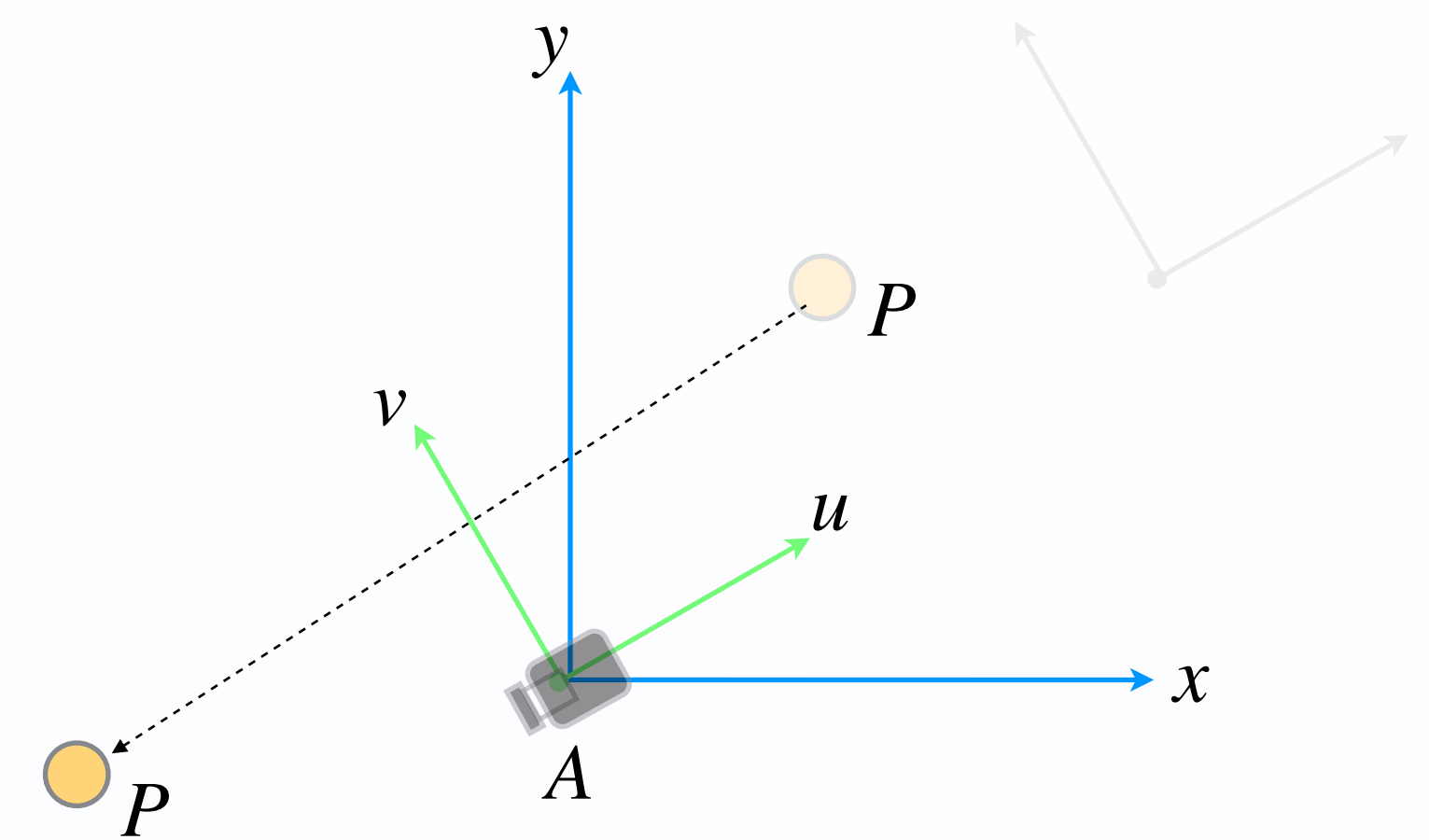
$$T = \begin{pmatrix} 1 & 0 & -A_x \\ 0 & 1 & -A_y \\ 0 & 0 & 1 \end{pmatrix}$$

- Rotation, so dass die Achsen übereinstimmen.

$$R = \begin{pmatrix} u_x & u_y & 0 \\ v_x & v_y & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

- Wir bestimmen die spezielle Projektionsmatrix gültig für

- Im Ursprung
- Achsenparallel
- in Richtung der negativen Z-Achse



Wozu das Ganze ?

A) Formel der völlig freien Projektion zu aufwendig. (folgt später)

Voraussetzungen schaffen um die Formel zu vereinfachen

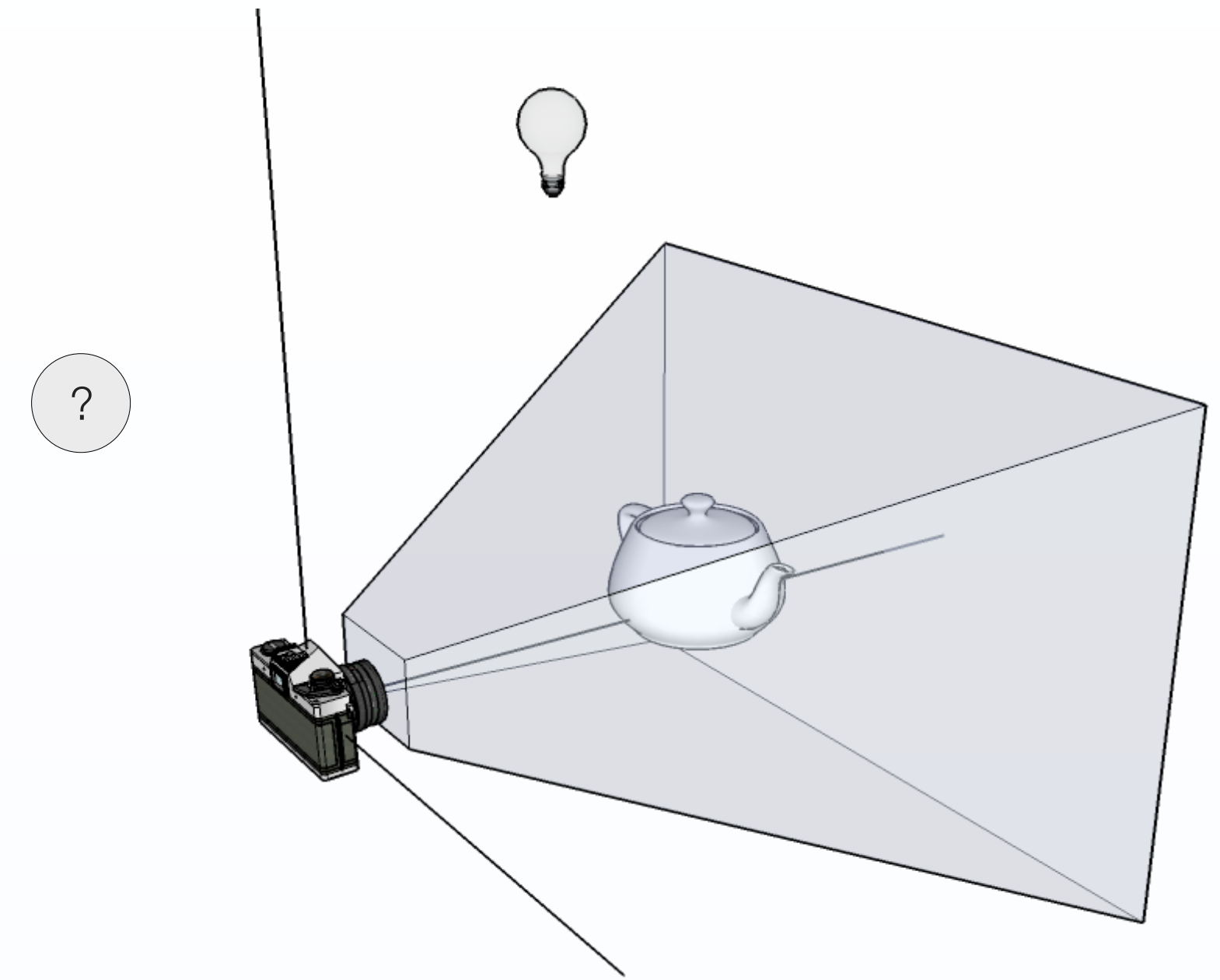
1. Kamera ist im Nullpunkt
2. Kamera ist achsenparallel ausgerichtet
3. Kamera blickt in Richtung der negative Z-Achse

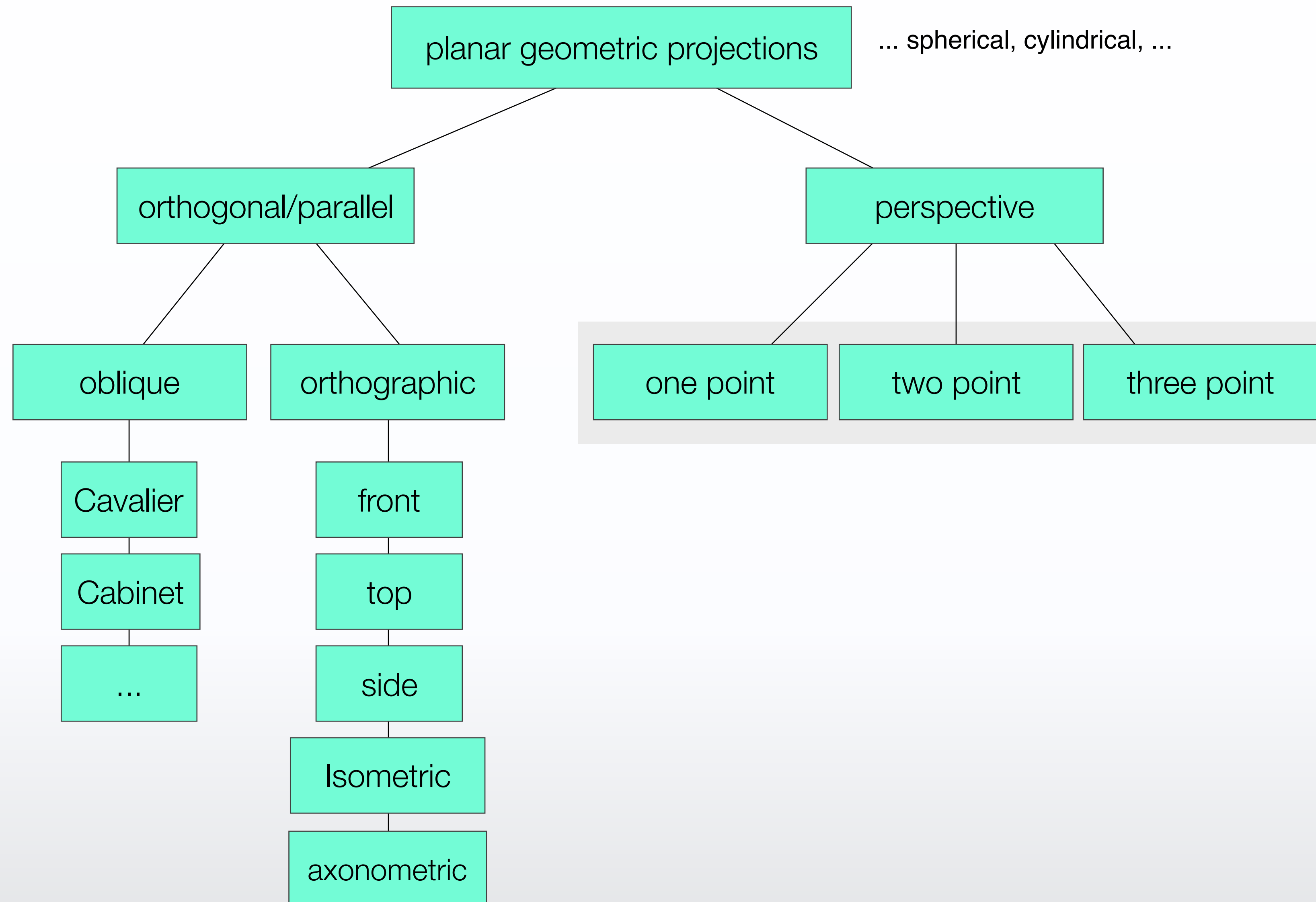
B) Clipping funktioniert nur mit achsenparallelen Ebenen.

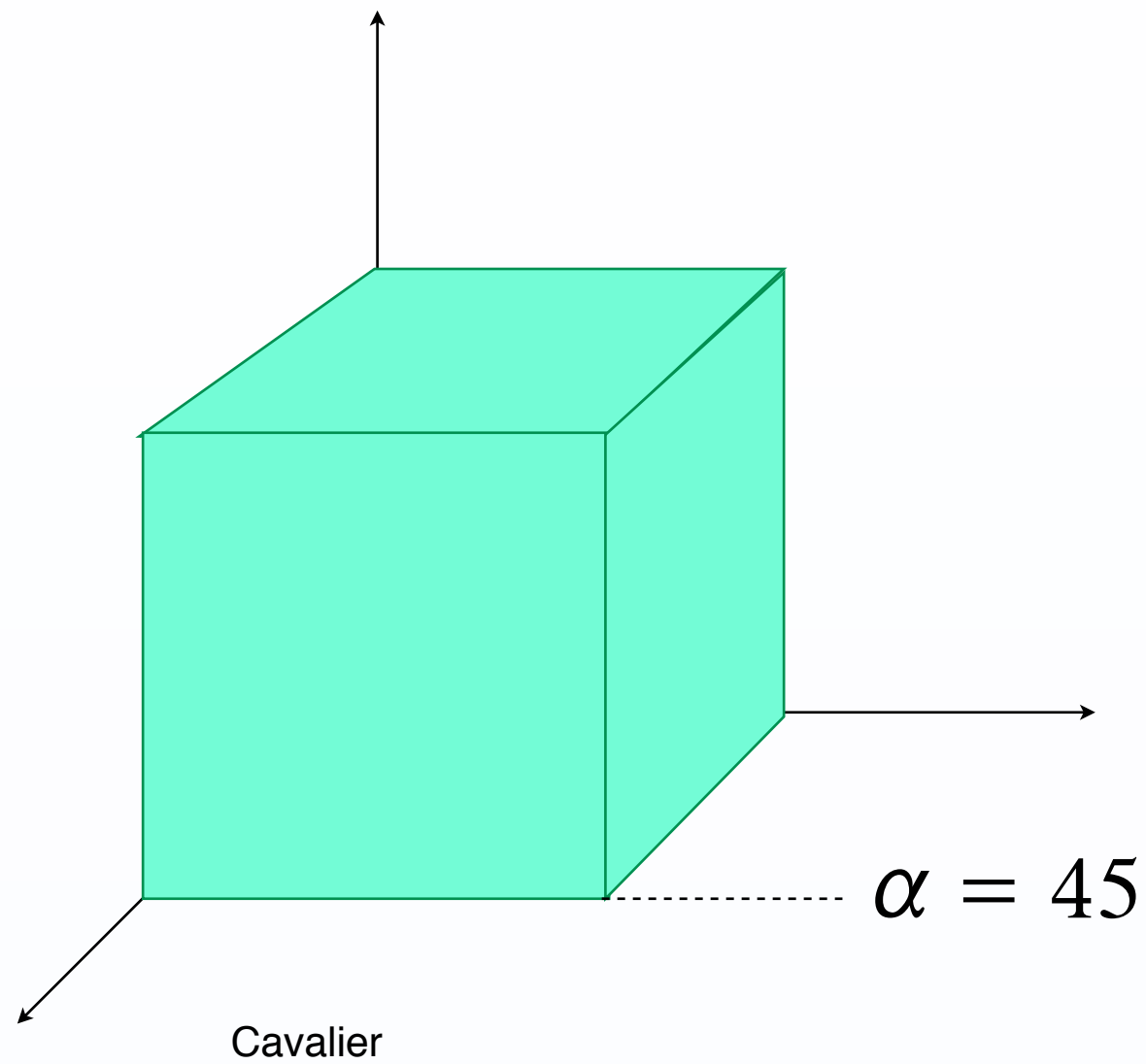
C) BackFaceCulling ist viel einfacher (*nur Vorzeichen z-Achse*)

Wo befinden wir uns ?

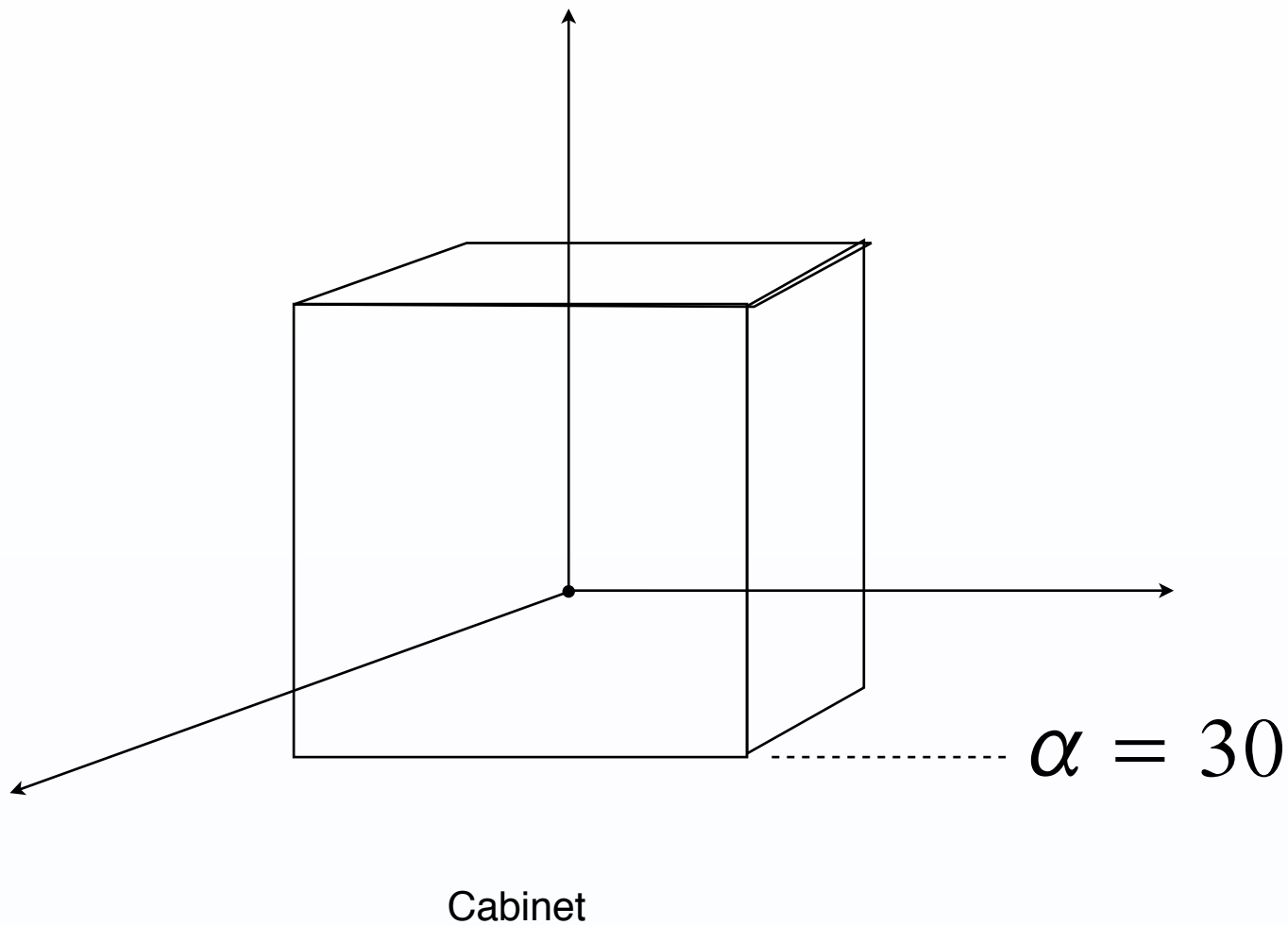
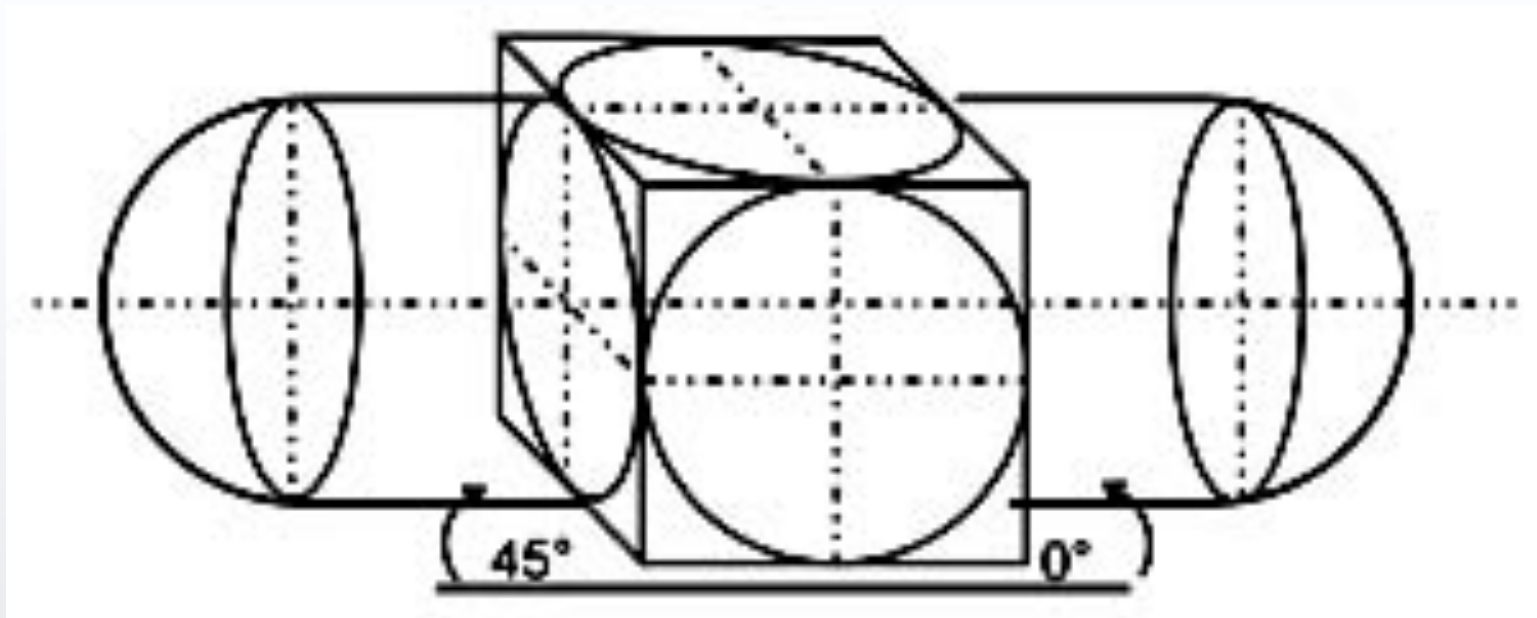
- Alle Objekt Transformationen sind durchgeführt
- Objekt ist im Worldmatrix transformiert
- View Koordinaten werden festgelegt mit
- `gluLookAt (0.0, 0.0, 2.0, // point of view
 0.0, 0.0, 0.0, // center of interest
 0.0, 1.0, 0.0) // up vector`
- Transformation in Viewmatrix
- $p' = V \cdot (T \cdot S \cdot R \cdot T \cdot \dots \cdot R) \cdot p$ Viewmatrix
- An dieser Stelle wird die Beleuchtung berechnet.
- Danach erfolgt die Projektion
- und damit die Frage welche Projektionsart eingesetzt wird



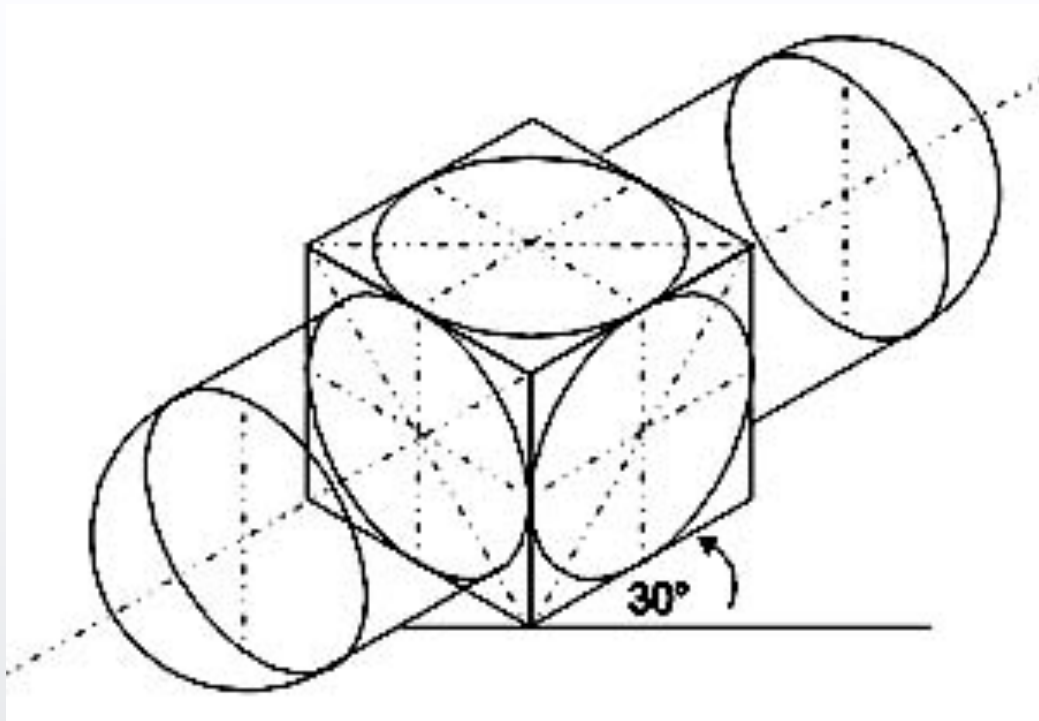




Längen bleiben erhalten

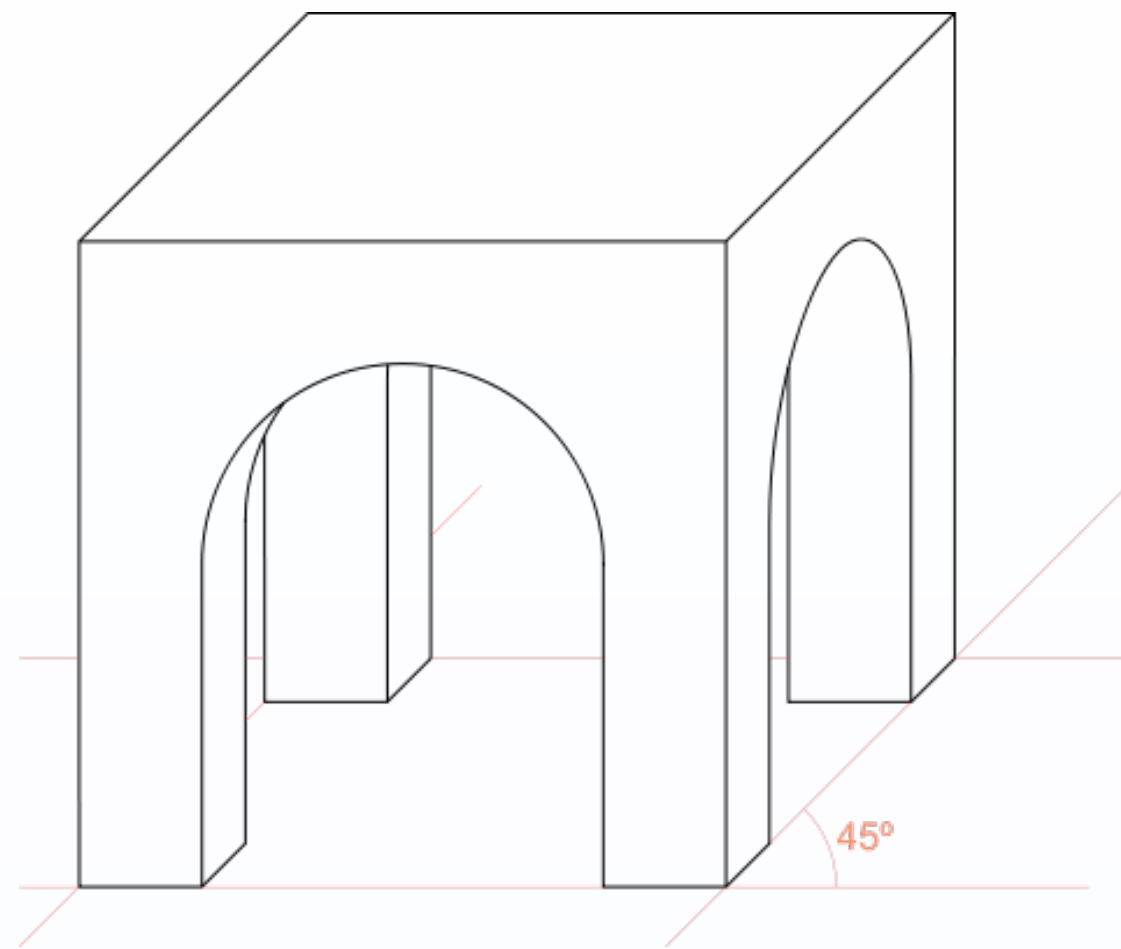


Längen sind um die Hälfte verkürzt

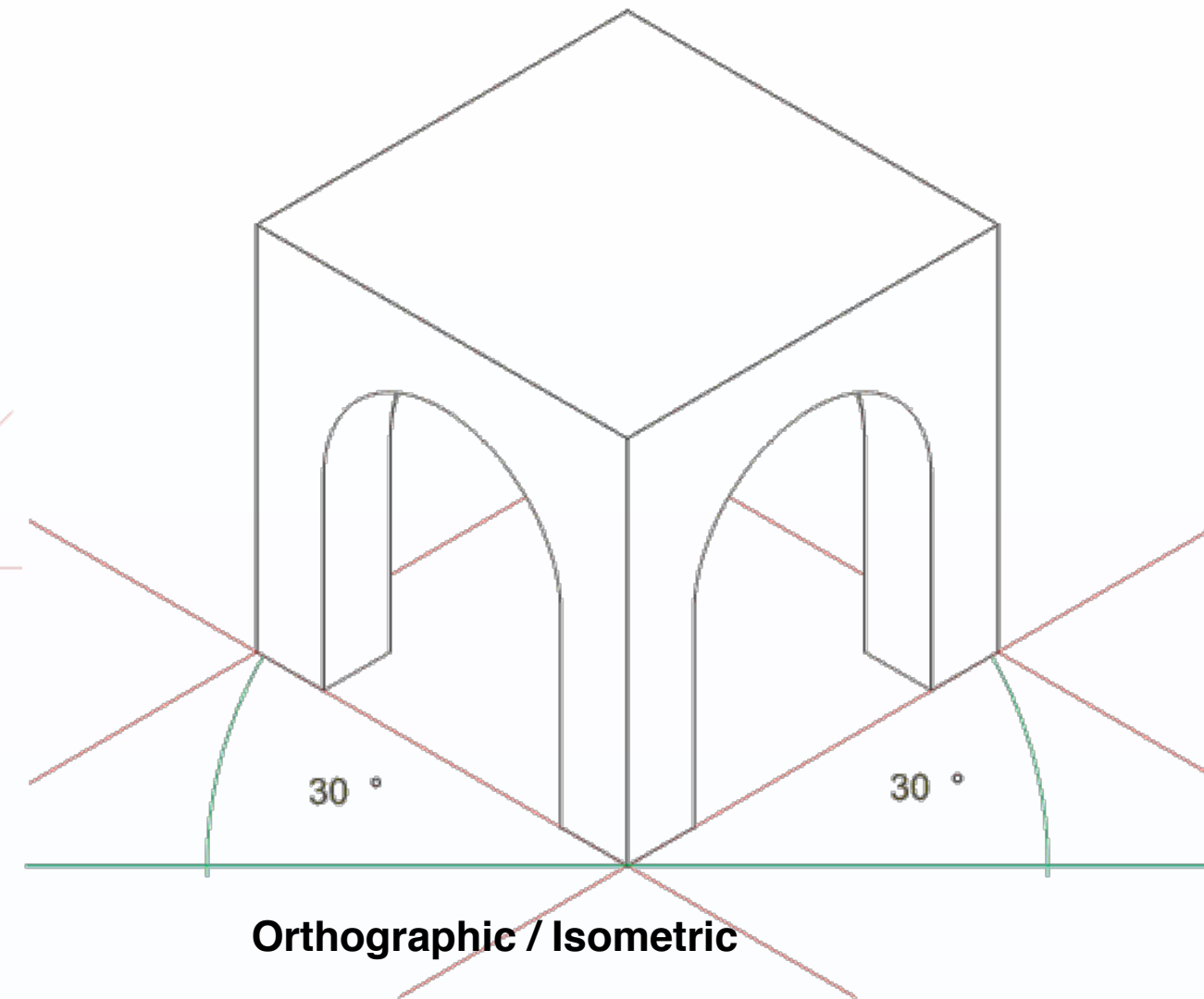




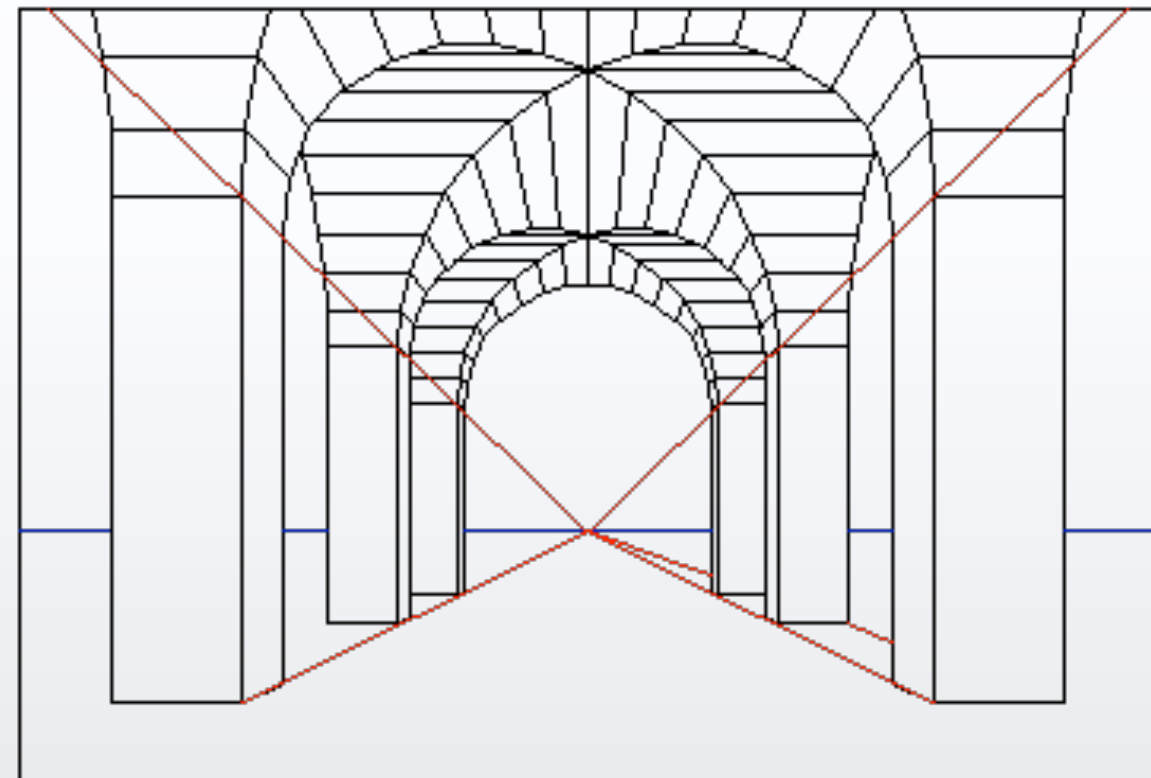
Planar Geometric Projection



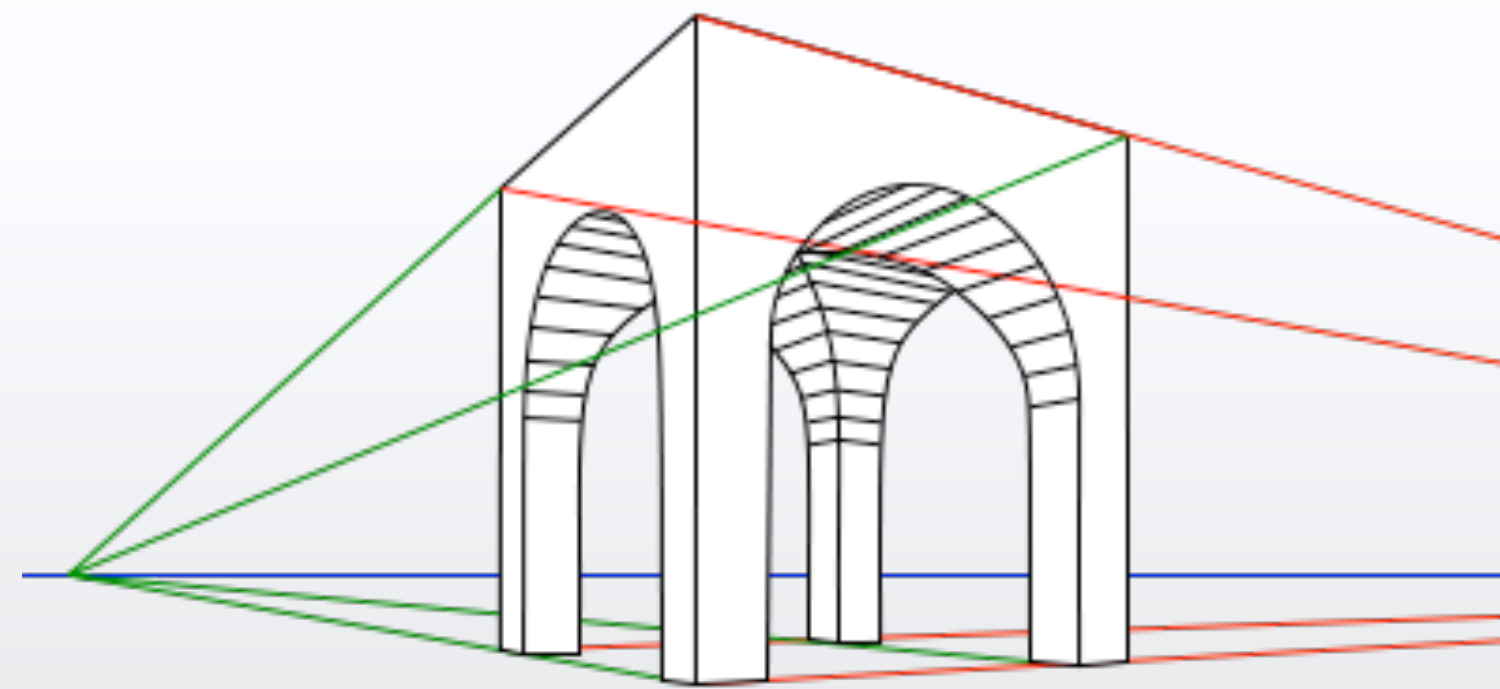
Oblique / Cavalier



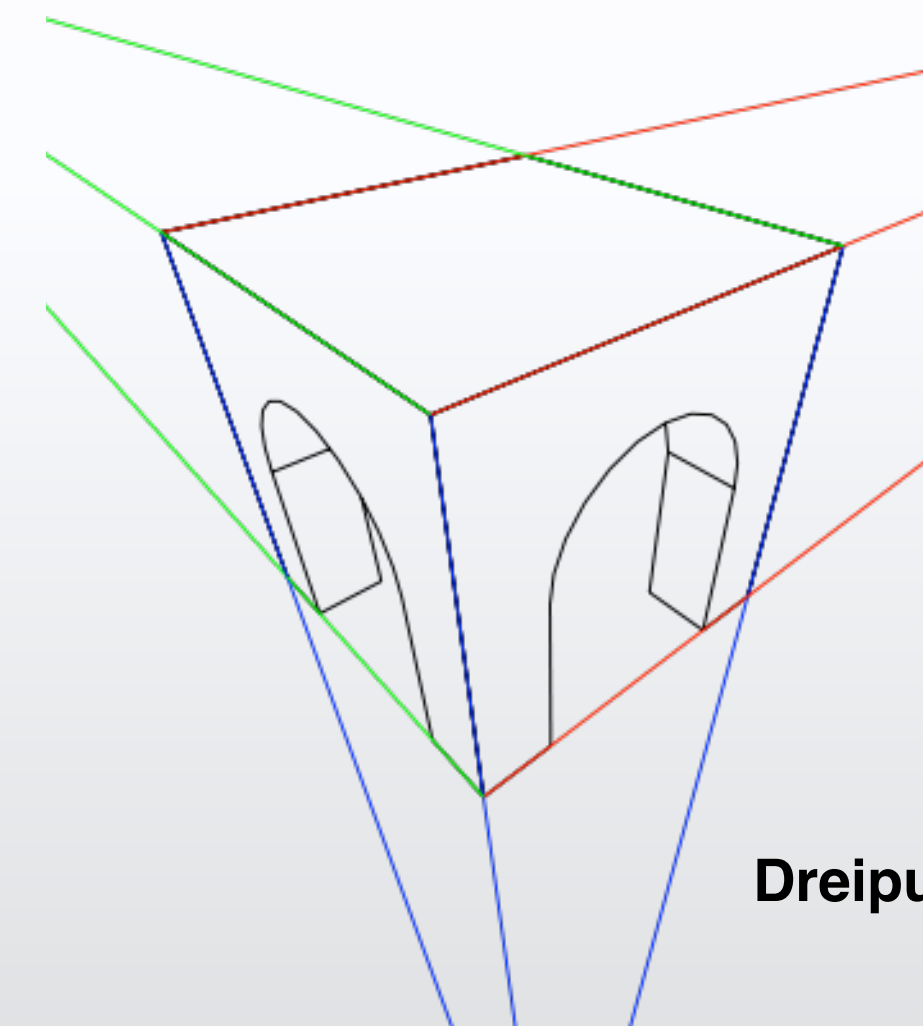
Orthographic / Isometric



Perspektive (Einpunkt / Zentral)



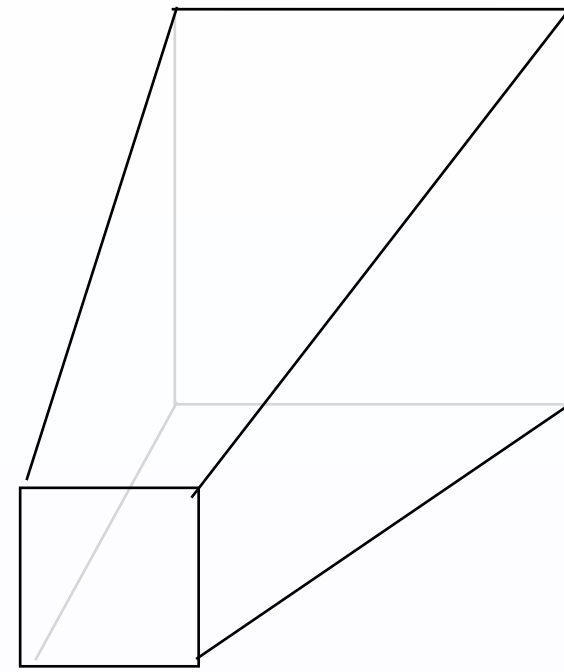
Zweipunkt



Dreipunkt

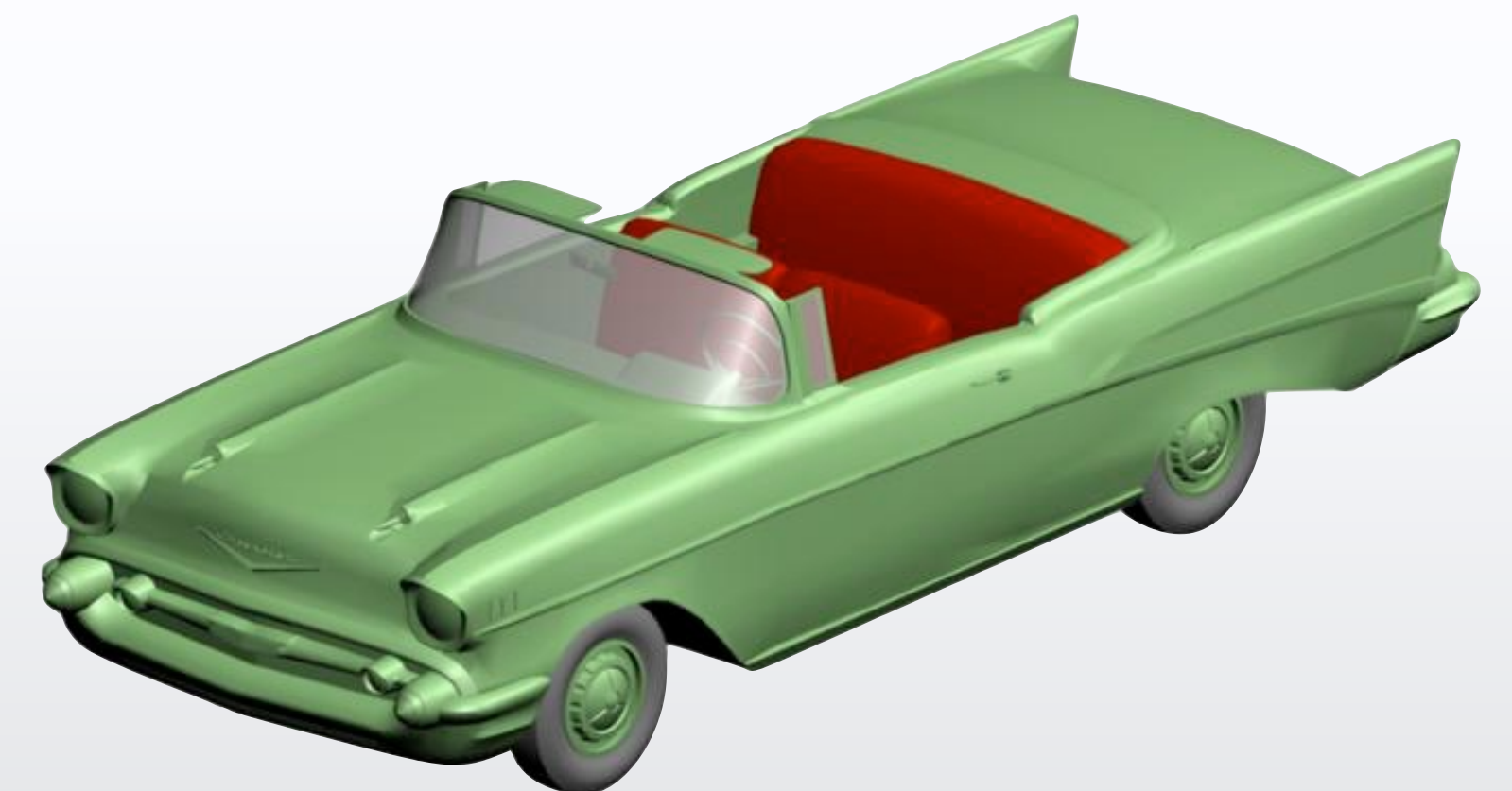
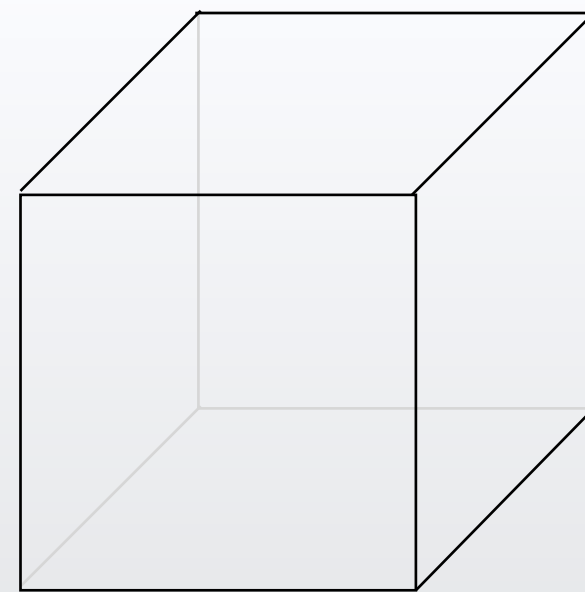
■ Perspective

- Vanishing points
(Fluchtpunkte)
- wo ist das Auge?

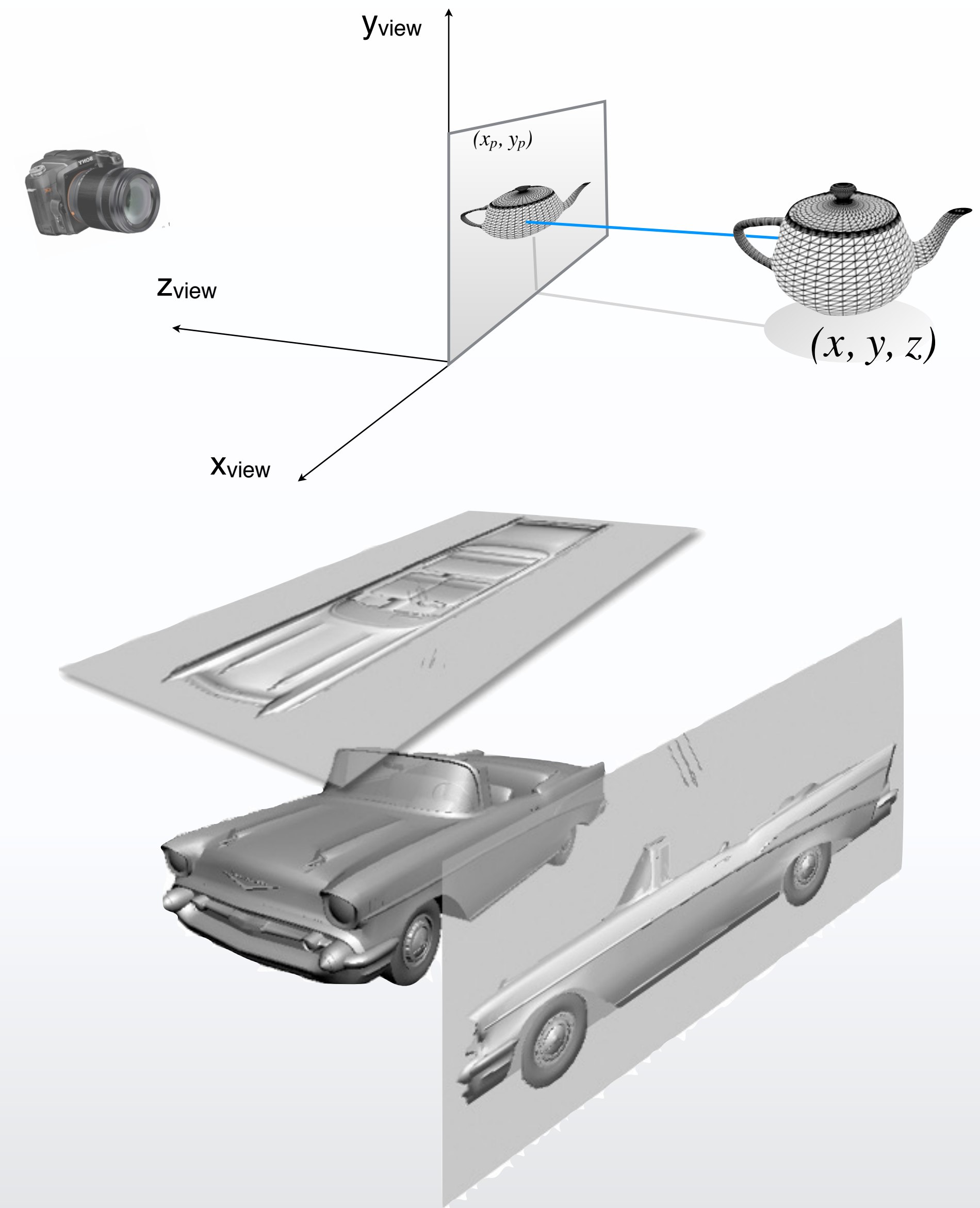


■ Orthogonal

- Parallelprojektion
- wo ist das Auge?

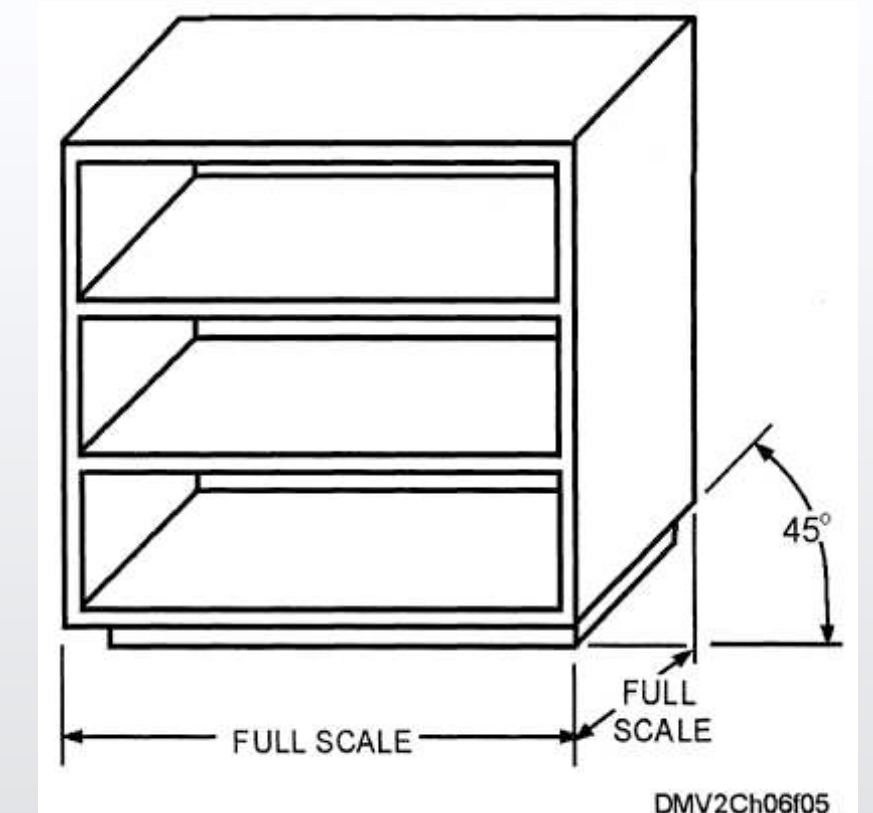
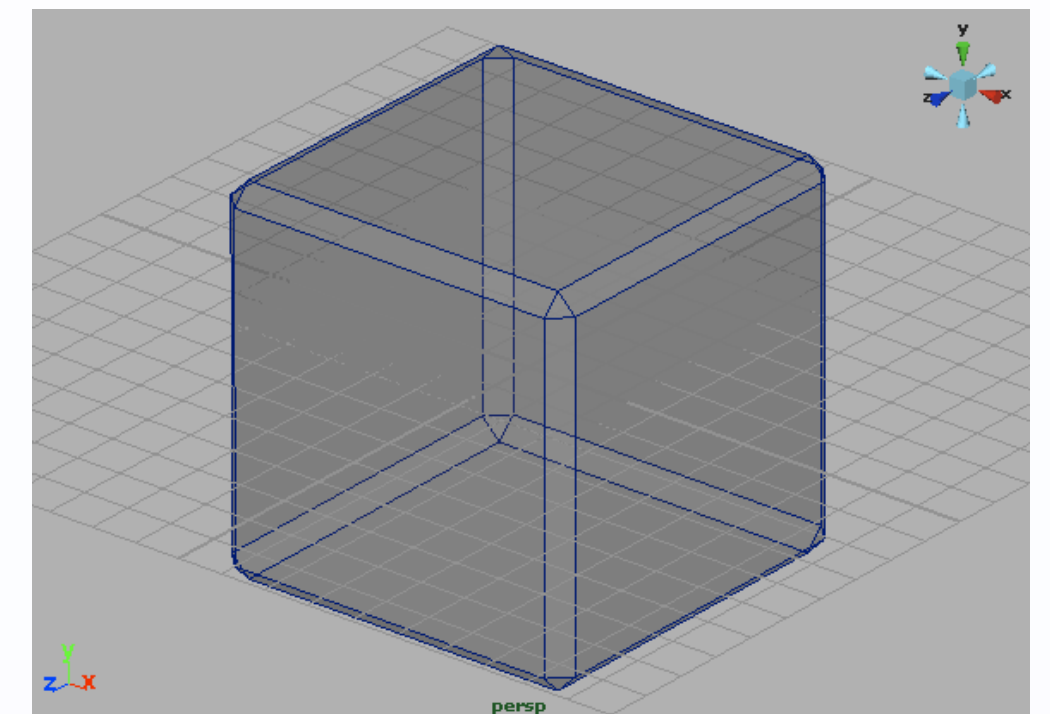
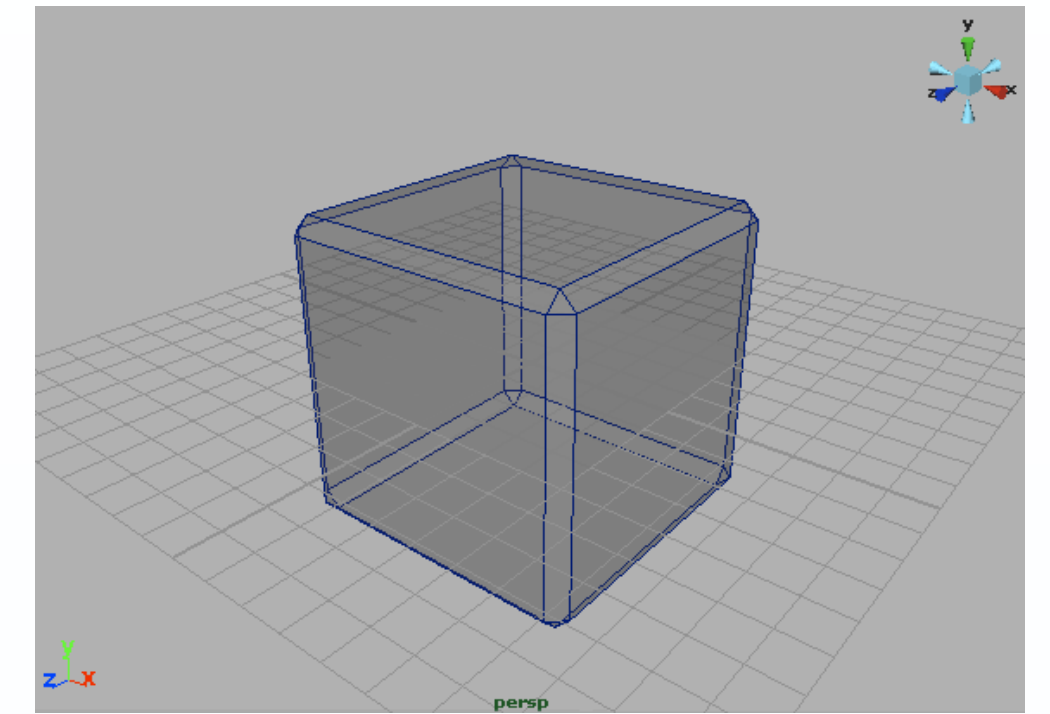
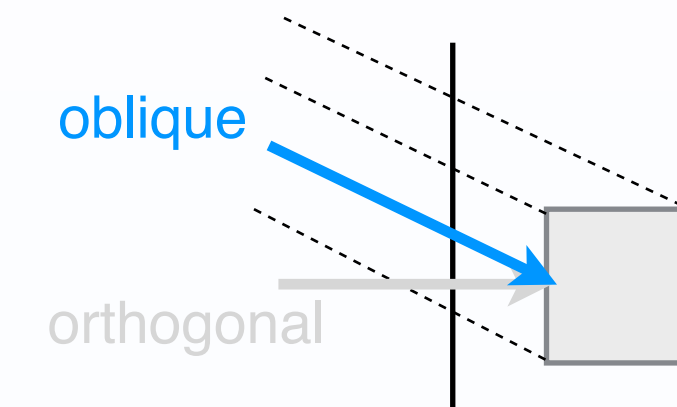
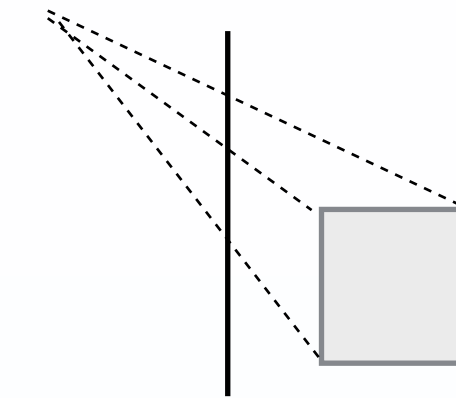


- Orthographic projection
 - Top, Side, Front
- Bekannt aus
 - Architektur
 - CAD-Anwendungen
- Viewplane perpendicular (senkrecht) to one principal axis
- z.B. $x_p=x$; $y_p=y$; $z=0$
- Z-Wert wird für die Verdeckung ausgewertet
- Längen bleiben gleich
- Wichtig für die Modellierung!!!



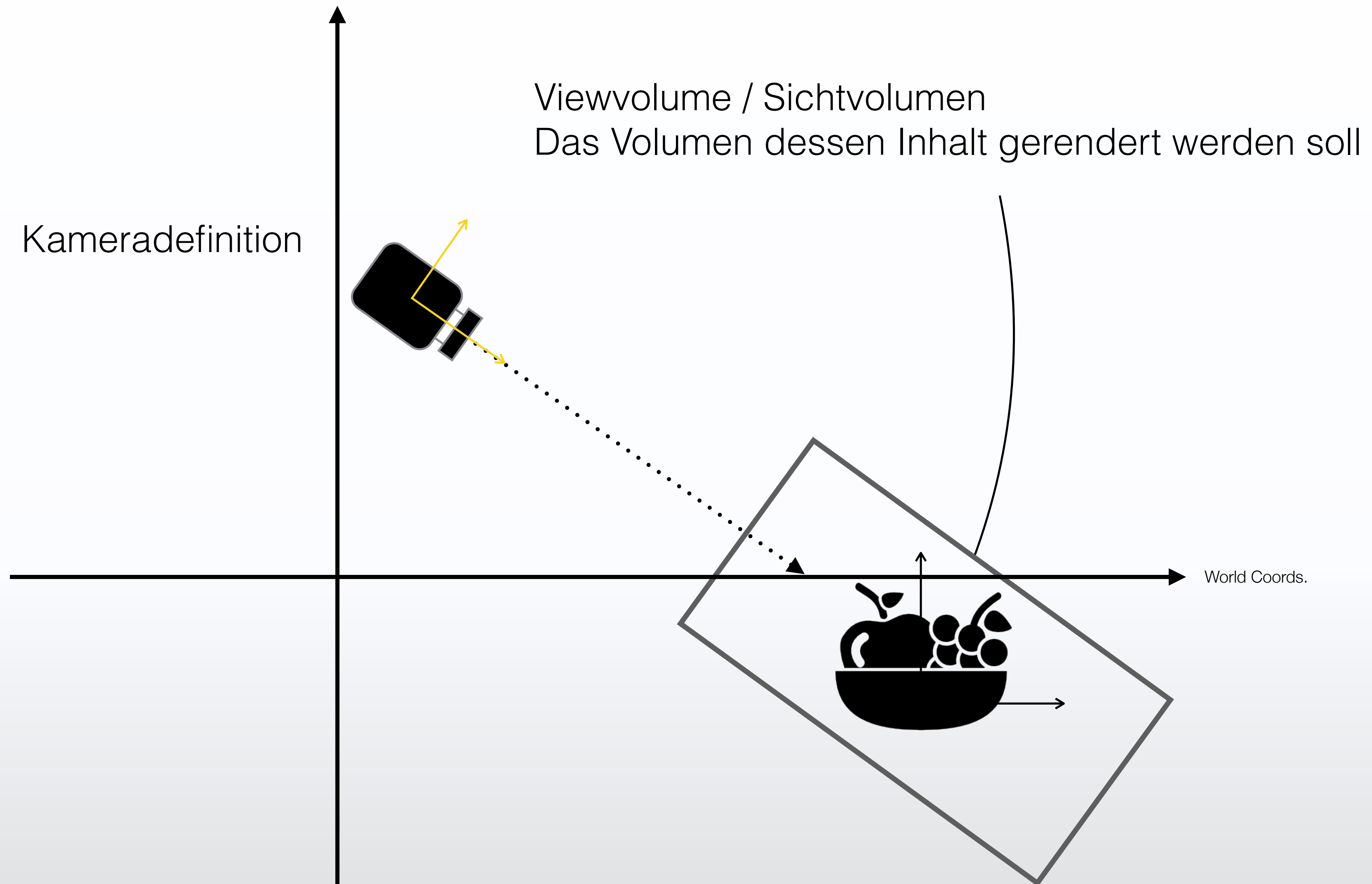
oblique parallel projection

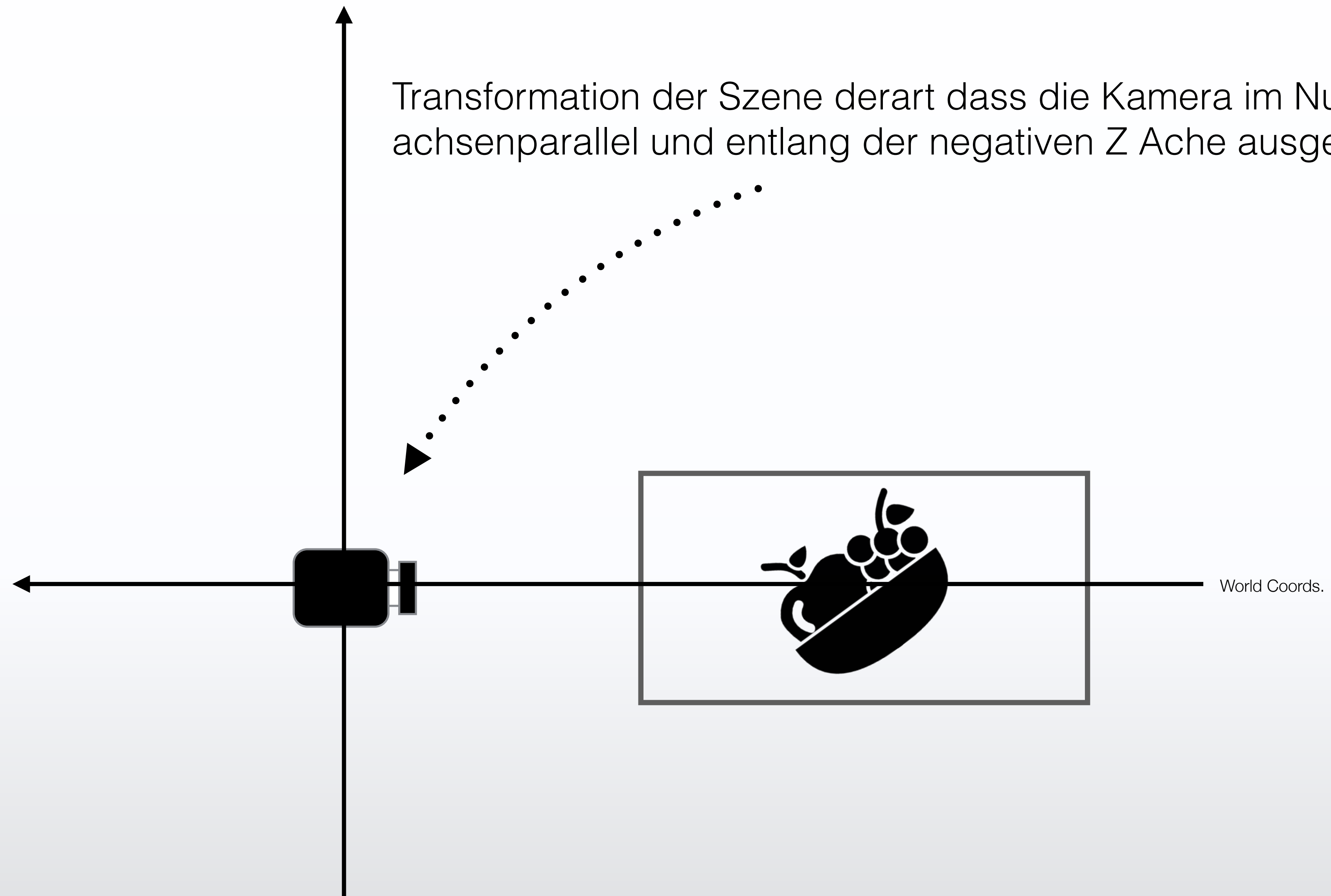
- Eine orthogonale Sicht reicht nicht, um ein Objekt zu beurteilen
- perspektivische Projektion:
- Vorteile:
 - realitätsnah
 - entspricht dem normalen Sehvorgang
- Nachteile
 - Längen schwer abzuschätzen
 - Ergebnis abhängig von Kameraparameter
- Darstellungsmethode für Designer, Architekten und Ingenieuren mit Maßeinheiten
 - Längen bleiben gleich
 - Wichtig für Bemaßungen und Verhältnisse

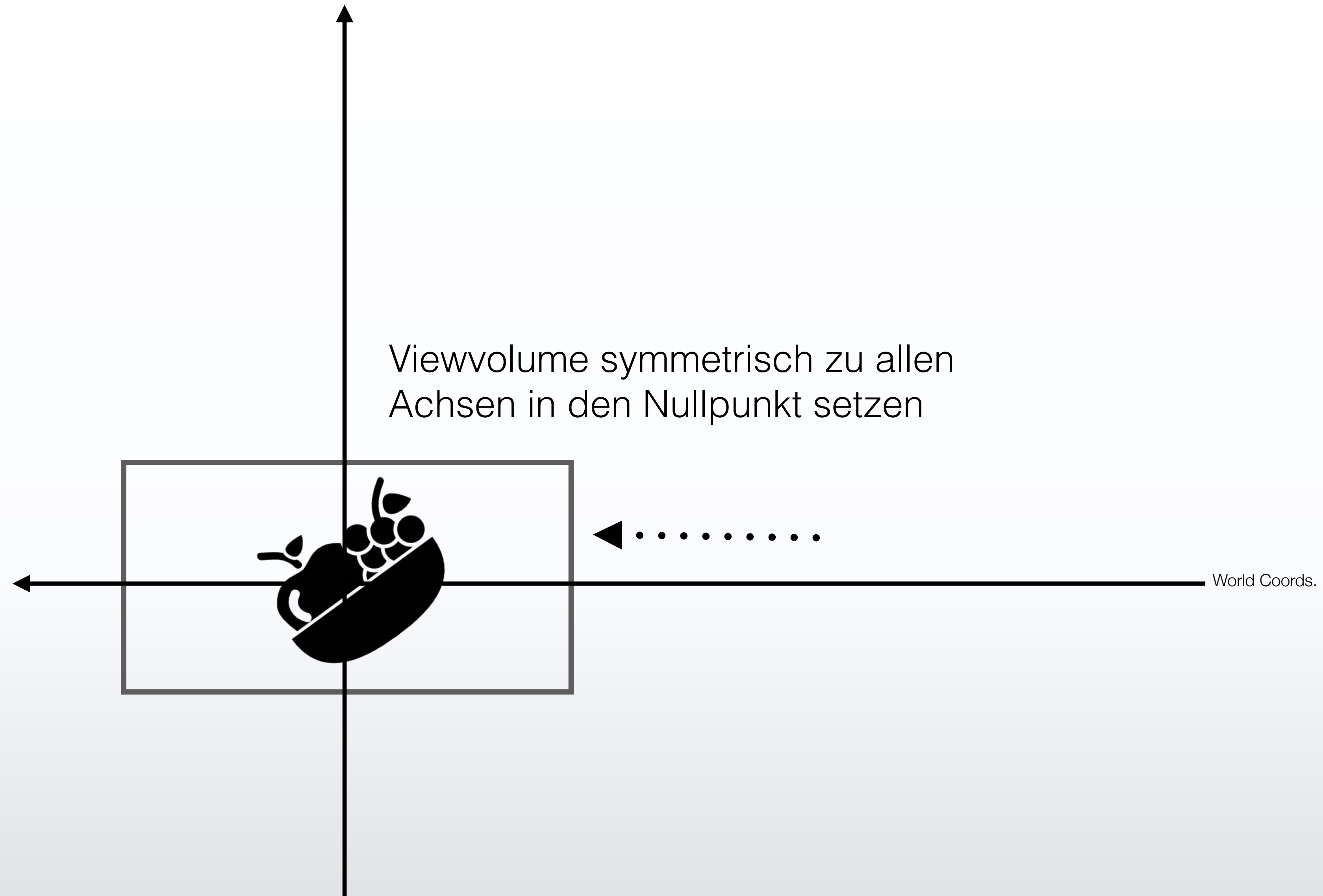


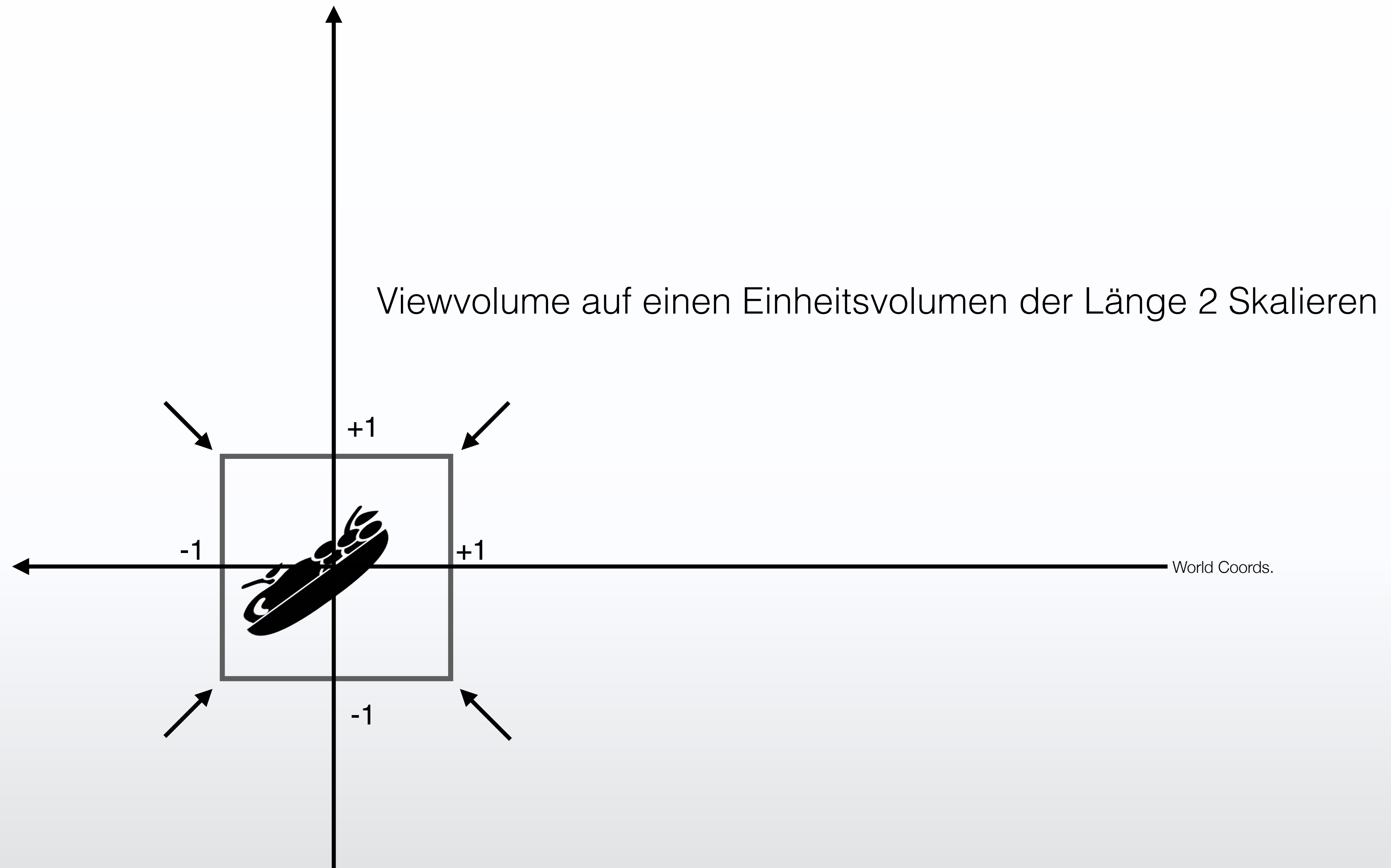
DMV2Ch06f05

Orthogonale Projektion in Bildern



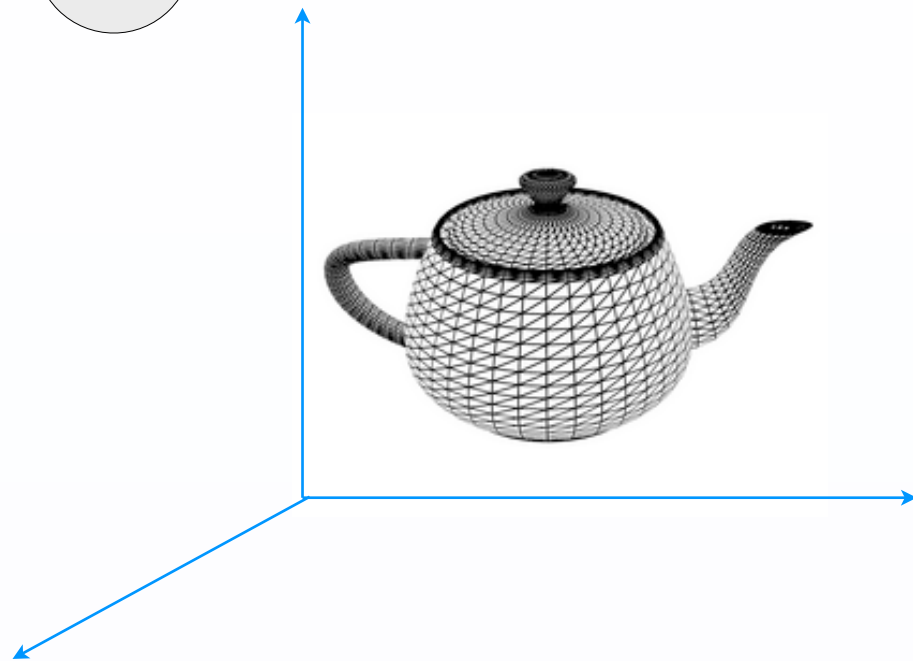






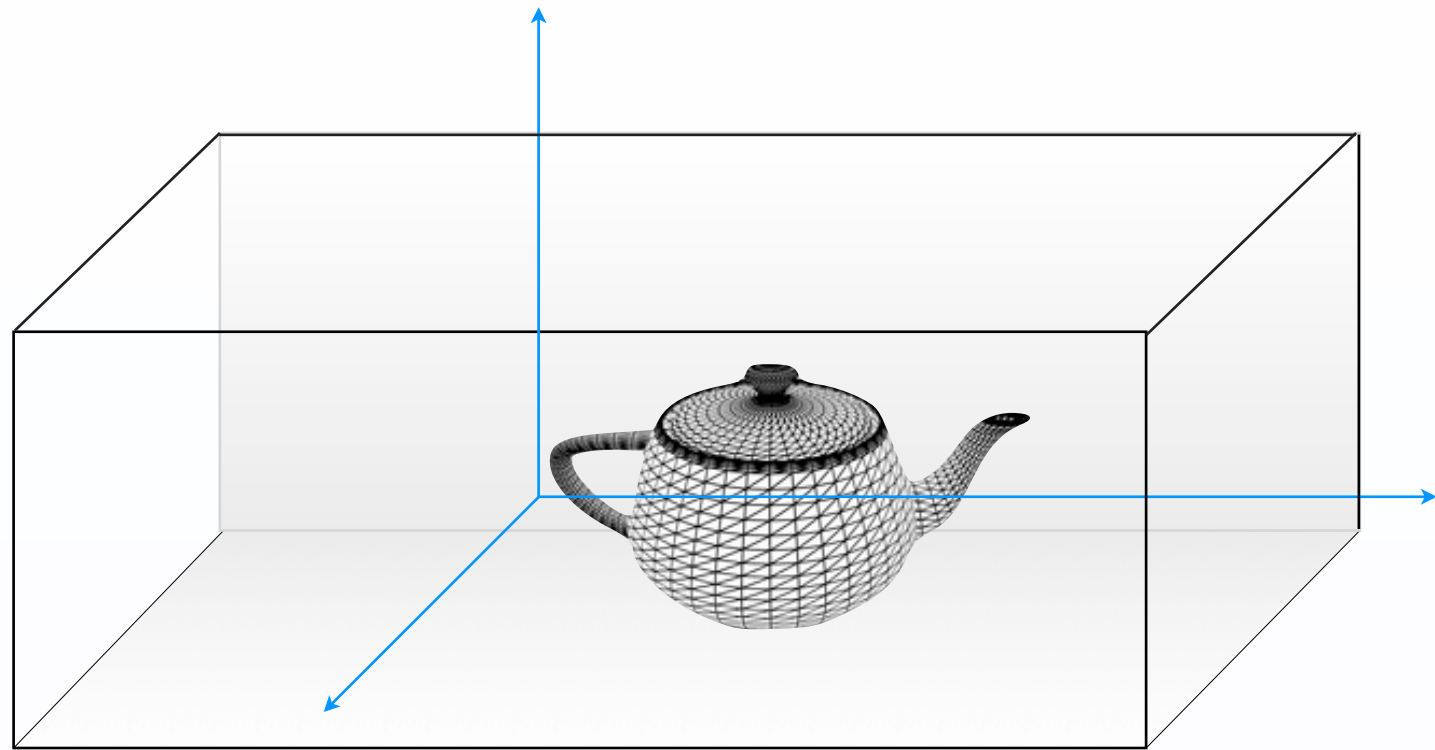
Orthogonal Projection - steps

1



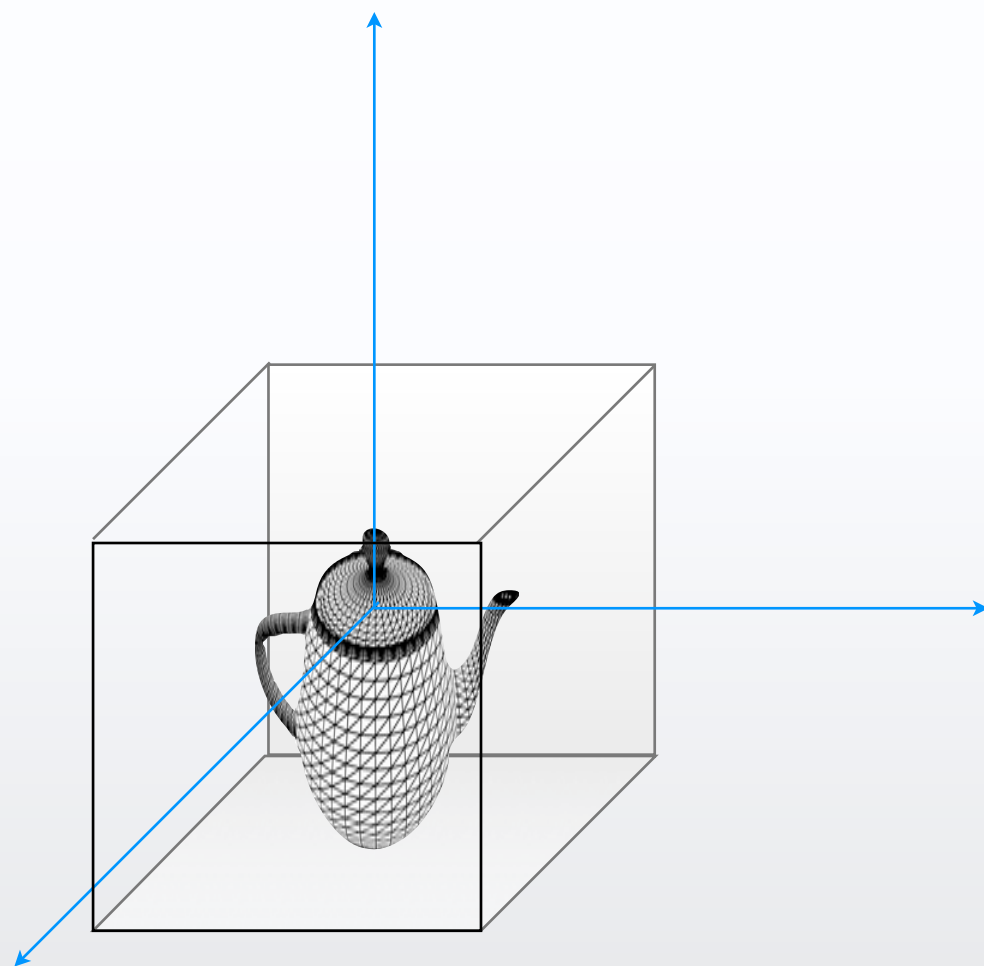
Globales Koordinatensystem

2



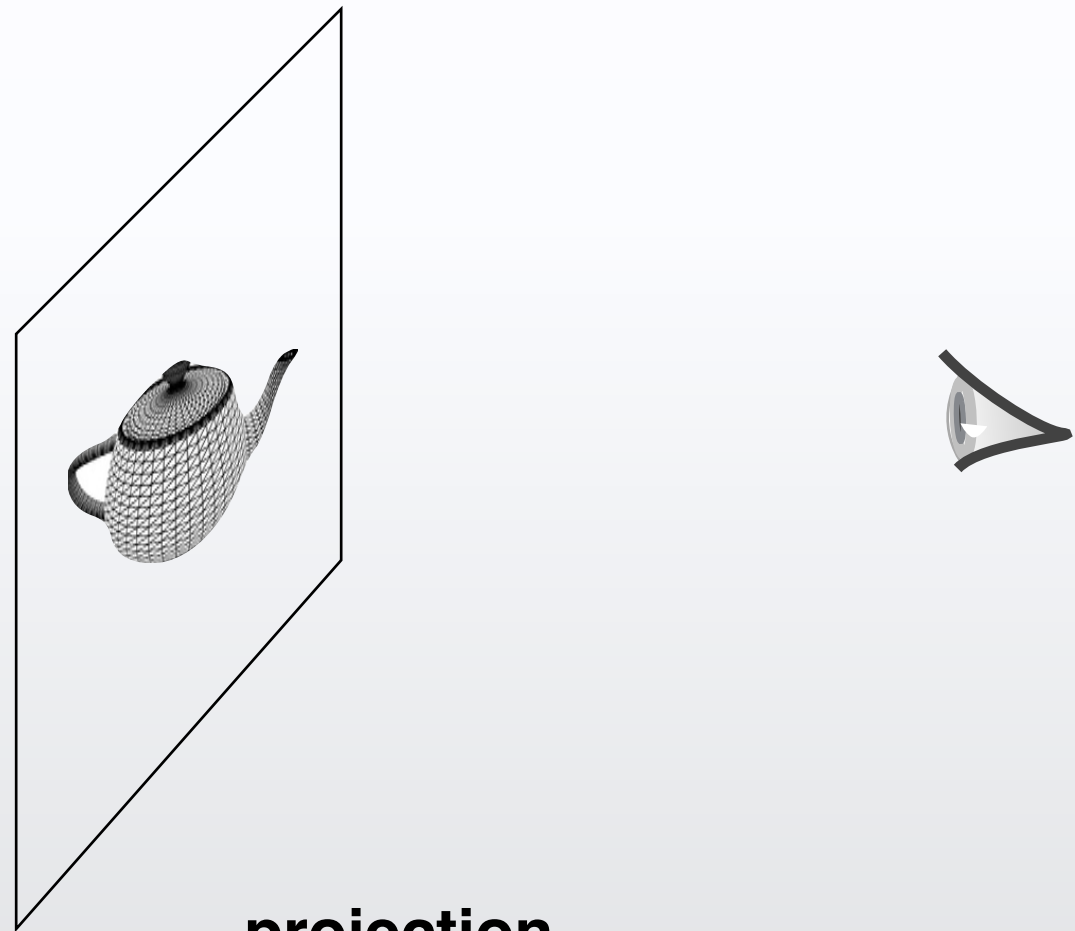
orthogonal projection view volume
Kanonisches Volumen

3



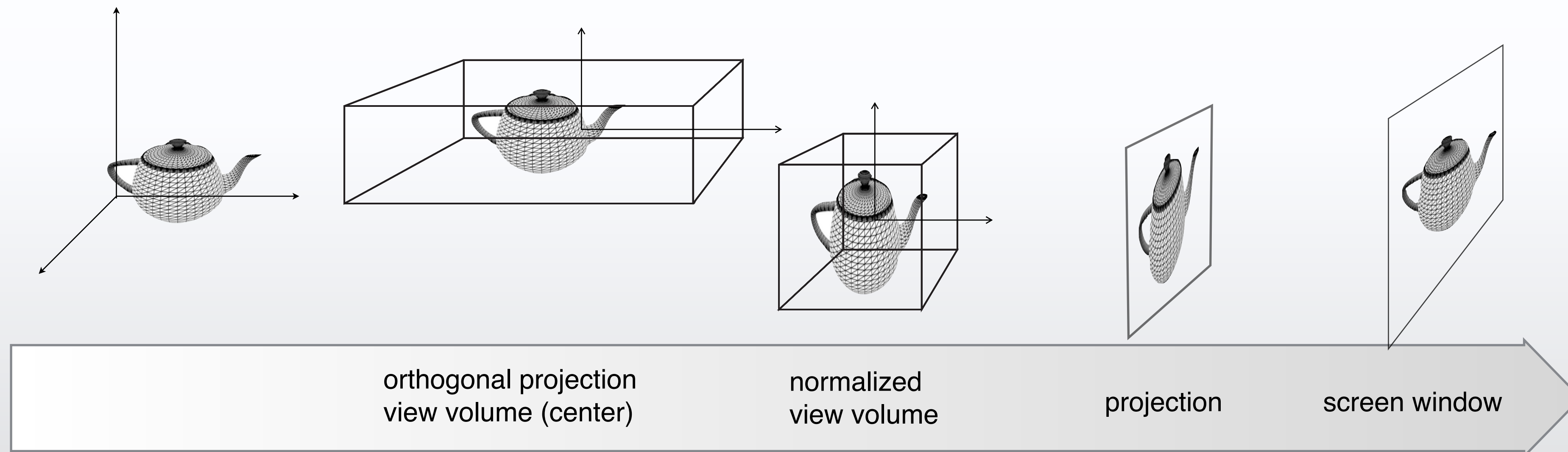
normalized view volume

4



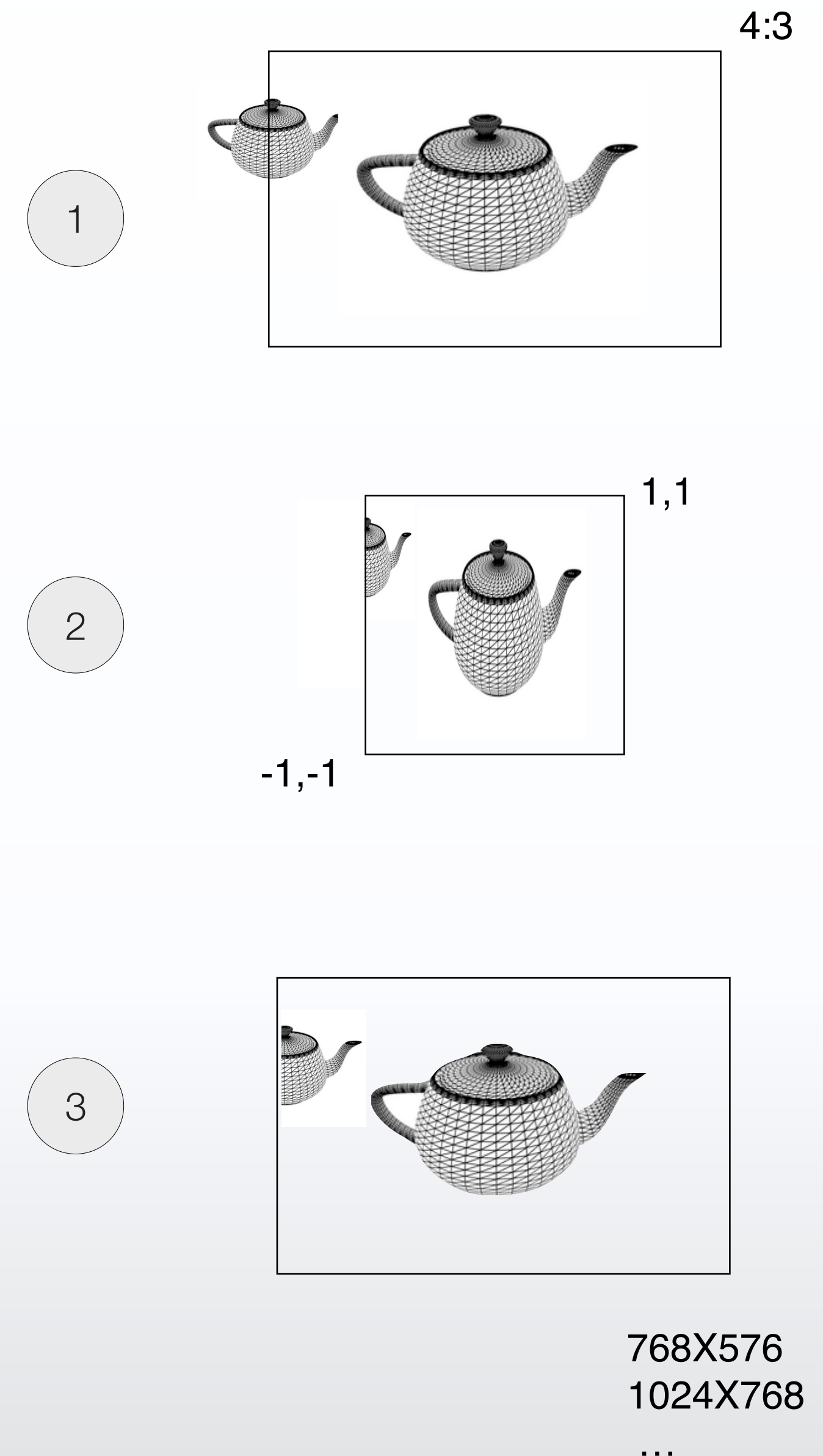
projection

- Position im Weltkoordinatensystem
- Position im Kamerasystem
- Forderung: Orthogonale Projektion
- Viewpoint & -Volume festlegen
- View Volume zentrieren
- View Volume normalisieren
- Back Face Culling
- Clipping - mit achsenparallelen Ebenen (+1,-1)
- Eine Koordinate eliminieren (Projektion)
 - Tiefenwert für Verdeckung nutzen
- View Port Transformation
 - Position in Screen Koordinatensystem

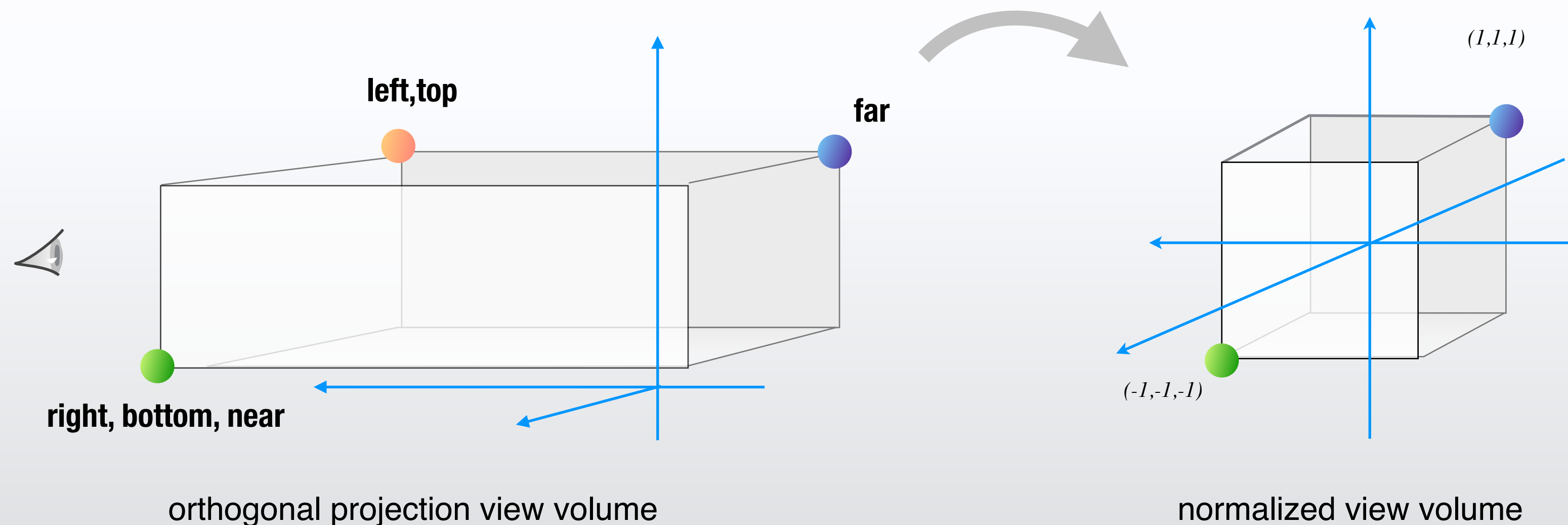


Warum Normierung ?

- Orthogonal Projection View Volume
- Normierung auf Einheitsvolumen (-1,1)
- **Culling** (Back Face Removal)
 - Testen ob der Z Wert einen negativen Wert hat
- **Clipping** der Polygone
 - Schnitttests und Clipping-Verfahren gegen Achsenparallele Ebenen
 - Alle Gleichungen und Berechnungen einfacher, wenn die Ebenen mit -1,+1 zu definieren sind.
- **Viewporttransformation**
 - „Entzerren“ des „Sichtquadrates“ auf die endgültige Fenstergröße des Sichtfensters
 - Mit den normierten Werten können die Werte einfach zum „Pixel-Wert“ umgerechnet werden.



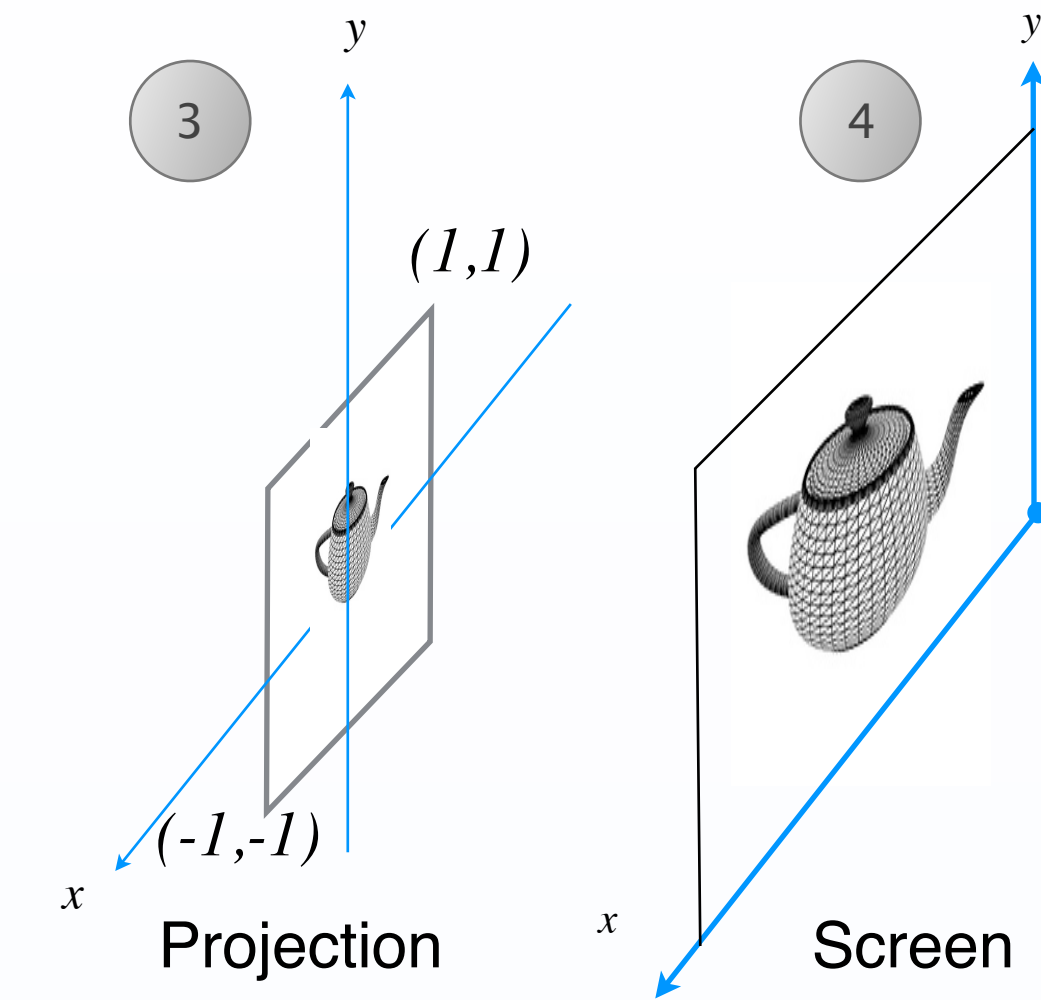
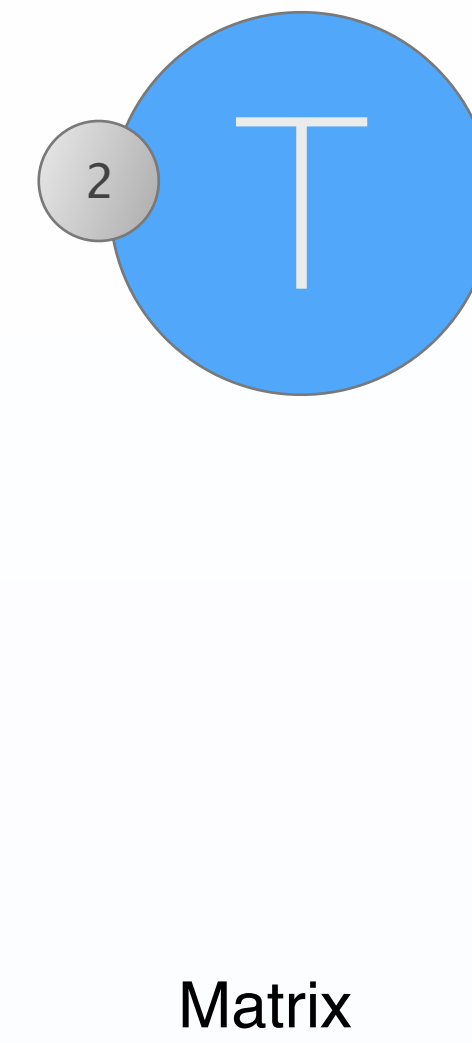
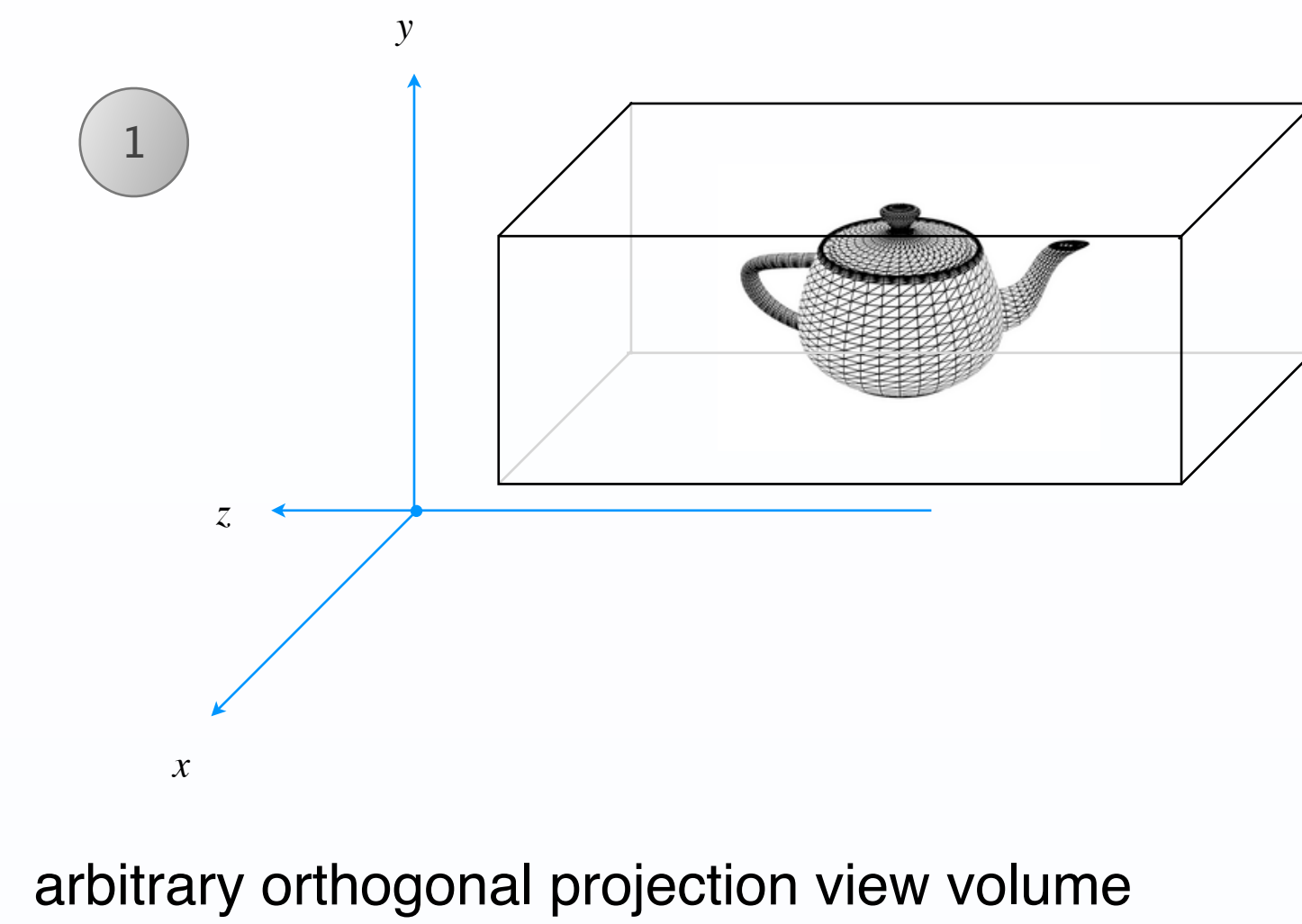
- Bisher ist die Viewplane zu bestimmen mit
 - (top,bottom, left, right)
- Einführung des achsenparallelen Sichtvolumens
AABB (Axisaligned BoundingBox) definiert durch 6-Tupel
 - **(n,f,l,r,b,t)** (*near,far,left,right,bottom,top*)
- Einführung von “Clipping Plane”
 - **near**, **far**
- Normalization, Clipping, Screencoordinates



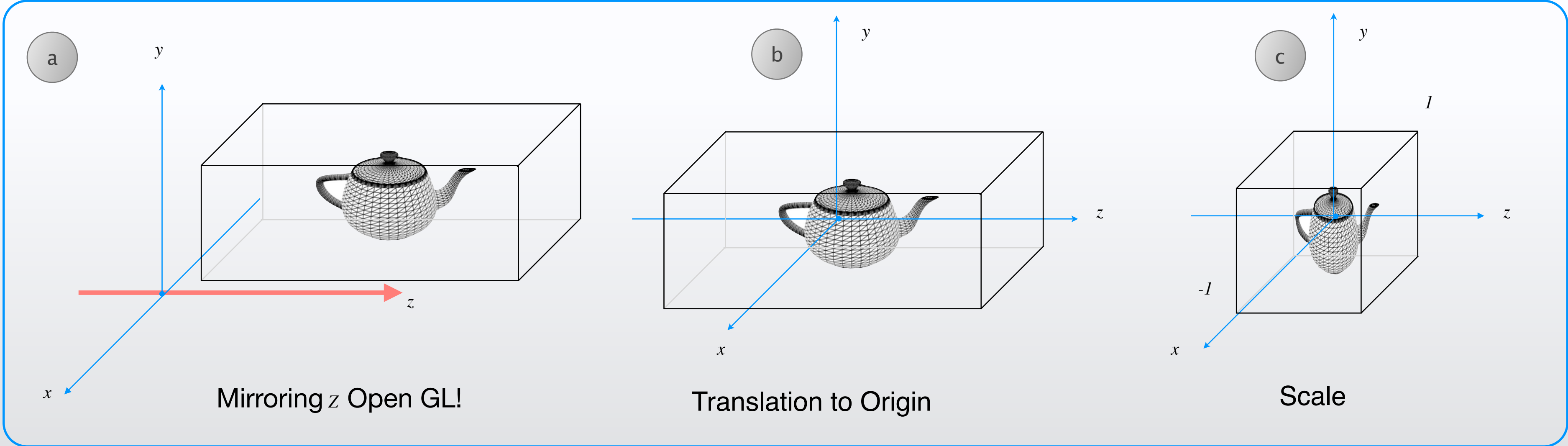
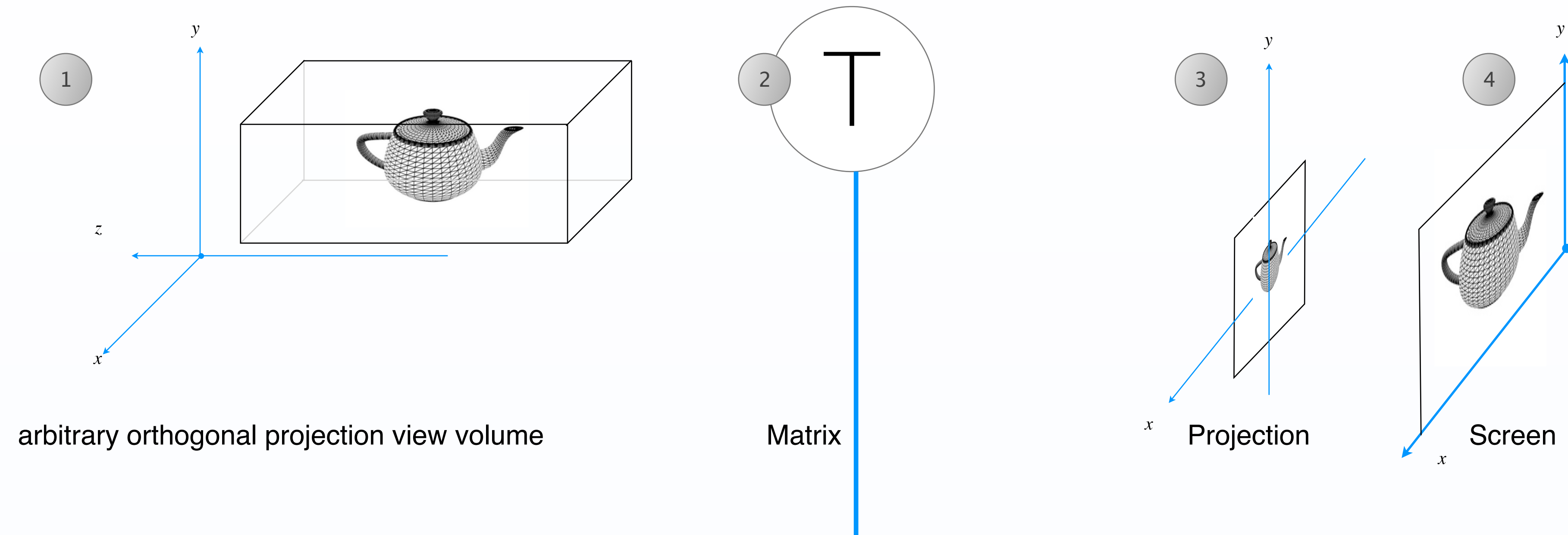
glOrtho(left,right,bottom,top,near,far)

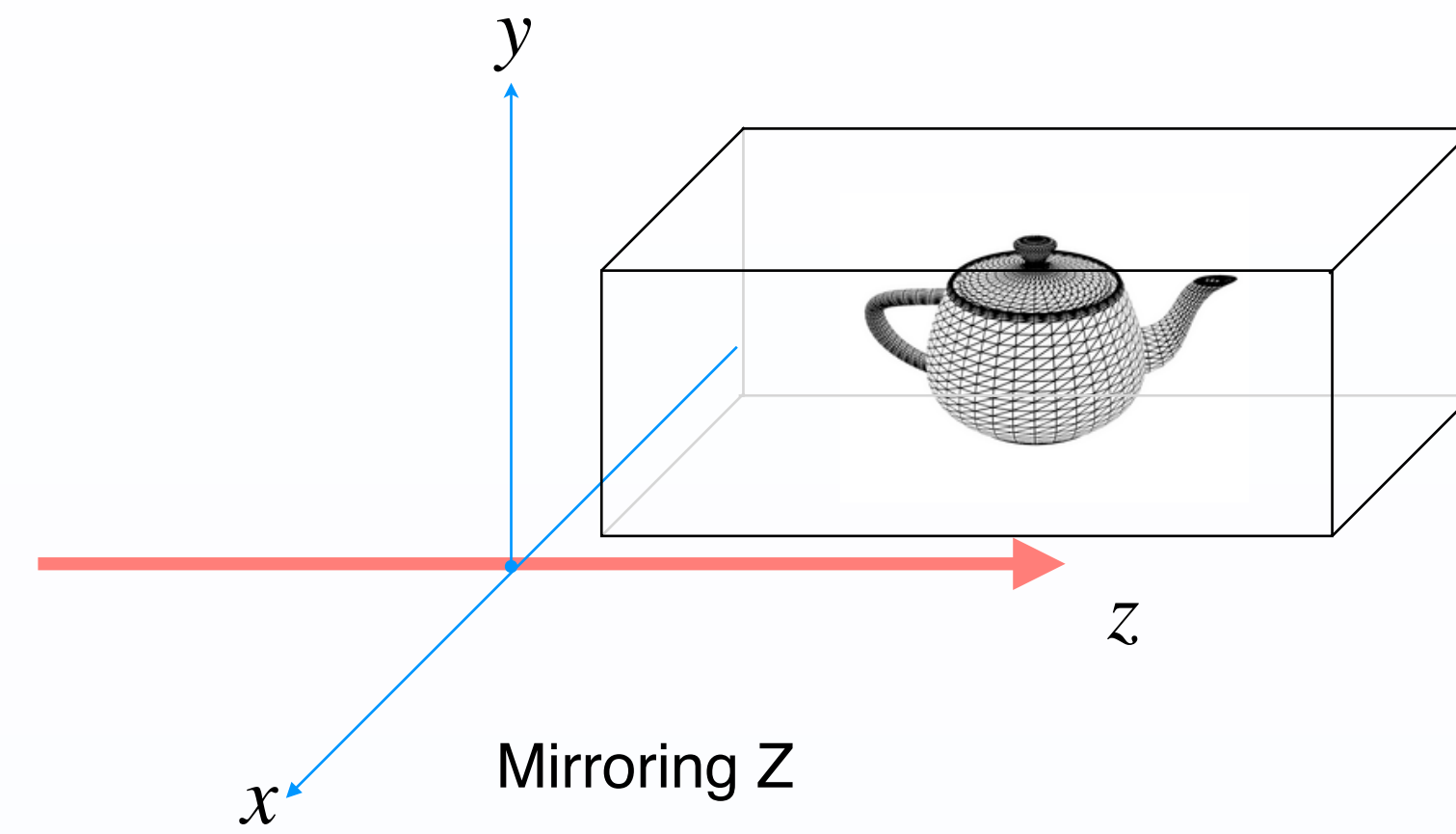
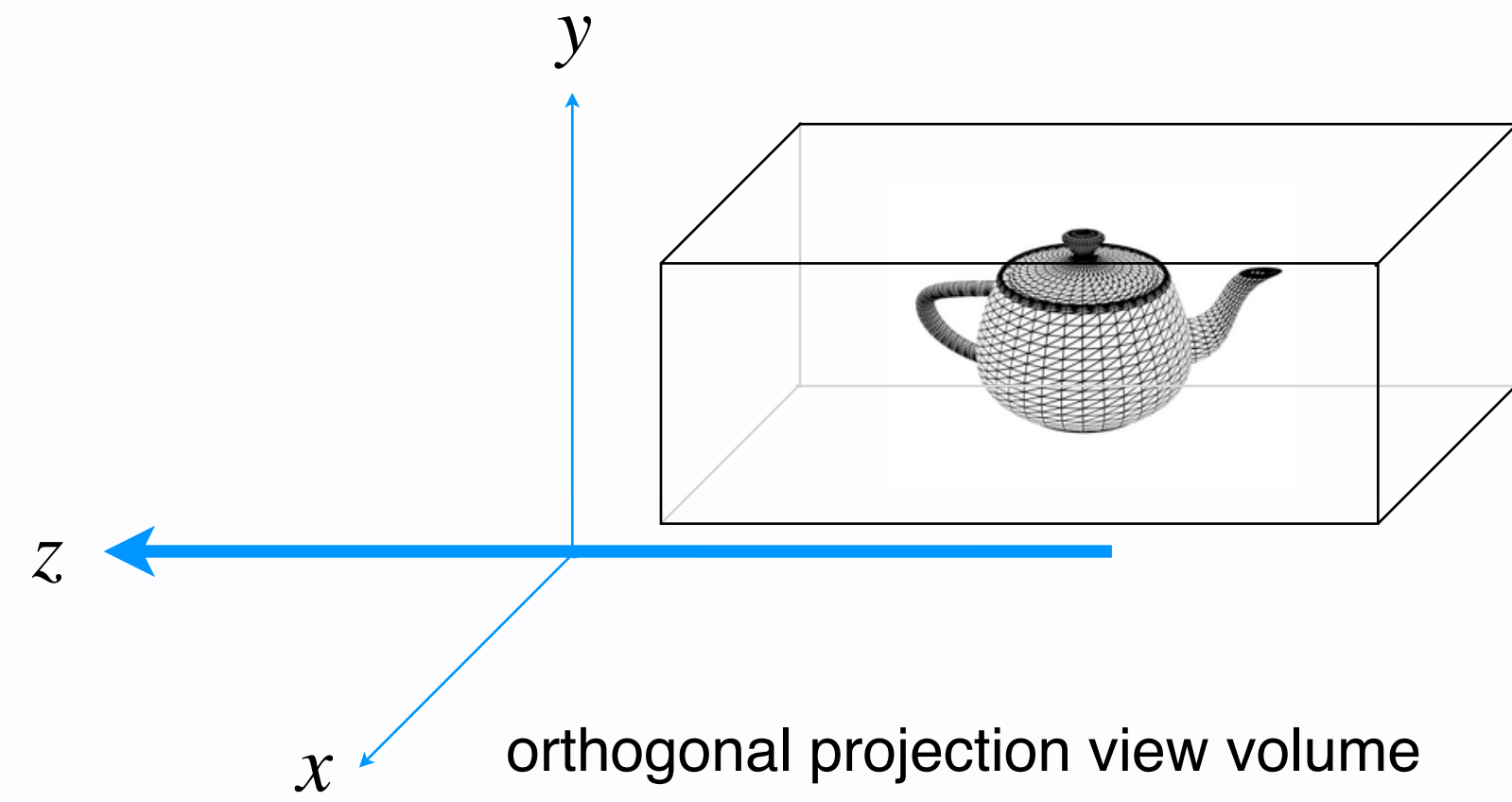
$$M = \begin{pmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{l+r}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & \frac{-2}{f-n} & -\frac{f+n}{f-n} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Was macht die Matrix ?



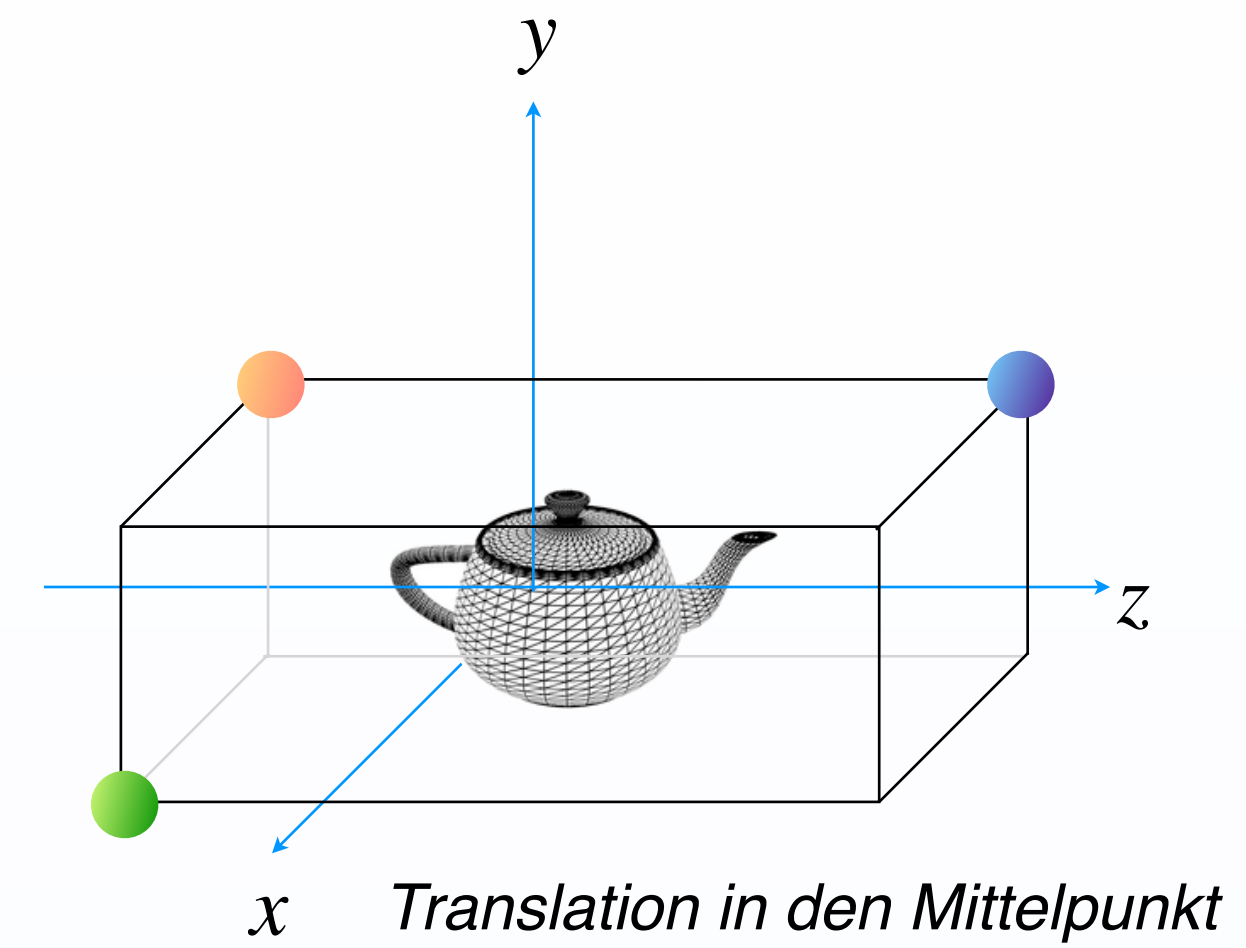
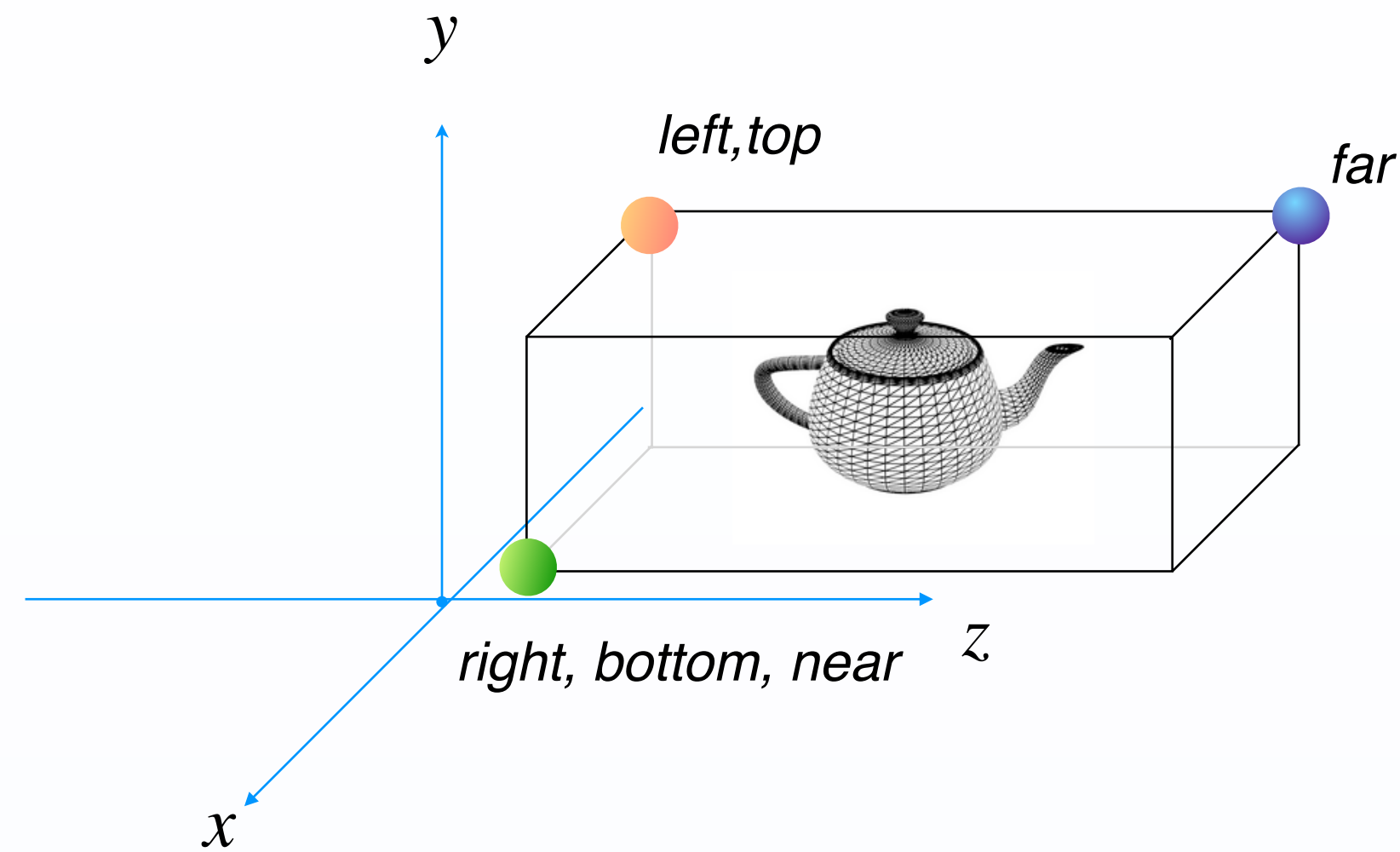
Was macht die Matrix ?



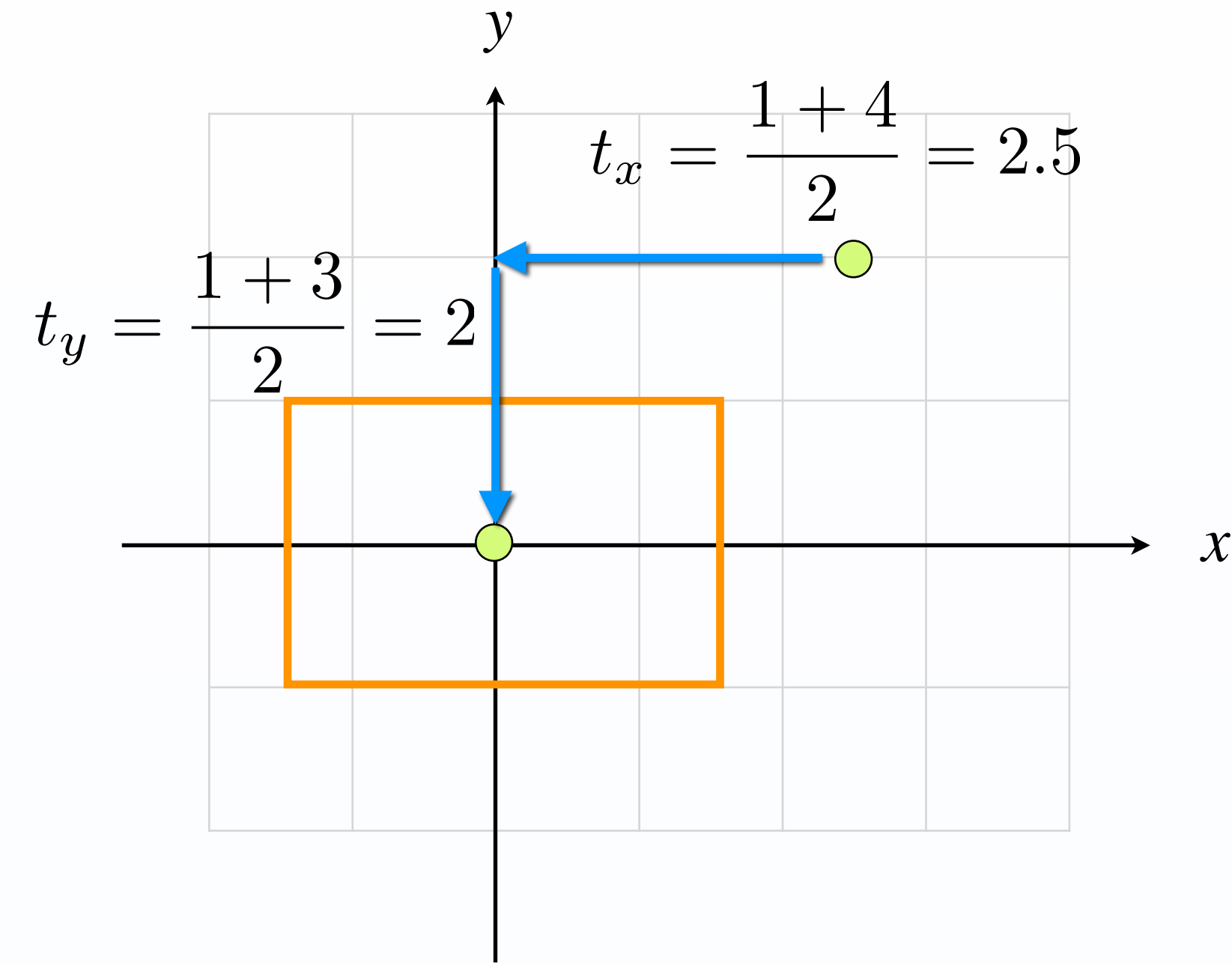
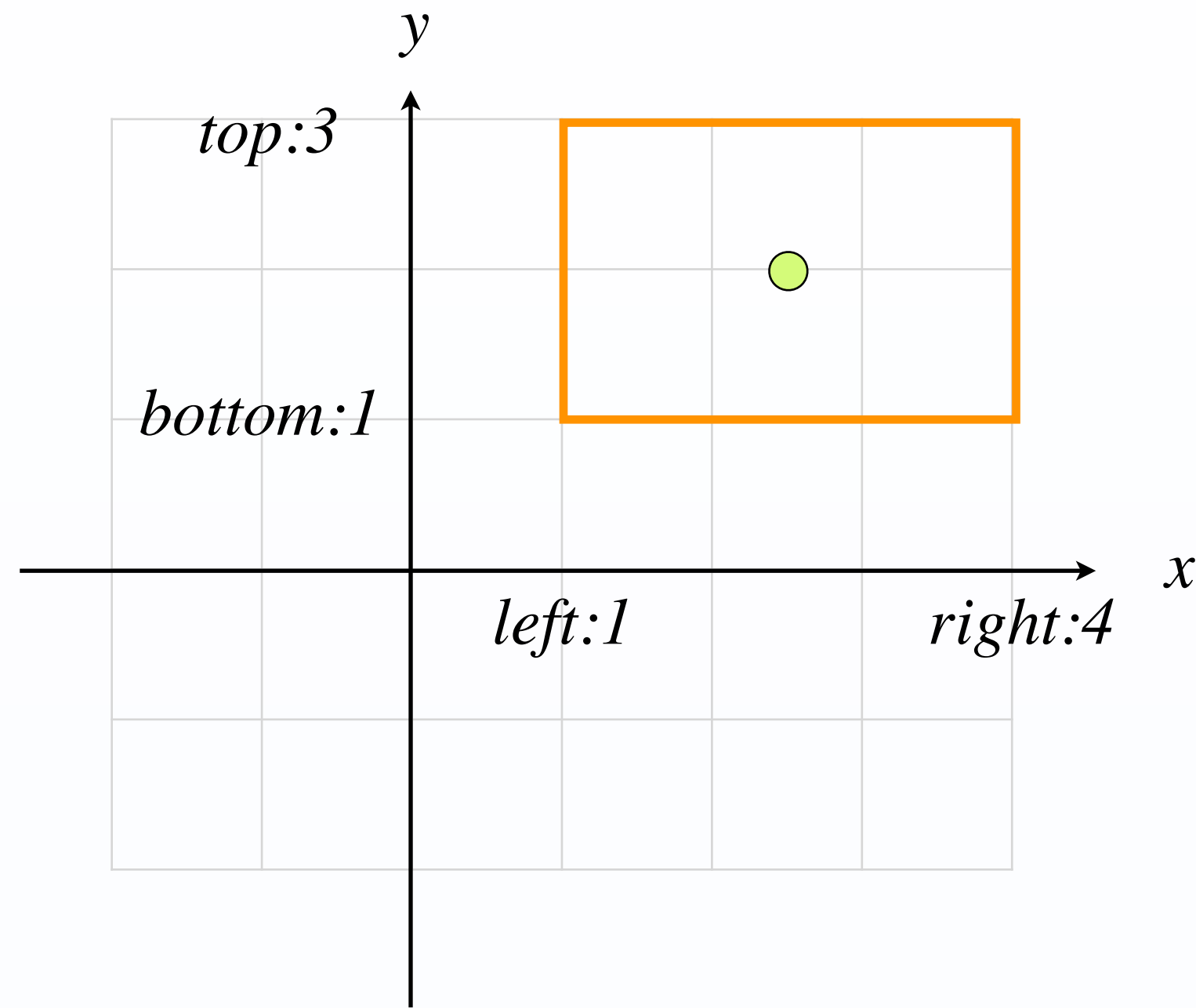


$$M_{Sp} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Translate to Origin



$$M_T = \begin{pmatrix} 1 & 0 & 0 & -\frac{l+r}{2} \\ 0 & 1 & 0 & -\frac{b+t}{2} \\ 0 & 0 & 1 & -\frac{f+n}{2} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$



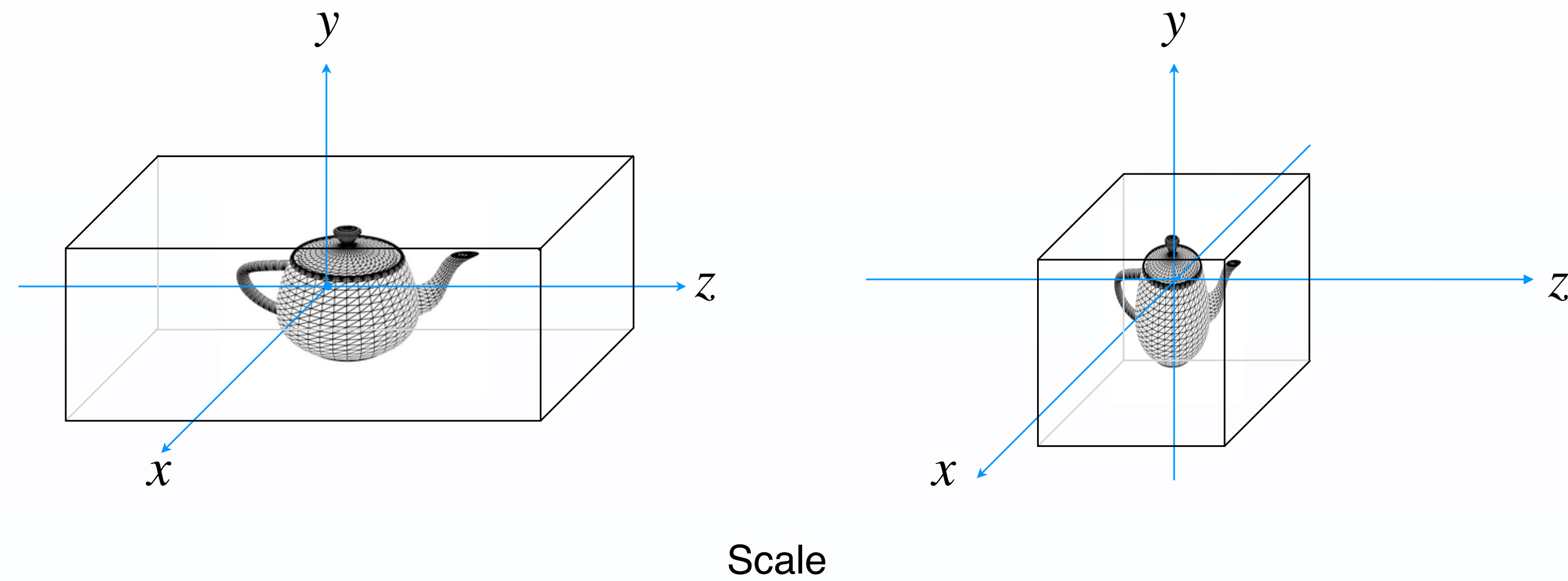
Allgemeine Position des Mittelpunktes
in X-Richtung:

$$l + \frac{1}{2}(r - l) = \frac{l + r}{2}$$

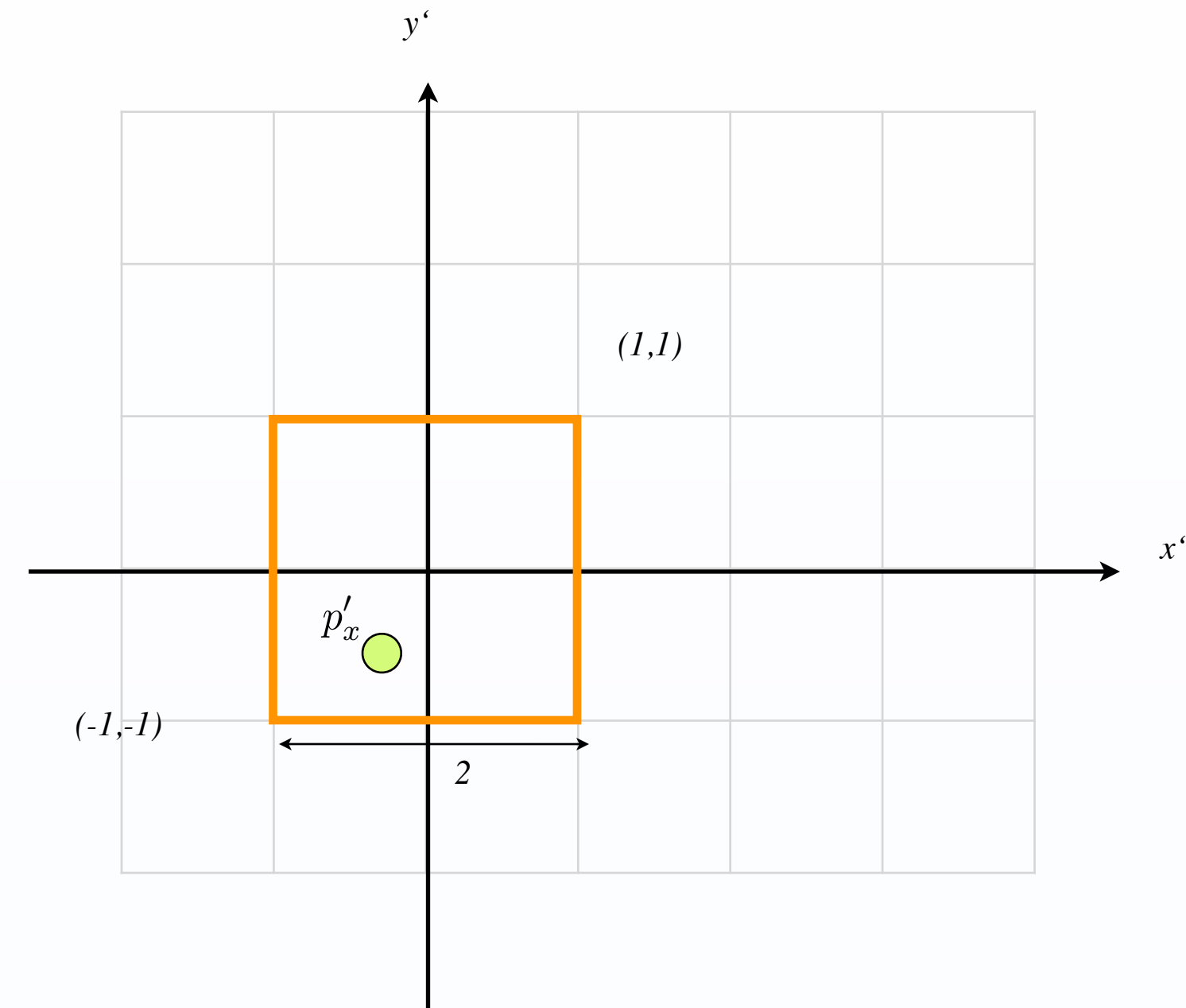
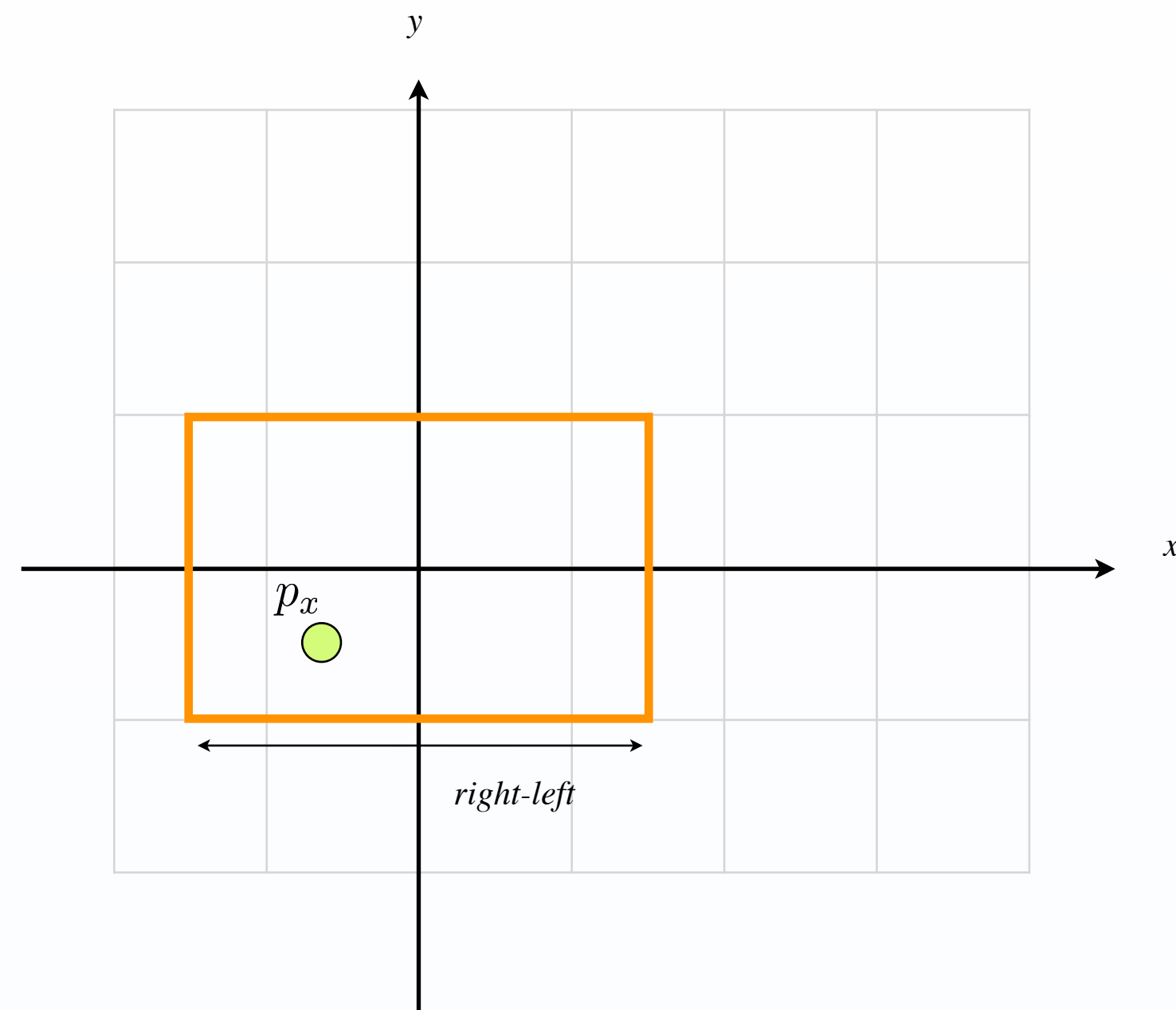
Breite des Rechtecks

Für das Volumen:

$$M_T = \begin{pmatrix} 1 & 0 & 0 & -\frac{l+r}{2} \\ 0 & 1 & 0 & -\frac{b+t}{2} \\ 0 & 0 & 1 & -\frac{f+n}{2} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$



$$M_T = \begin{pmatrix} \frac{2}{r-l} & 0 & 0 & 0 \\ 0 & \frac{2}{t-b} & 0 & 0 \\ 0 & 0 & \frac{2}{f-n} & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$



Allgemein: $\frac{p'_x}{p_x} = \frac{2}{r-l}$

damit: $p'_x = \frac{2}{r-l} \cdot p_x$

Für das Volumen: $M_T = \begin{pmatrix} \frac{2}{r-l} & 0 & 0 & 0 \\ 0 & \frac{2}{t-b} & 0 & 0 \\ 0 & 0 & \frac{2}{f-n} & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$

Die Konkatination der Transformationen:

$$M = \begin{matrix} \text{Skalierung} & & \text{Translation} & & \text{Spiegelung} \\ \begin{pmatrix} \frac{2}{r-l} & 0 & 0 & 0 \\ 0 & \frac{2}{t-b} & 0 & 0 \\ 0 & 0 & \frac{2}{f-n} & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} & \cdot & \begin{pmatrix} 1 & 0 & 0 & -\frac{l+r}{2} \\ 0 & 1 & 0 & -\frac{b+t}{2} \\ 0 & 0 & 1 & -\frac{f+n}{2} \\ 0 & 0 & 0 & 1 \end{pmatrix} & \cdot & \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \end{matrix}$$



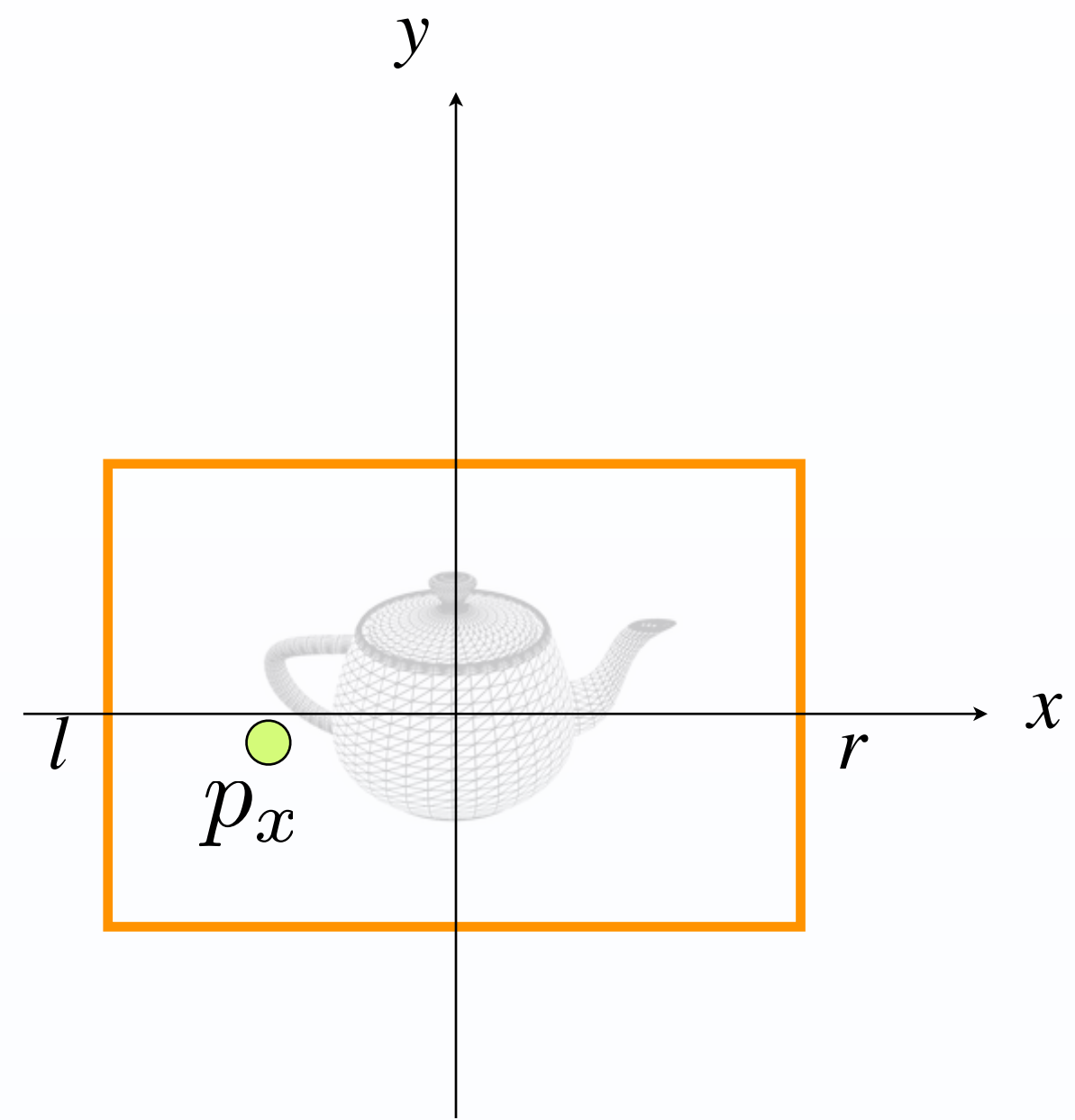
M_{Ortho}



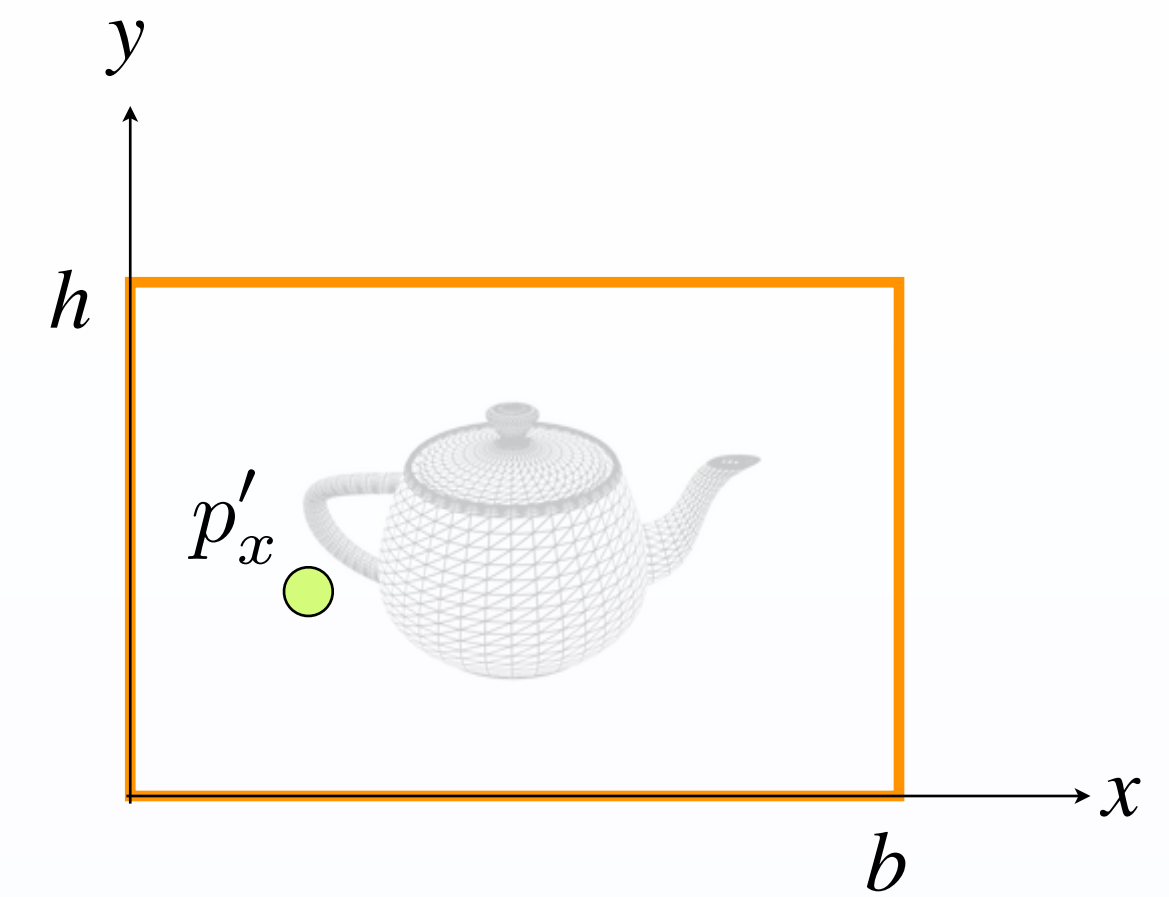
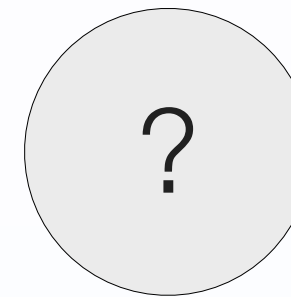
$M_{R \sim L}$

$$M = \begin{pmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{l+r}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & \frac{-2}{f-n} & -\frac{f+n}{f-n} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

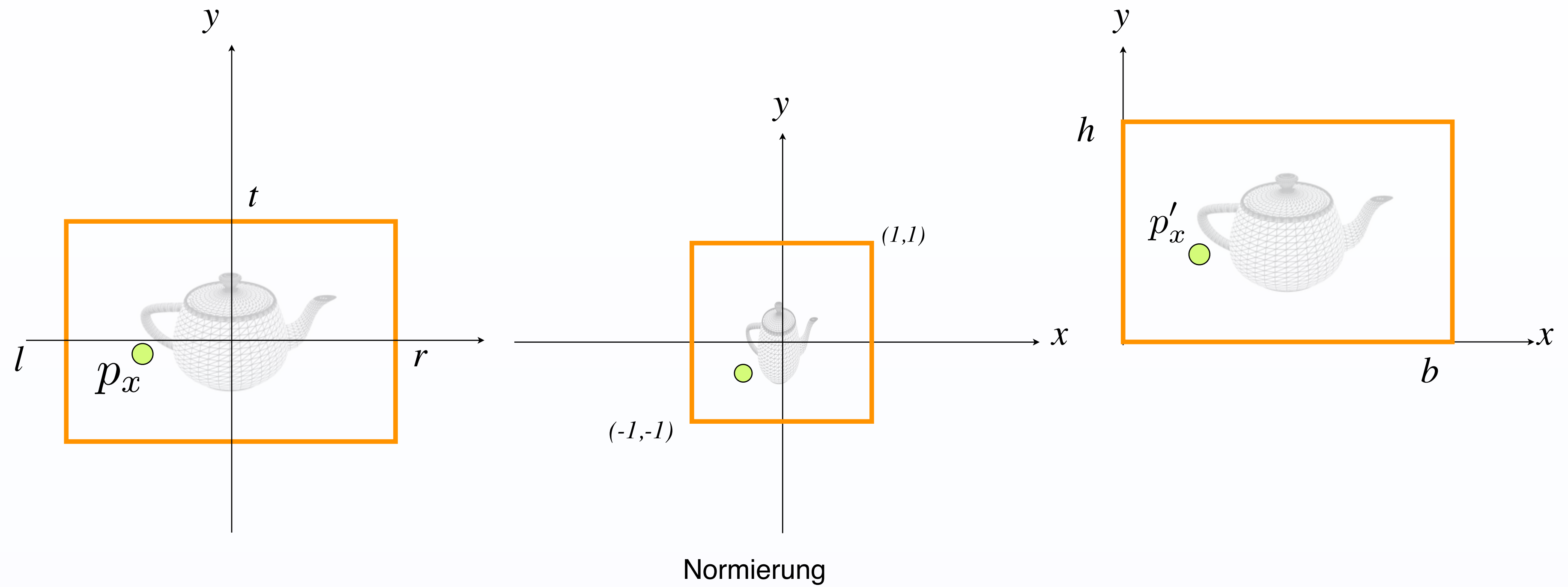
`glOrtho(left,right,bottom,top,near,far)`



right / left / top in Weltkoordinaten

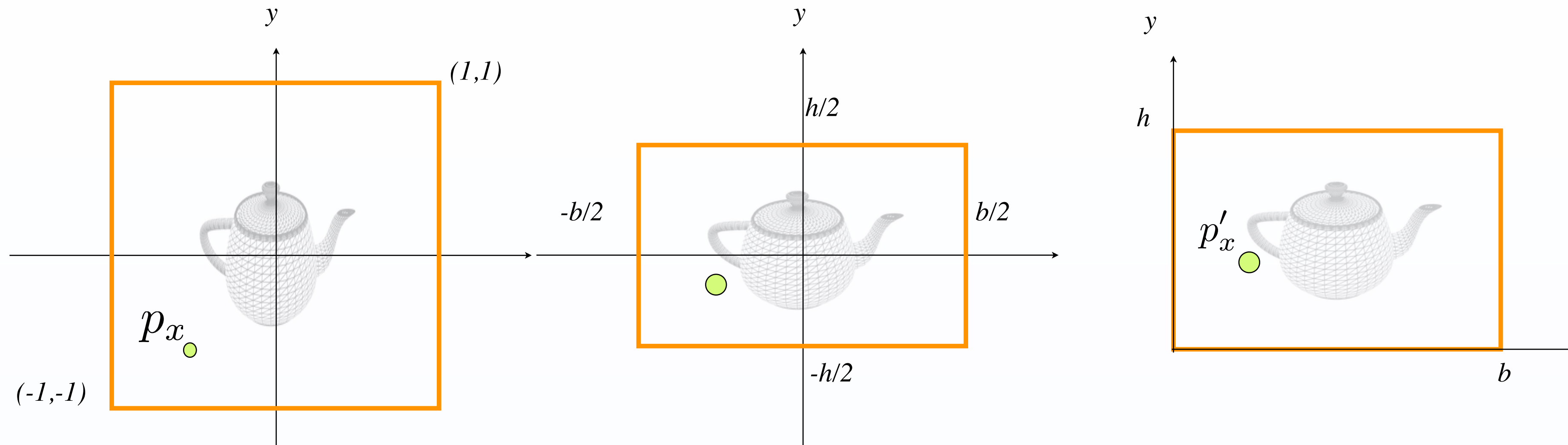


Höhe / Breite in Pixel





Entzerren des Einheits-Quadrates auf das Sichtfenster



$$\begin{aligned}
 p'_x &= \frac{b}{2} \cdot p_x + \frac{b}{2} \\
 p'_y &= \frac{h}{2} \cdot p_y + \frac{h}{2} \\
 p'_z &= \frac{1}{2} \cdot (p_z + 1) \quad \text{Wertebereich des Z Buffers?}
 \end{aligned}$$

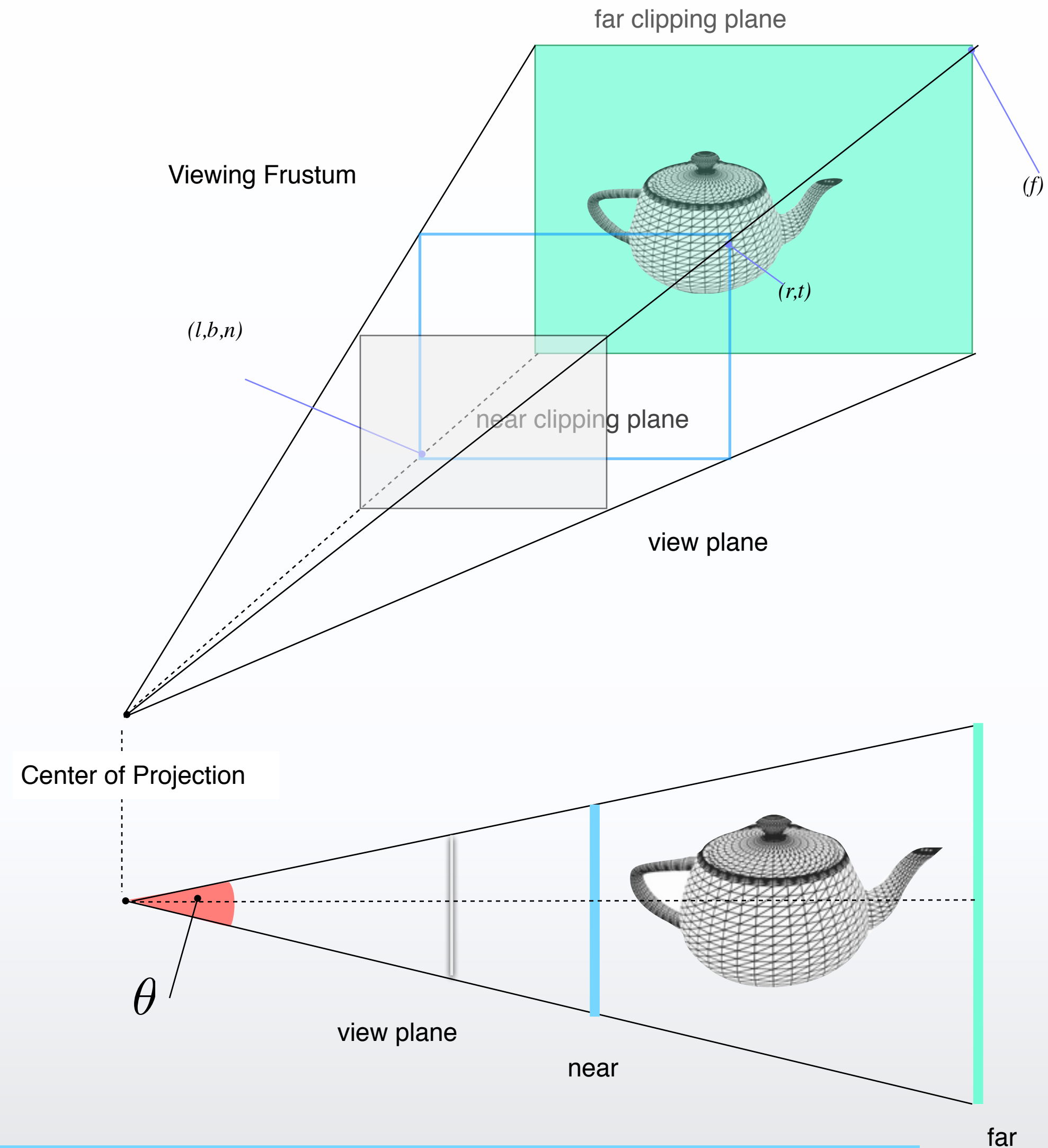

```
void gluPerspective(GLdouble fovy, GLdouble aspect, GLdouble near, GLdouble far);
```

$$M = \begin{pmatrix} \frac{2n}{r-l} & 0 & \frac{l+r}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{b+t}{t-b} & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$



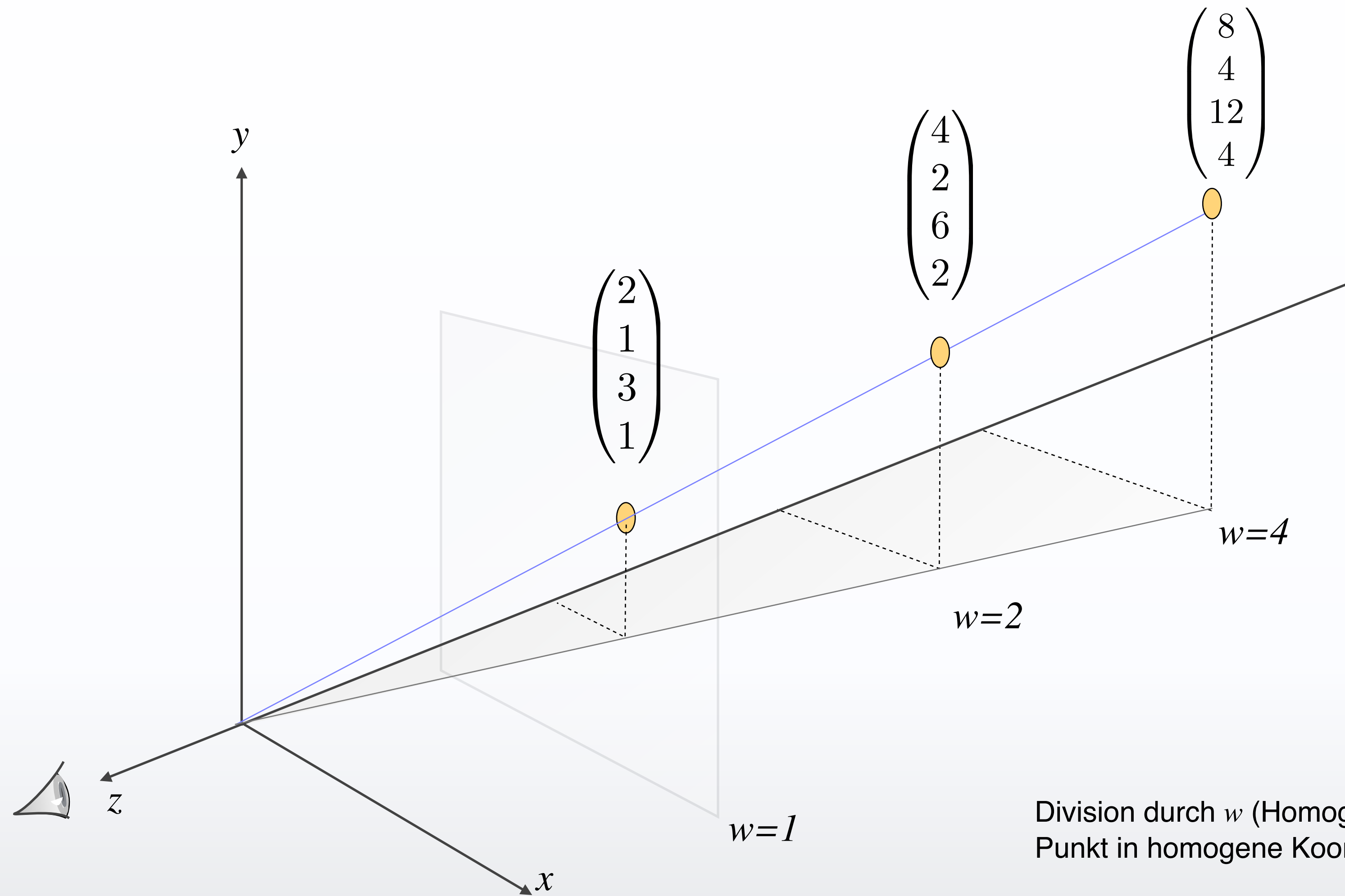
■ Sichtvolumen

- Center of Projection (*Augpunkt*)
- Field of View H (*fov*)
- Aspect ratio (*Seitenverhältnis*)
- Near Clipping Plane (*near*)
- Far Clipping Plane (*far*)
- üblich viewplane = nearplane
- Bei symmetrischen Sichtvolumen
 - $r=-l, t=-b$



```
void gluPerspective(GLdouble fov, GLdouble aspect, GLdouble near, GLdouble far);
```

Die Formel der reinen Projektion

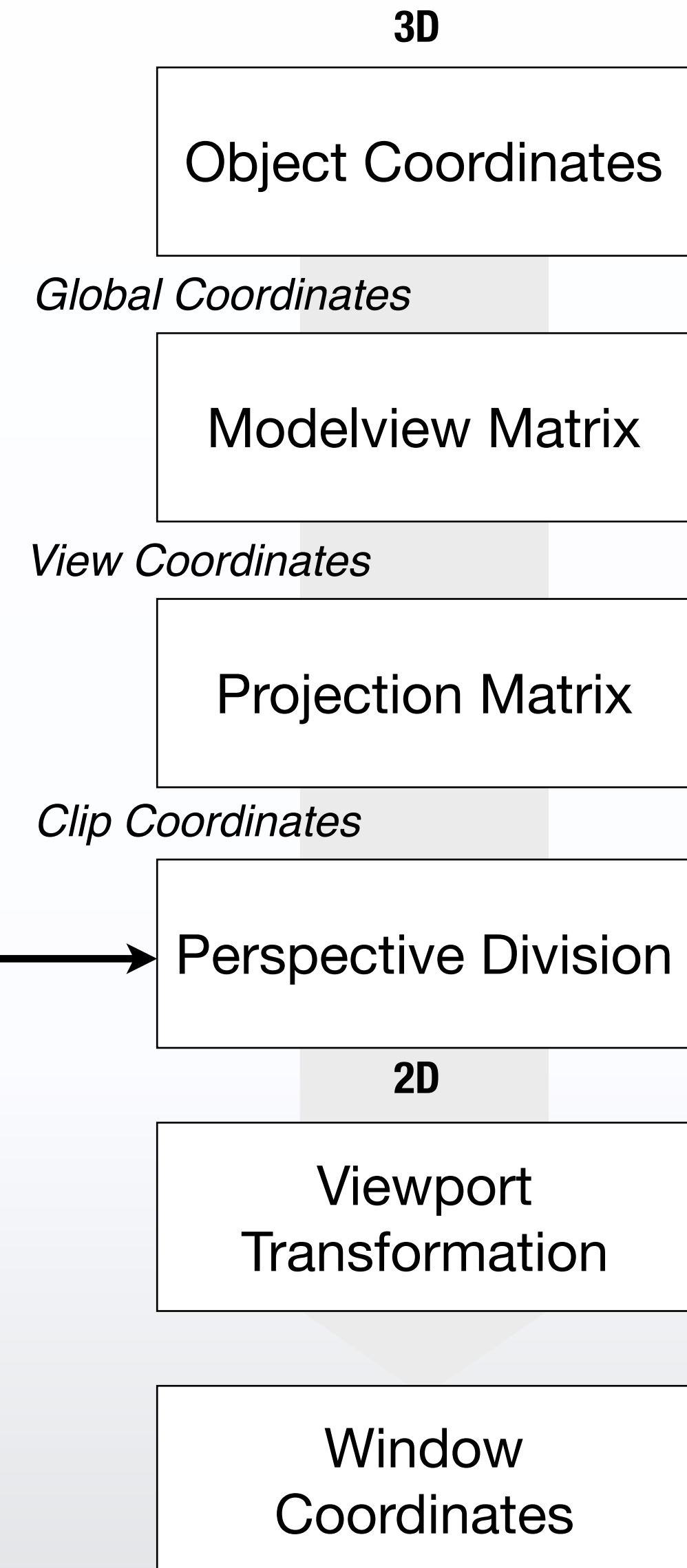


Perspective Projection

- Erst die Division durch die w -Komponente werden die x, y Werte projiziert
- Division durch w (homogenisierung **ergibt** erst Projektion)
- Dies wird erst spät in der Pipeline durchgeführt

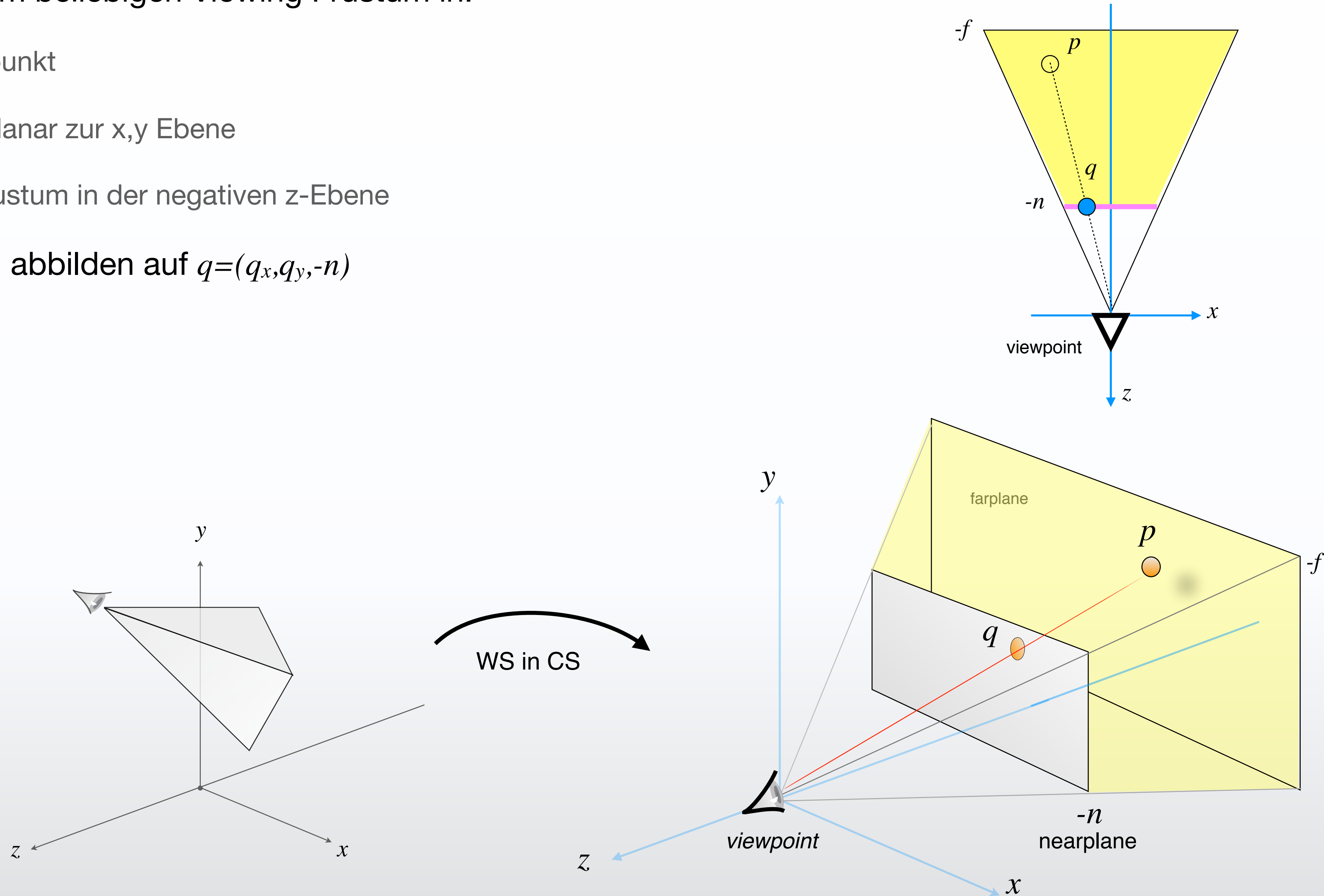
$$\begin{pmatrix} p_x \\ p_y \\ p_z \\ p_z/n \end{pmatrix} \rightarrow \begin{pmatrix} \frac{n}{p_z} \cdot p_x \\ \frac{n}{p_z} \cdot p_y \\ n \\ 1 \end{pmatrix}$$

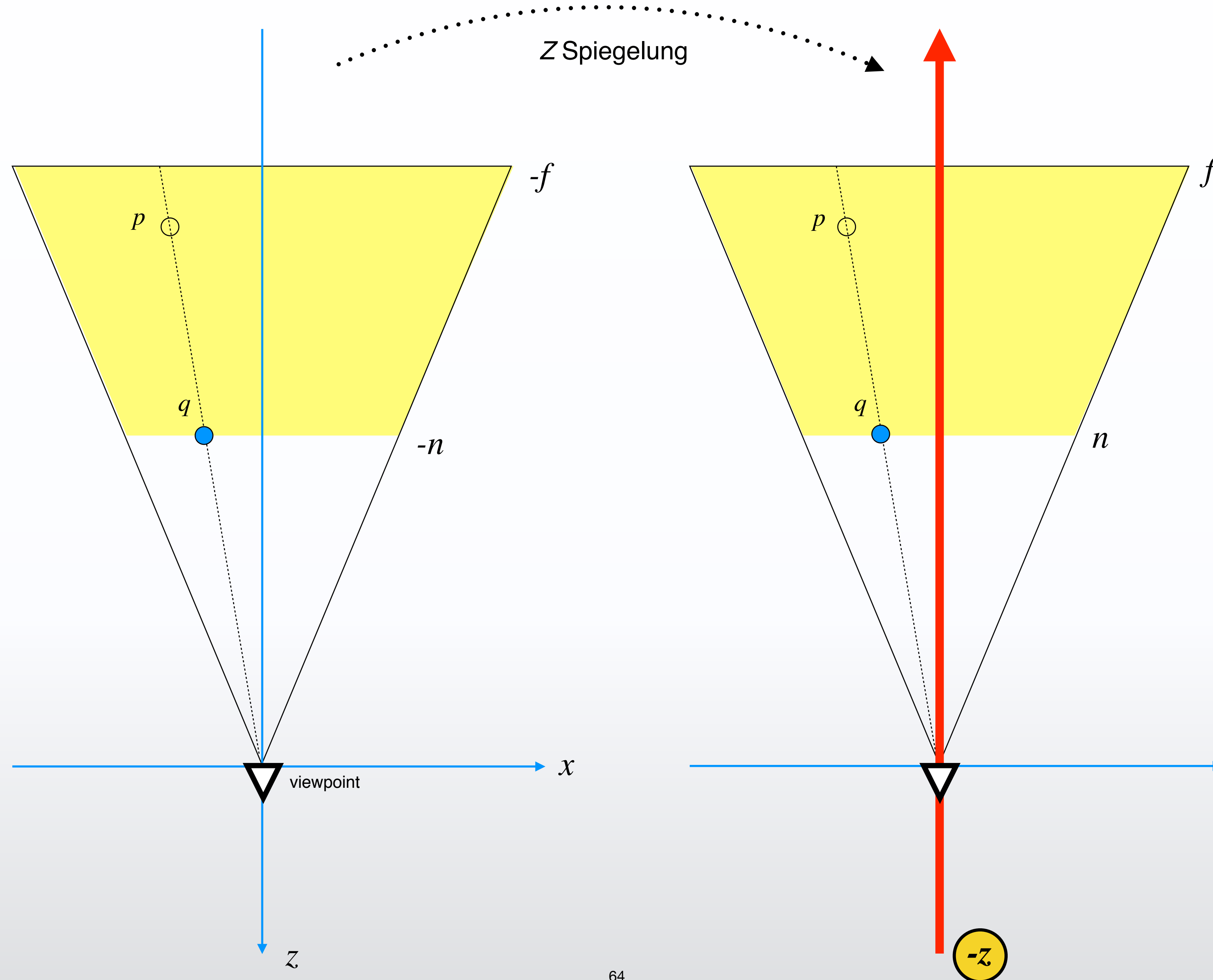
DIVISION DURCH W



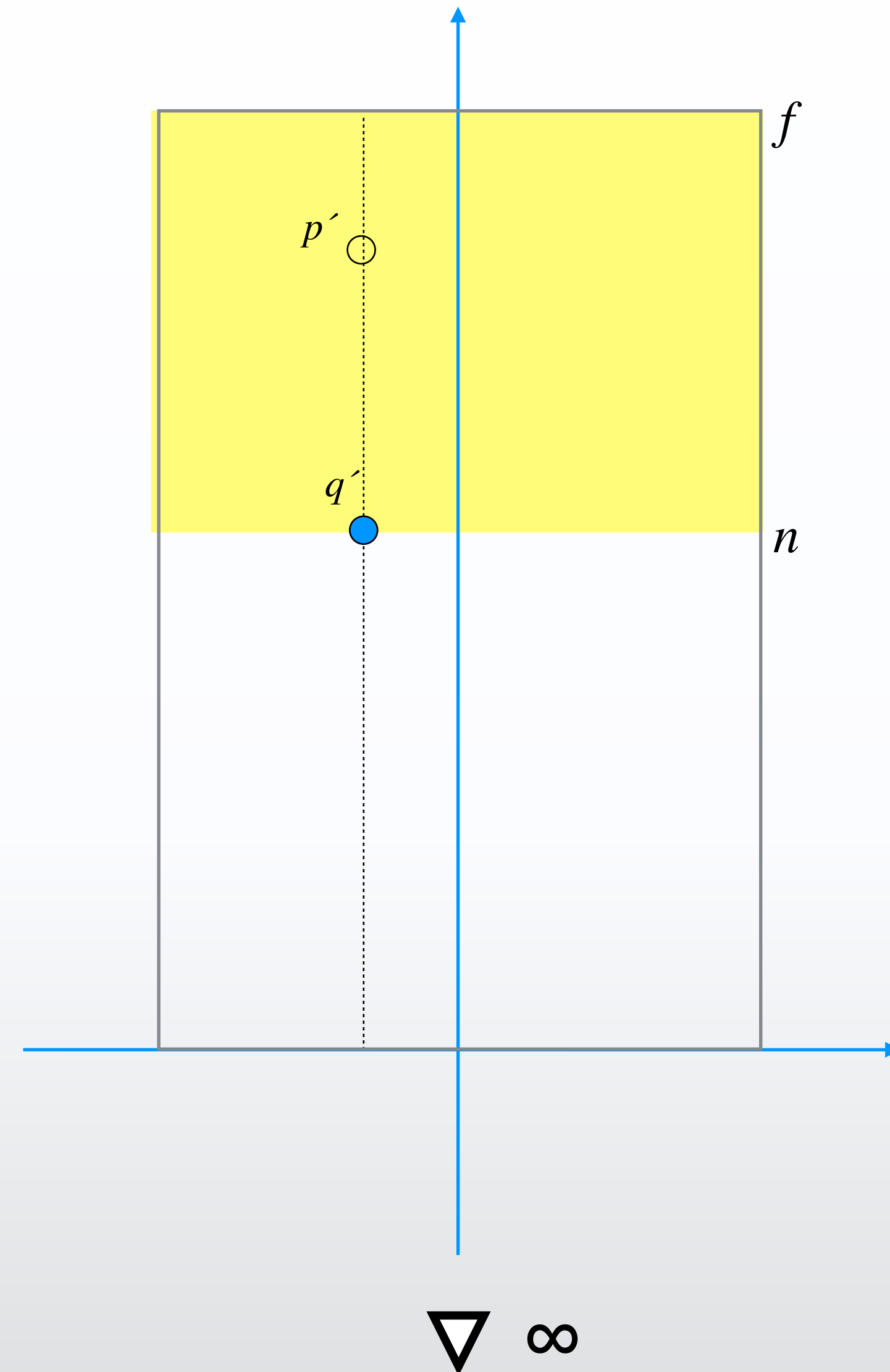
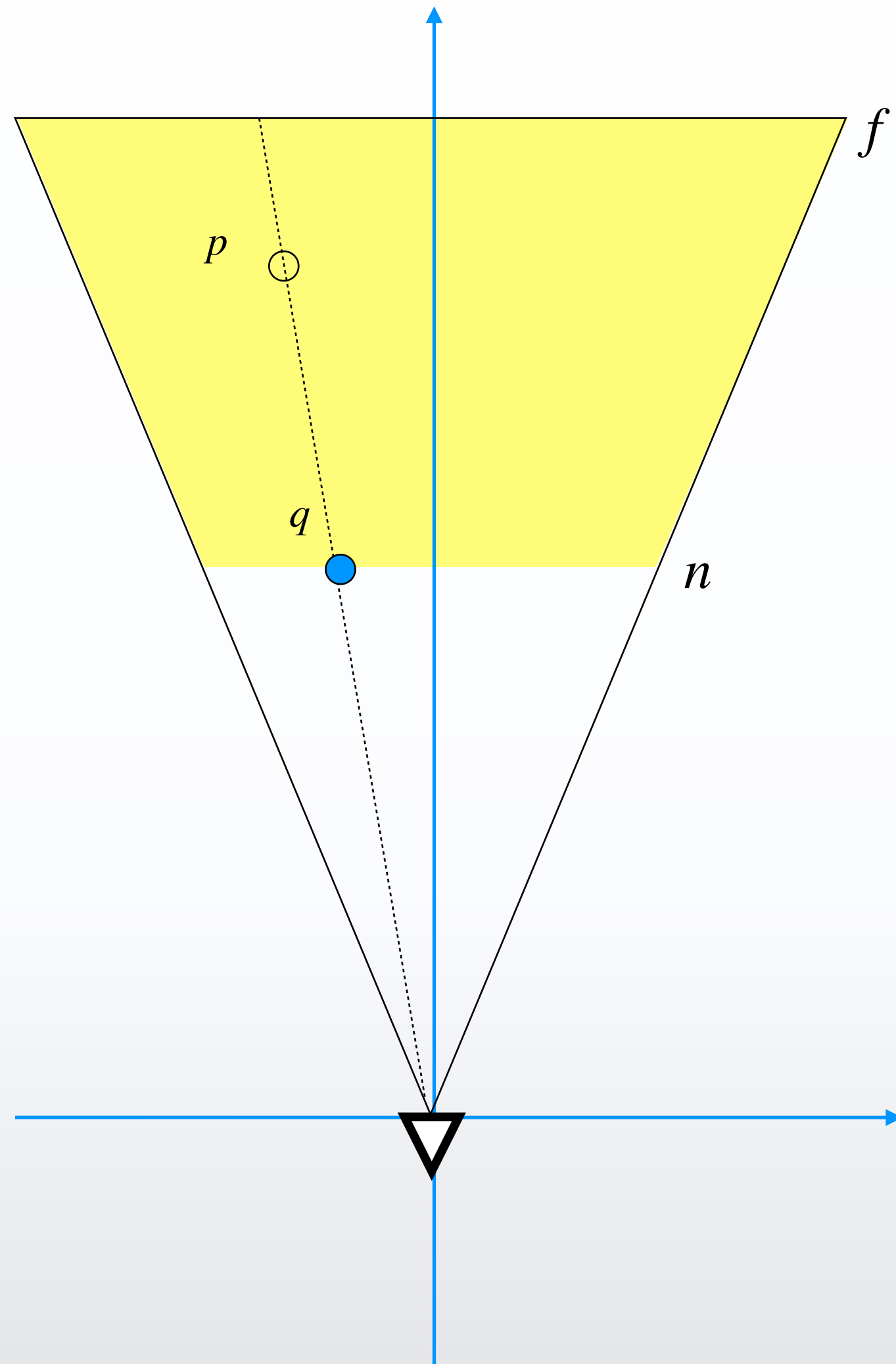
■ Transformation vom beliebigen Viewing Frustum in:

- Viewpoint im Nullpunkt
 - UV-Viewplane koplanar zur x,y Ebene
 - damit: Viewing Frustum in der negativen z-Ebene
- ## ■ Punkt $p=(p_x, p_y, p_z)$: abbilden auf $q=(q_x, q_y, -n)$

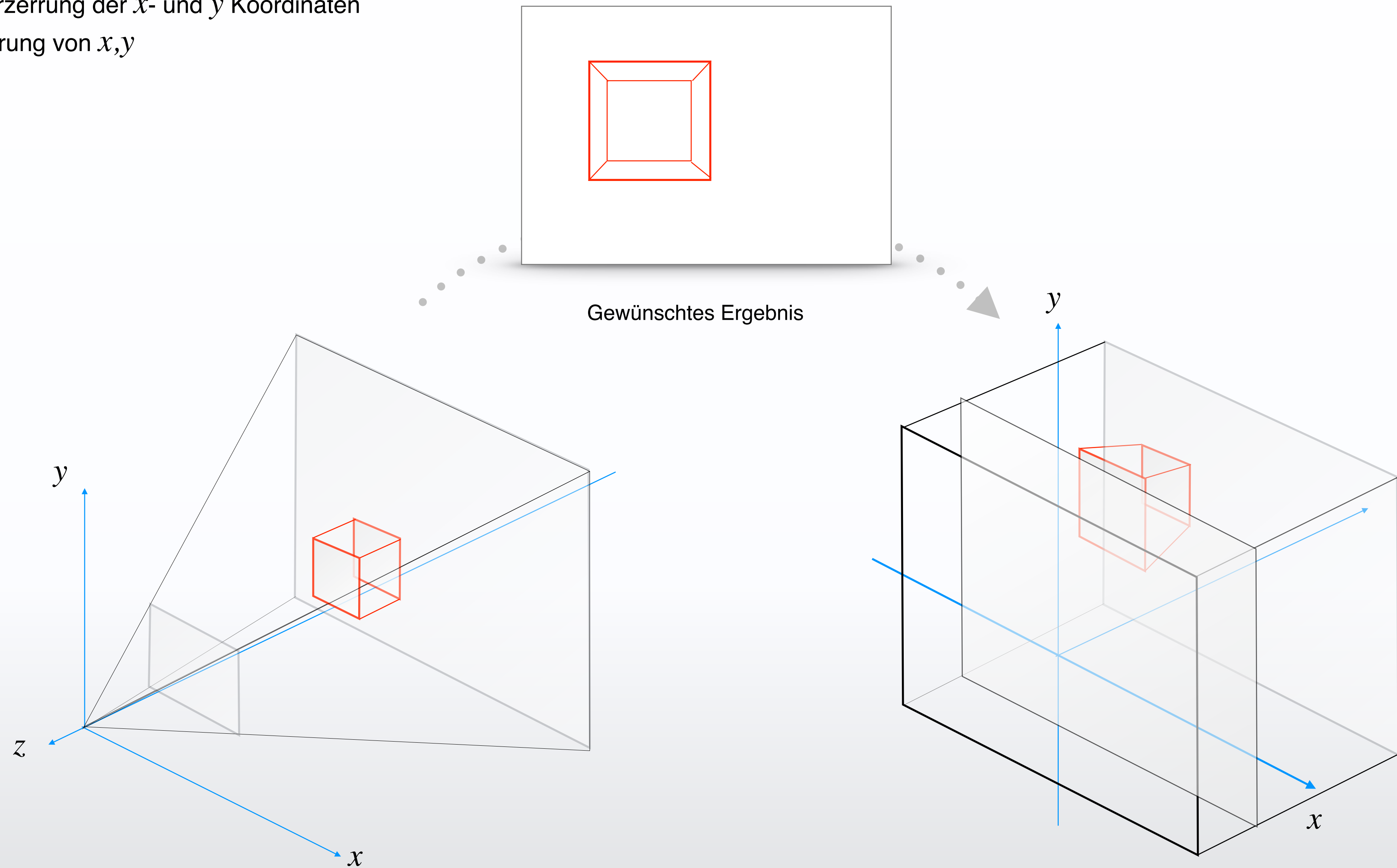




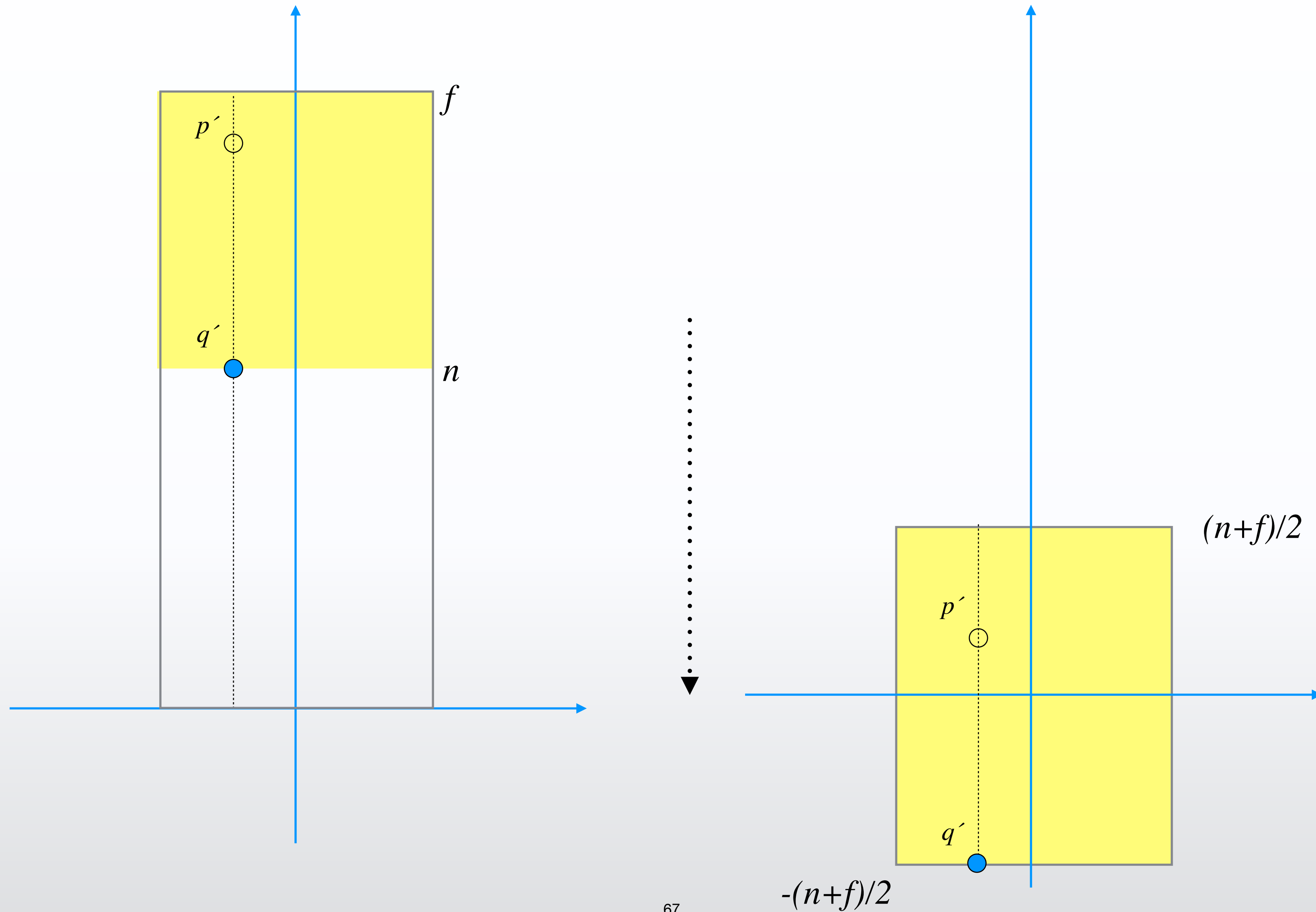
Perspektivische Verzerrung der x - und y Koordinaten
 z -abhängige Skalierung von x, y



Perspektivische Verzerrung der x - und y Koordinaten
 z -abhängige Skalierung von x,y

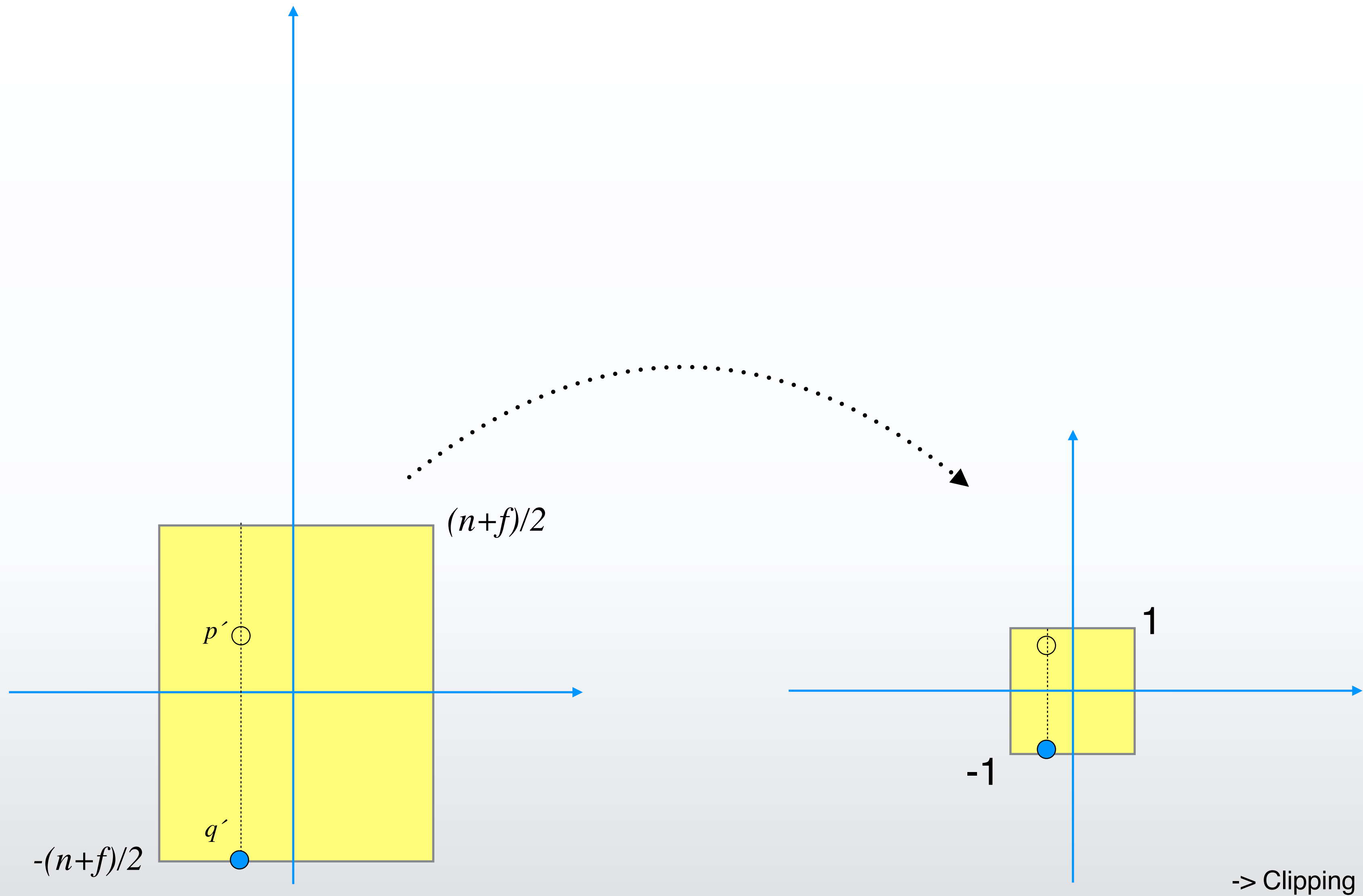


Analog zur orthogonalen Transformation, zentrieren: Translation (nur z -Werte)



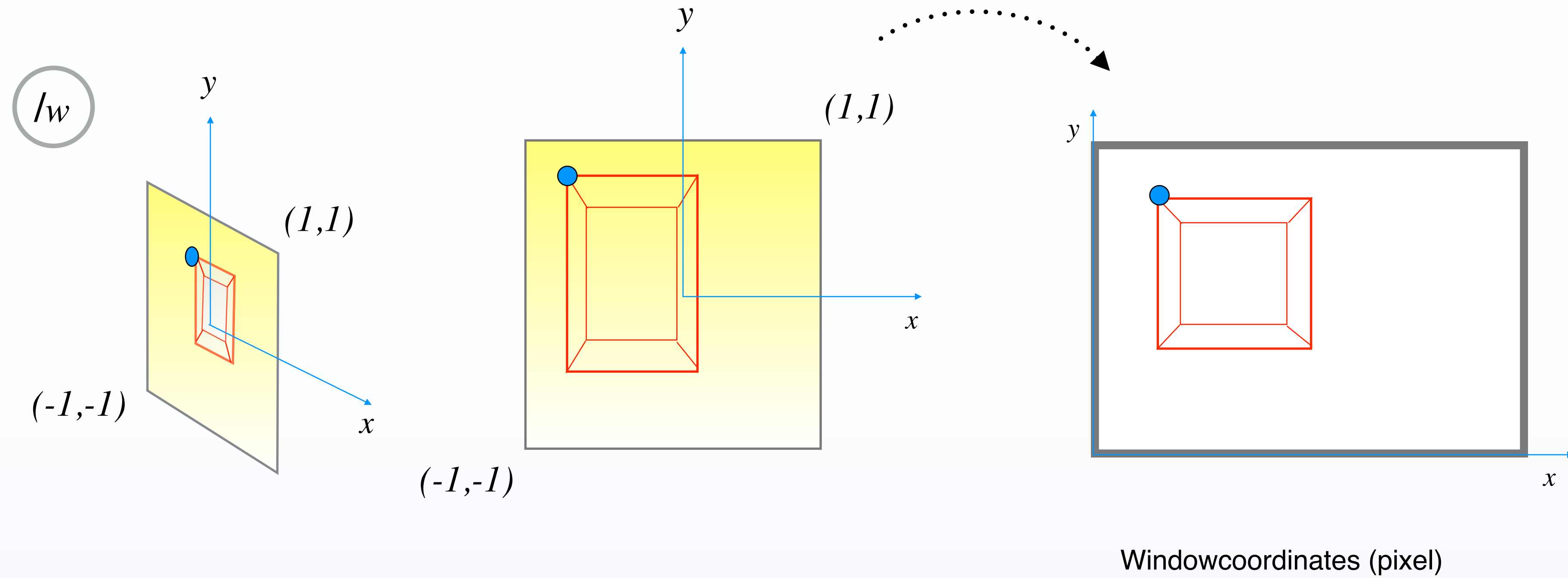
Perspektivische Projektion

Analog zur orthogonalen Transformation, Skalierung auf Einheitswürfel



Perspective Projection

Analog zur orth. Transformation : Viewport Transformation



Analog zur Orthogonalen Transformation, z Werte spiegeln

z Achse spiegeln :

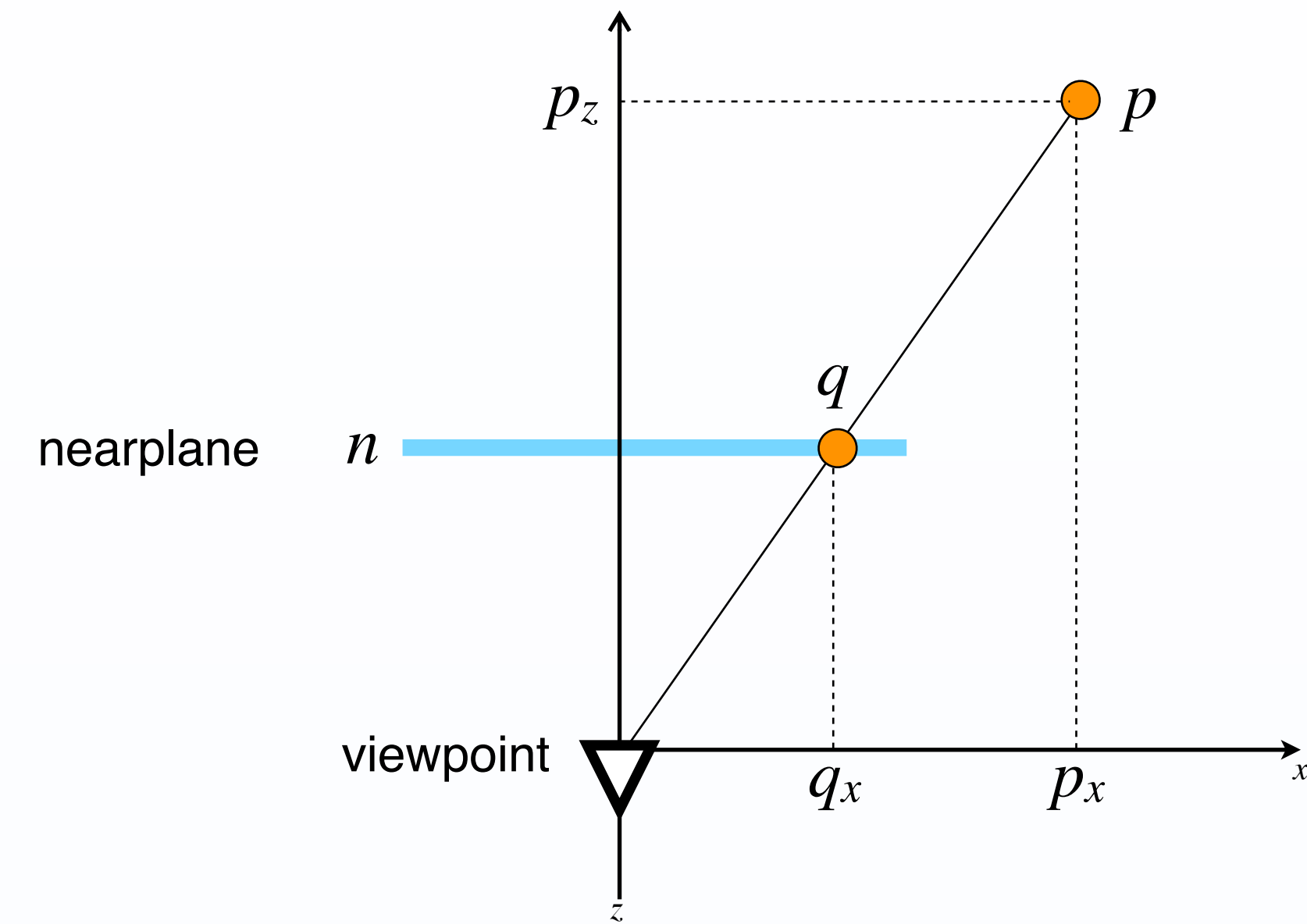
$$M = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

1 Strahlensatz

$$\frac{q_x}{p_x} = \frac{n}{p_z} \rightarrow q_x = n \cdot \frac{p_x}{p_z}$$

$$\frac{q_y}{p_y} = \frac{n}{p_z} \rightarrow q_y = n \cdot \frac{p_y}{p_z}$$

2 Wie kriegen wir das in einer 4X4 Matrix rein



$$\frac{q_x}{p_x} = \frac{n}{p_z} \rightarrow q_x = n \cdot \frac{p_x}{p_z}$$

$$\frac{q_y}{p_y} = \frac{n}{p_z} \rightarrow q_y = n \cdot \frac{p_y}{p_z}$$

$$\begin{pmatrix} n & 0 & 0 & 0 \\ 0 & n & 0 & 0 \\ 0 & 0 & n & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} p_x \\ p_y \\ p_z \\ 1 \end{pmatrix} = \begin{pmatrix} n \cdot p_x \\ n \cdot p_y \\ n \cdot p_z \\ p_z \end{pmatrix} \xrightarrow{\text{homog.}} \frac{1}{p_z} \rightarrow \begin{pmatrix} (n \cdot p_x)/p_z \\ (n \cdot p_y)/p_z \\ n \\ 1 \end{pmatrix}$$

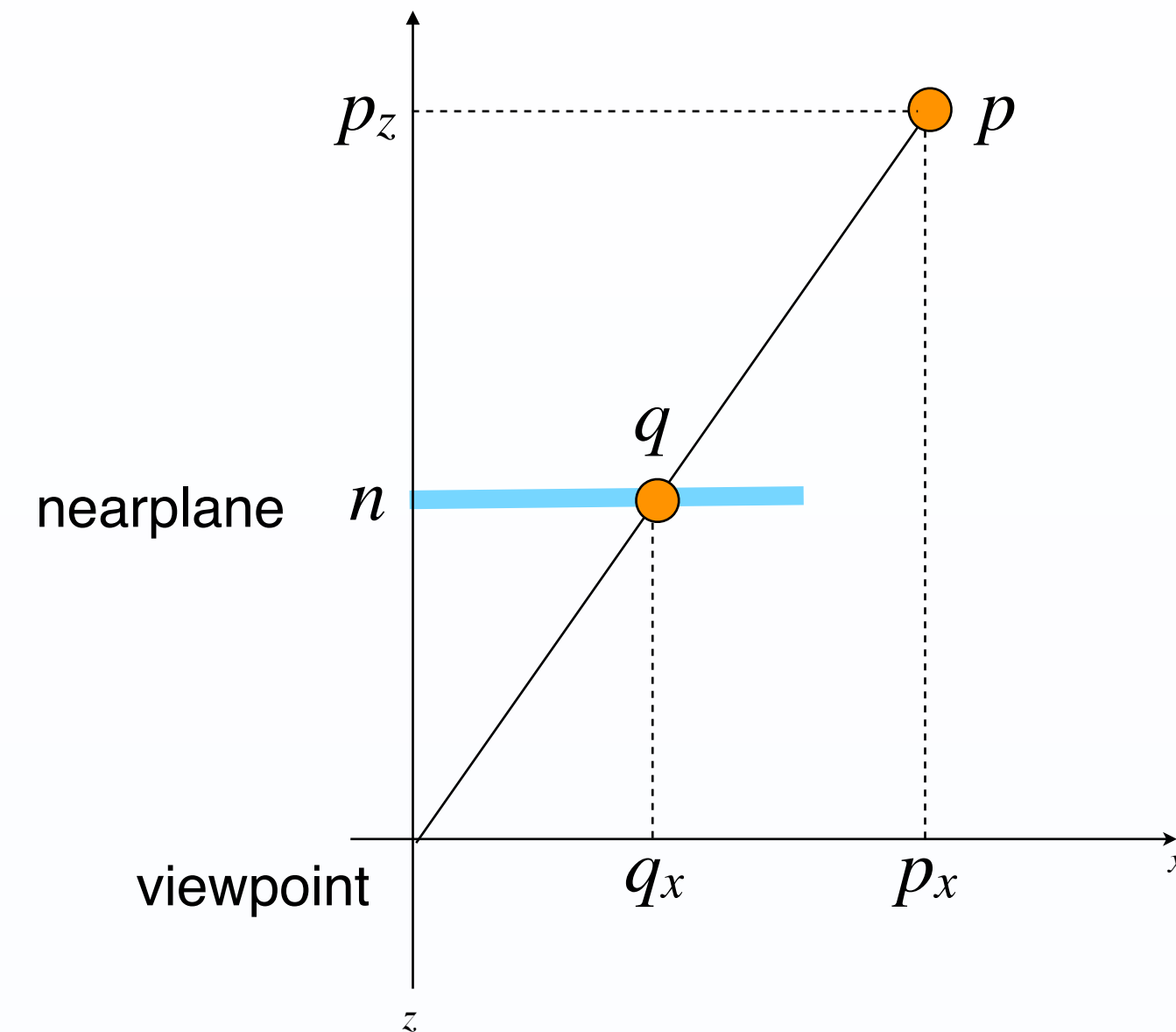
! PROBLEM ?

- Was passiert mit den z Werten
- Integrierbar in einer 4x4 Matrix
- Tiefeninformation (Sichtbarkeit) soll nicht verlorengehen
- Ansatz:

$$\begin{pmatrix} q_x \\ q_y \\ q_z \\ 1 \end{pmatrix} \rightarrow \begin{pmatrix} n \cdot p_x / p_z \\ n \cdot p_y / p_z \\ ? \\ 1 \end{pmatrix}$$

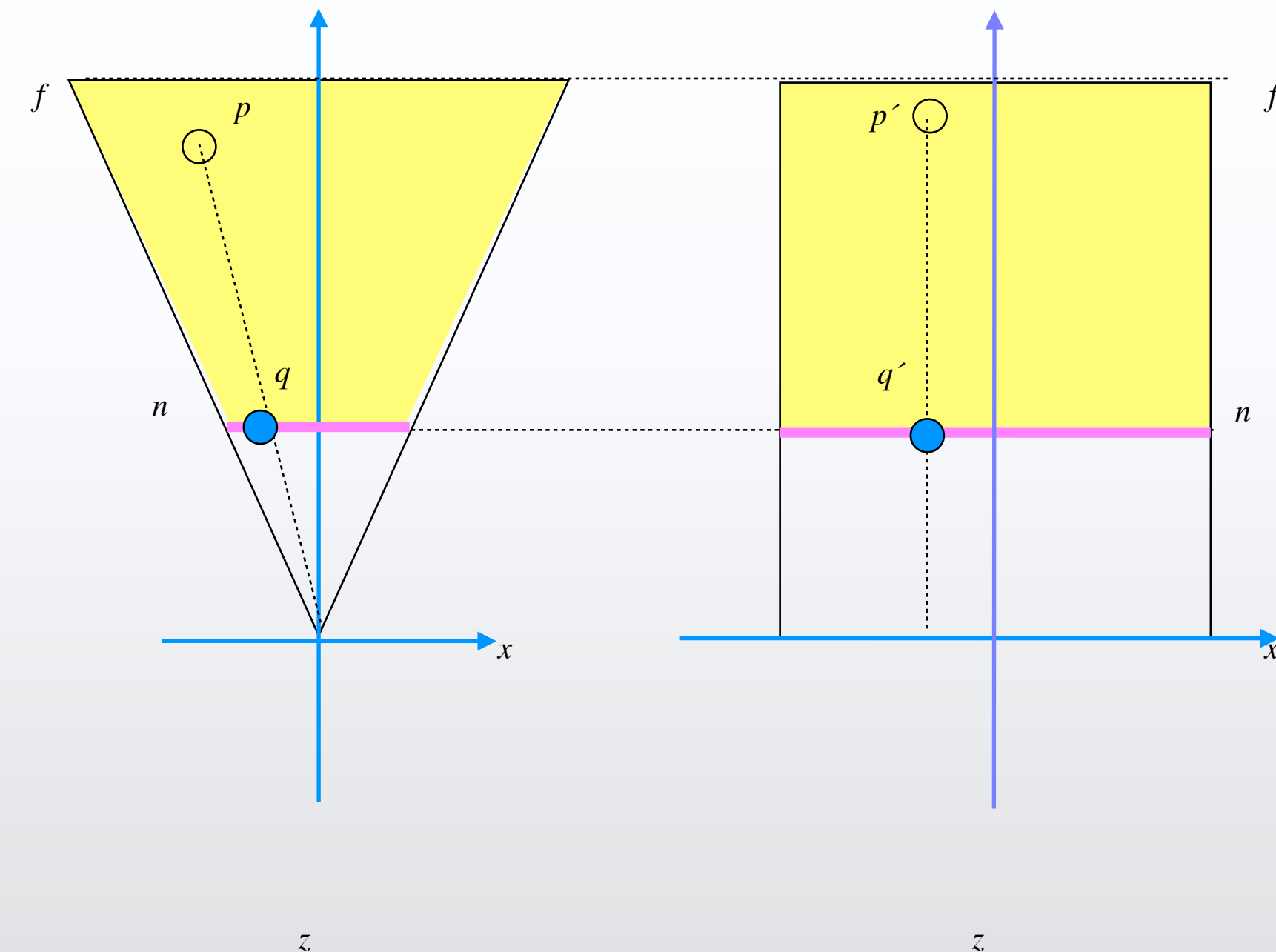
$$\begin{pmatrix} q_x \\ q_y \\ q_z \\ 1 \end{pmatrix} \rightarrow \begin{pmatrix} np_x / p_z \\ np_y / p_z \\ \sim p_z \\ 1 \end{pmatrix}$$

Ein Wert welcher Information über die Tiefe enthält.



- x und y Werte gemäß Projektion verzerren
- Die z Werte müssen folgende Bedingungen erfüllen
 - Die near Werte (n) werden auf near (n) abgebildet
 - Die far Werte (f) werden auf far (f) abgebildet
 - Alle z Werte dazwischen behalten ihre Ordnung (Monoton steigend) / Proportional p_z
- Alles muss in einer 4X4 Matrix integrierbar sein

$$\begin{pmatrix} q_x \\ q_y \\ q_z \\ 1 \end{pmatrix} = \begin{pmatrix} np_x/p_z \\ np_y/p_z \\ \sim p_z \\ 1 \end{pmatrix}$$



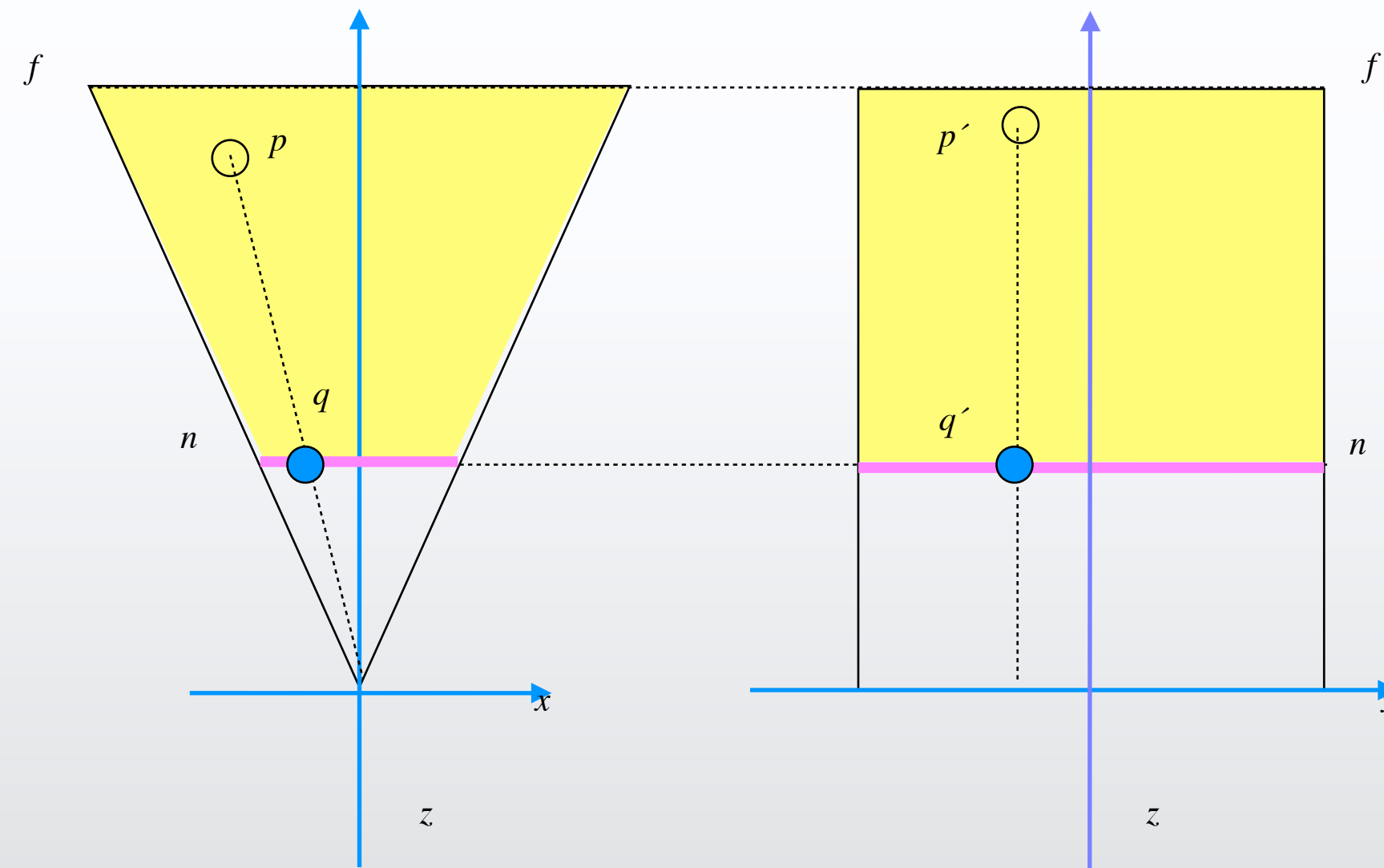
- Die z Werte müssen folgende Bedingungen erfüllen
 - Die near Werte (n) werden auf near (n) abgebildet
 - Die far Werte (f) werden auf far (f) abgebildet
 - Alle z Werte dazwischen behalten ihre Ordnung (Monoton steigend) / Proportional p_z
- Alles muss in einer 4X4 Matrix integrierbar sein

$$\frac{n \cdot f}{p_z} \begin{cases} p_z = n \rightarrow f \\ p_z = f \rightarrow n \end{cases} \rightarrow (n + f) - \frac{n \cdot f}{p_z}$$

$$\begin{pmatrix} n & 0 & 0 & 0 \\ 0 & n & 0 & 0 \\ 0 & 0 & n + f & -n \cdot f \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} p_x \\ p_y \\ p_z \\ 1 \end{pmatrix} = \begin{pmatrix} n \cdot p_x \\ n \cdot p_y \\ (n + f) \cdot p_z - n \cdot f \\ p_z \end{pmatrix}$$

1 Projektion von x,y Koordinaten (z-abhängige Skalierung)

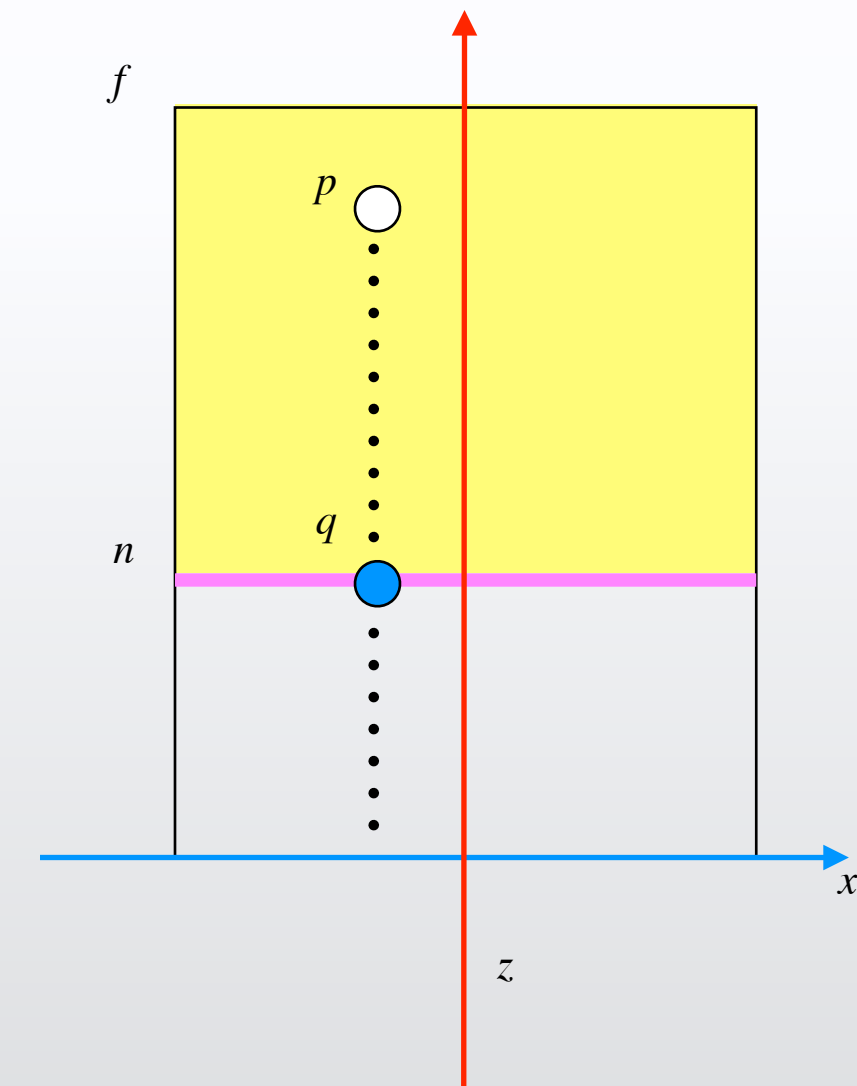
$$M = \begin{pmatrix} n & 0 & 0 & 0 \\ 0 & n & 0 & 0 \\ 0 & 0 & n + f & -n \cdot f \\ 0 & 0 & 1 & 0 \end{pmatrix}$$



$$\begin{pmatrix} n & 0 & 0 & 0 \\ 0 & n & 0 & 0 \\ 0 & 0 & n+f & -n \cdot f \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} p_x \\ p_y \\ p_z \\ 1 \end{pmatrix} = \begin{pmatrix} n \cdot p_x \\ n \cdot p_y \\ (n+f) \cdot p_z - n \cdot f \\ p_z \end{pmatrix}$$

$$\begin{pmatrix} n \cdot p_x \\ n \cdot p_y \\ (n+f) \cdot p_z - n \cdot f \\ p_z \end{pmatrix} \Rightarrow \begin{pmatrix} n \cdot p_x / p_z \\ n \cdot p_y / p_z \\ n+f - n \cdot f / p_z \\ 1 \end{pmatrix}$$

homogenisiert

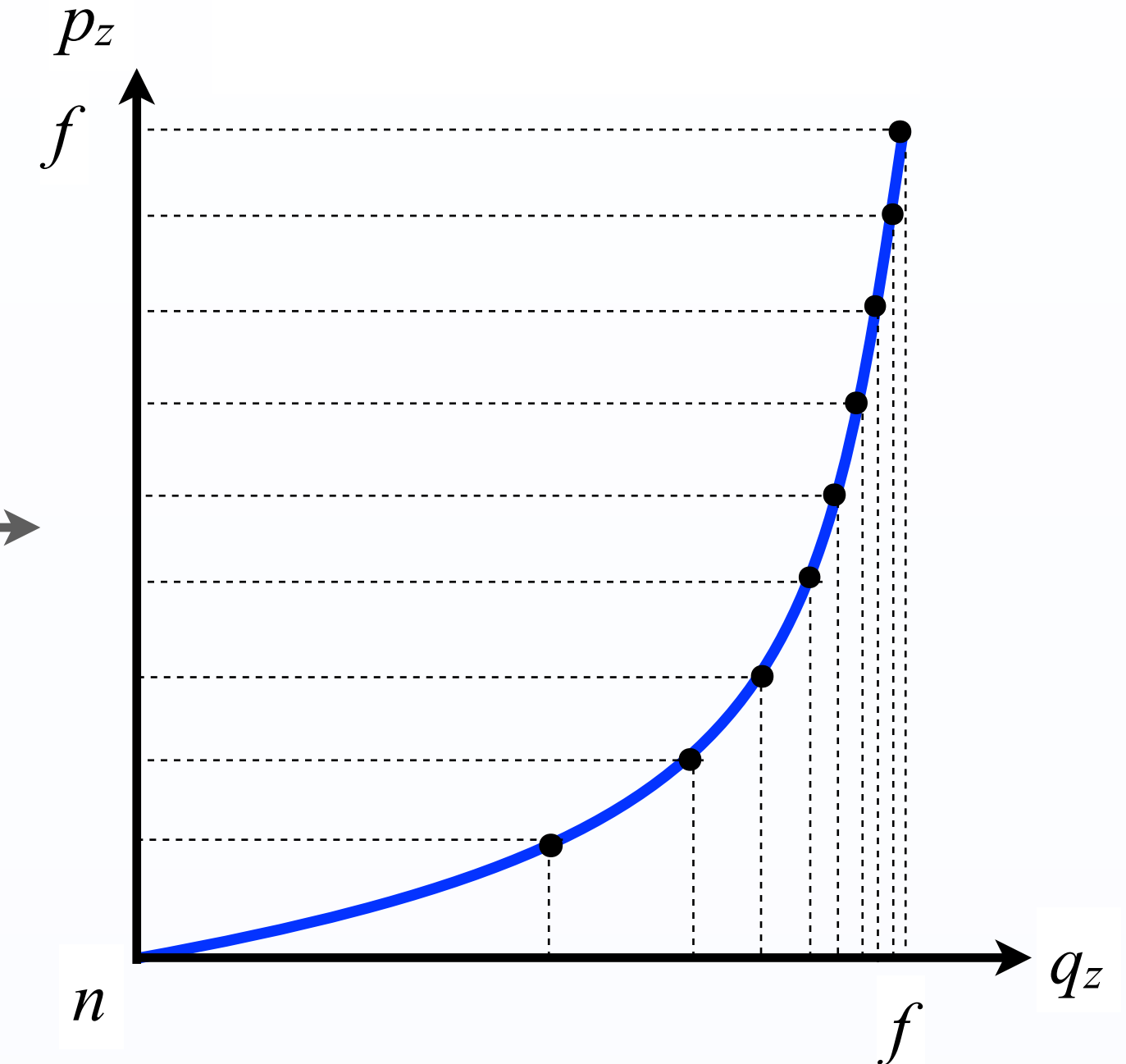


$$\begin{pmatrix} q_x \\ q_y \\ q_z \\ w \end{pmatrix} = \begin{pmatrix} np_x/p_z \\ np_y/p_z \\ n + f - nf/p_z \\ 1 \end{pmatrix}$$

- nicht lineare Skalierung der Z-Koordinaten
- n wird auf n abgebildet
- f wird auf f abgebildet

? Erkenntnis?

- „Vorne“ gibt es eine höhere Auflösung als „hinten“
- Achtung: Flackern bei fernen Objekten
- Was bedeutet das praktisch?
- Es ist sehr wichtig near, und far- Clippingplane intelligent zu bestimmen.



$$q_z = n + f - n \cdot f / p_z$$

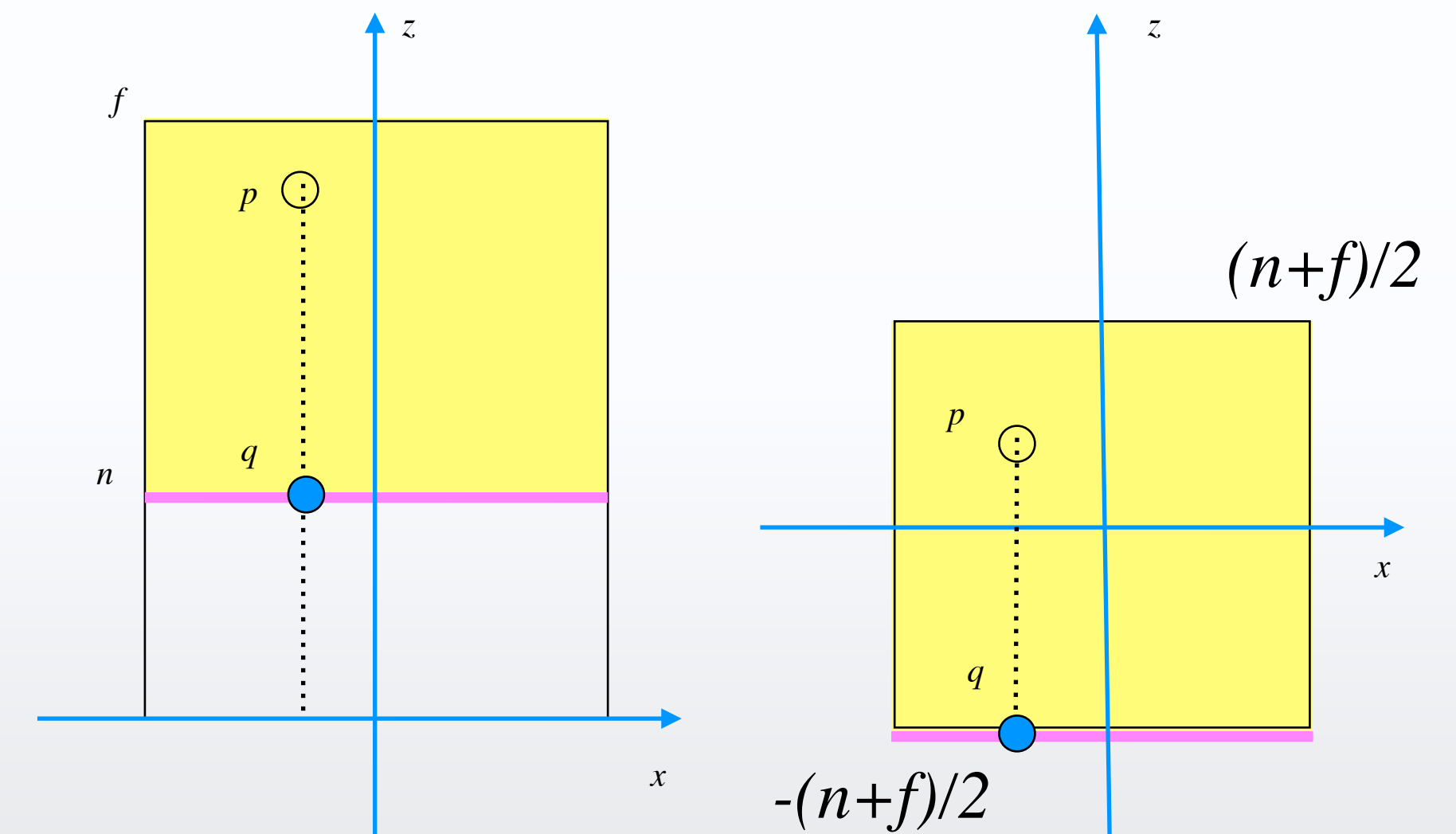
- 2 Da Quader schon bzgl. x , y zentriert nur noch um z verschieben

$$M_T = \begin{pmatrix} 1 & 0 & 0 & -\frac{l+r}{2} \\ 0 & 1 & 0 & -\frac{b+t}{2} \\ 0 & 0 & 1 & -\frac{f+n}{2} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Translation bekannt aus orthogonaler Projektion

$$M = \begin{pmatrix} n & 0 & 0 & 0 \\ 0 & n & 0 & 0 \\ 0 & 0 & \frac{n+f}{2} & -n \cdot f \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

Eingebaut in die bisherige Matrix
Von rechts ran multiplizieren



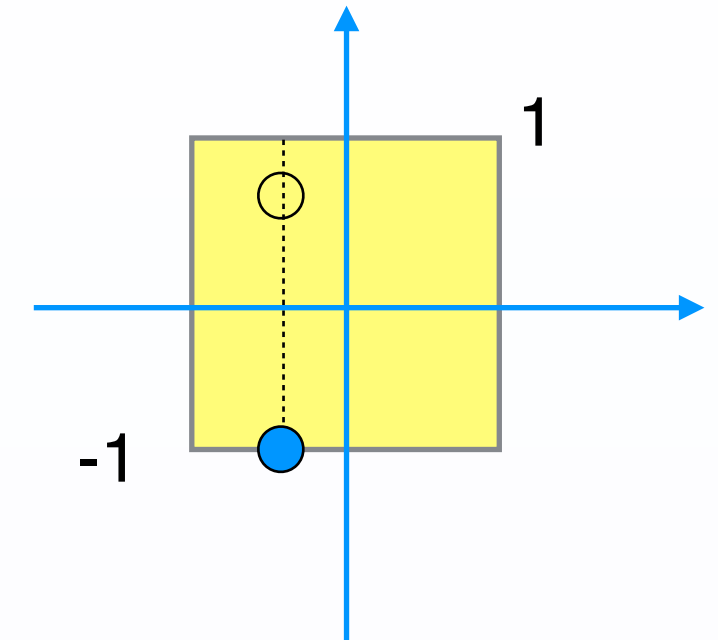
3 Skalierung der Werte zu einem Einheitswürfel

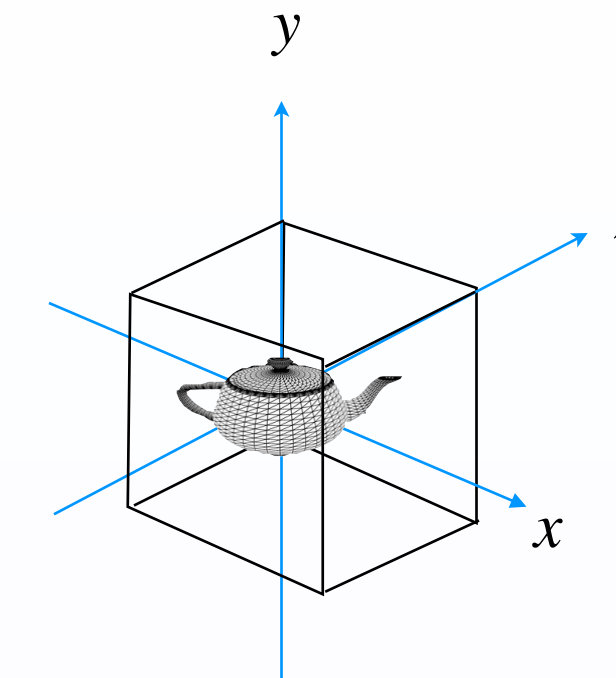
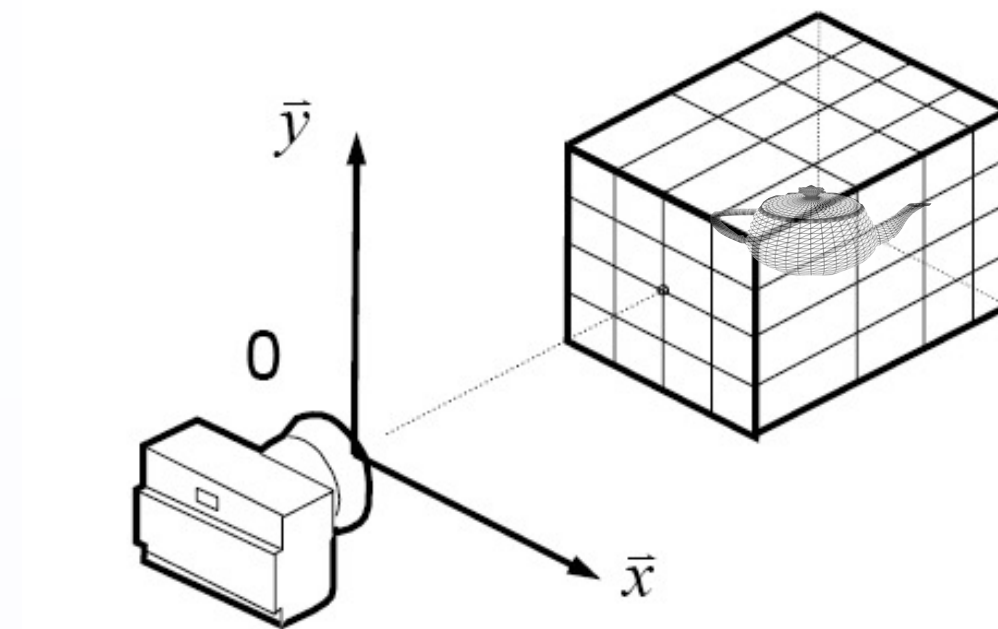
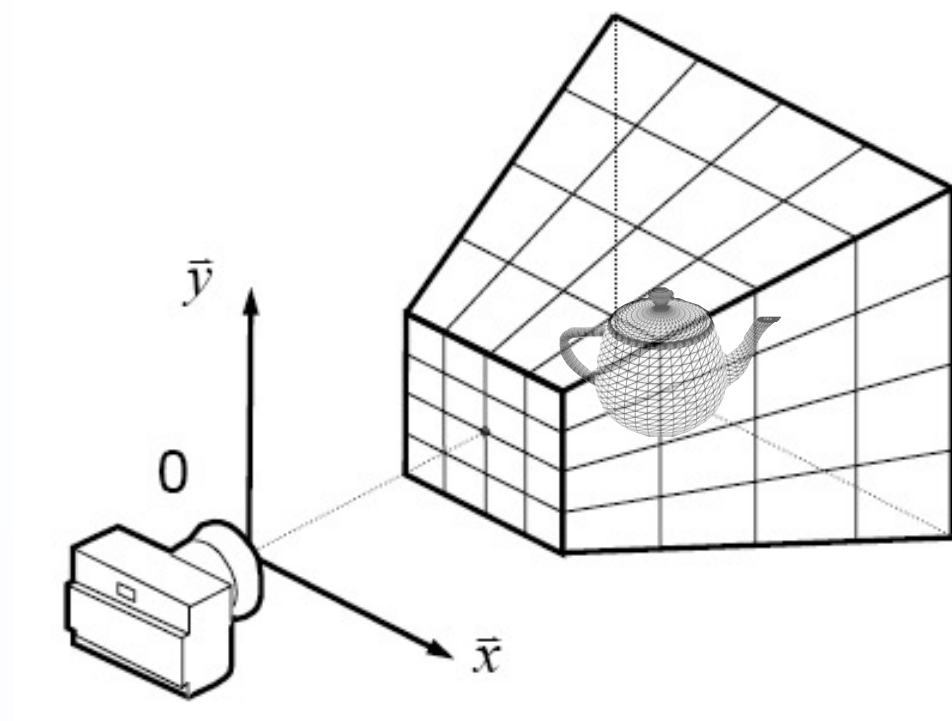
$$M_S = \begin{pmatrix} \frac{2}{r-l} & 0 & 0 & 0 \\ 0 & \frac{2}{t-b} & 0 & 0 \\ 0 & 0 & \frac{2}{f-n} & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Skalierung bekannt aus orthogonaler Projektion

$$M = \begin{pmatrix} \frac{2n}{r-l} & 0 & 0 & 0 \\ 0 & \frac{2n}{t-b} & 0 & 0 \\ 0 & 0 & -\frac{n+f}{f-n} & -\frac{2n \cdot f}{f-n} \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

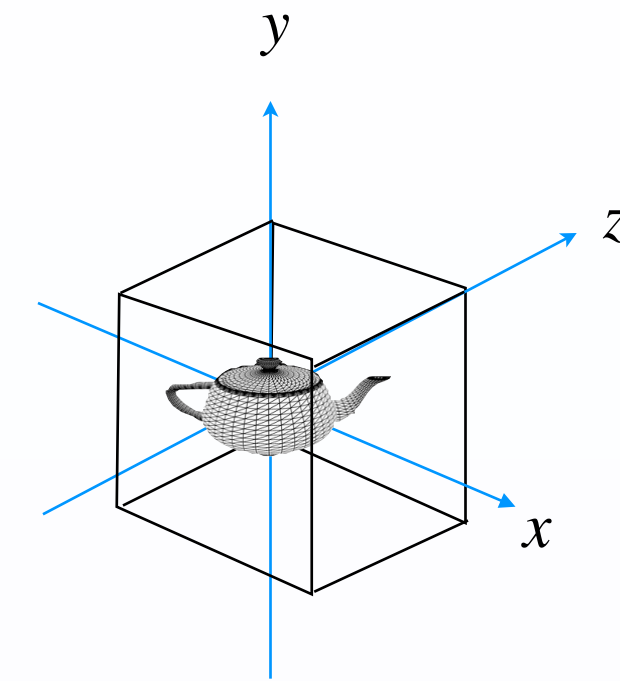
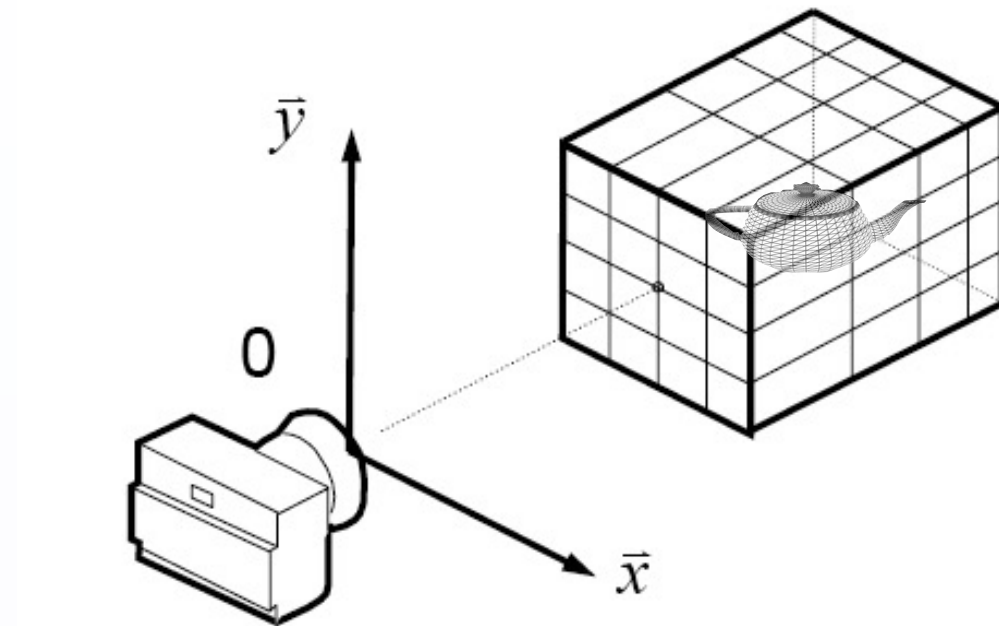
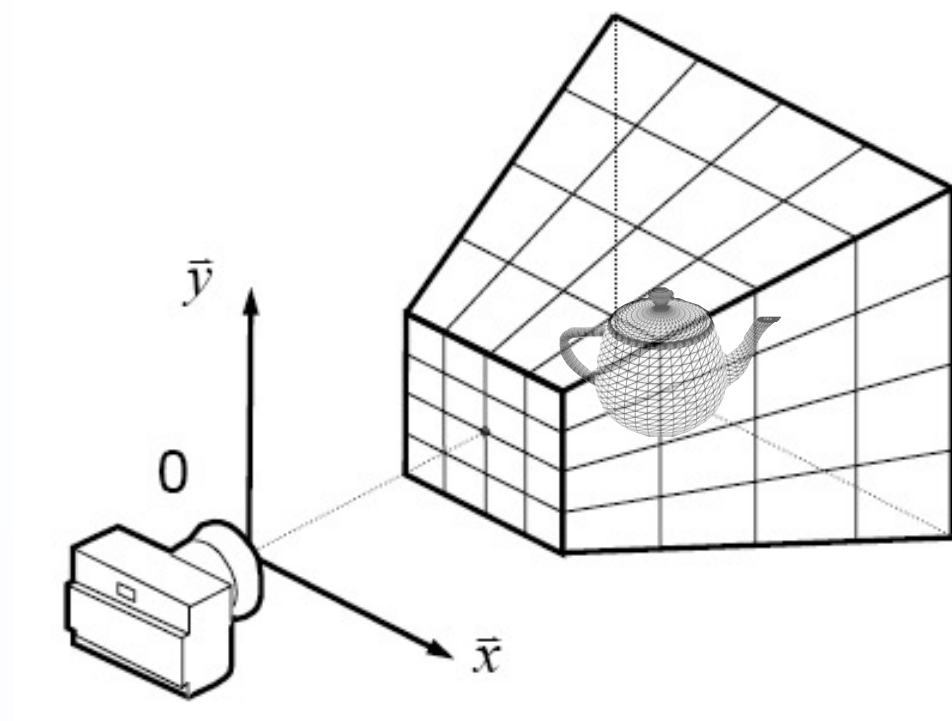
Eingebaut in die bisherige Matrix
Von rechts ran multiplizieren





$$M = \begin{pmatrix} \frac{2n}{r-l} & 0 & 0 & 0 \\ 0 & \frac{2n}{t-b} & 0 & 0 \\ 0 & 0 & -\frac{n+f}{f-n} & -\frac{2n \cdot f}{f-n} \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

$$q = \underbrace{M_{Ortho} \cdot M_{Persp} \cdot M_{R \sim L}}_M \cdot V \cdot T \cdot \dots \cdot S \cdot R \cdot p$$

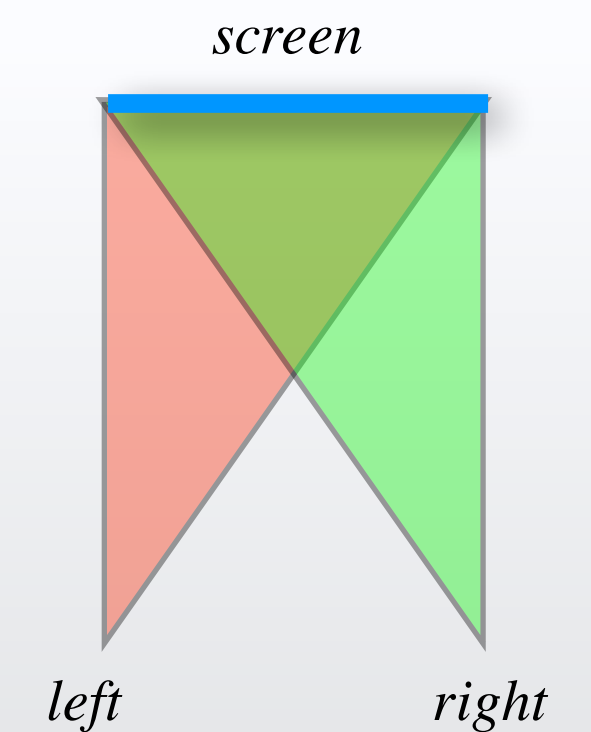
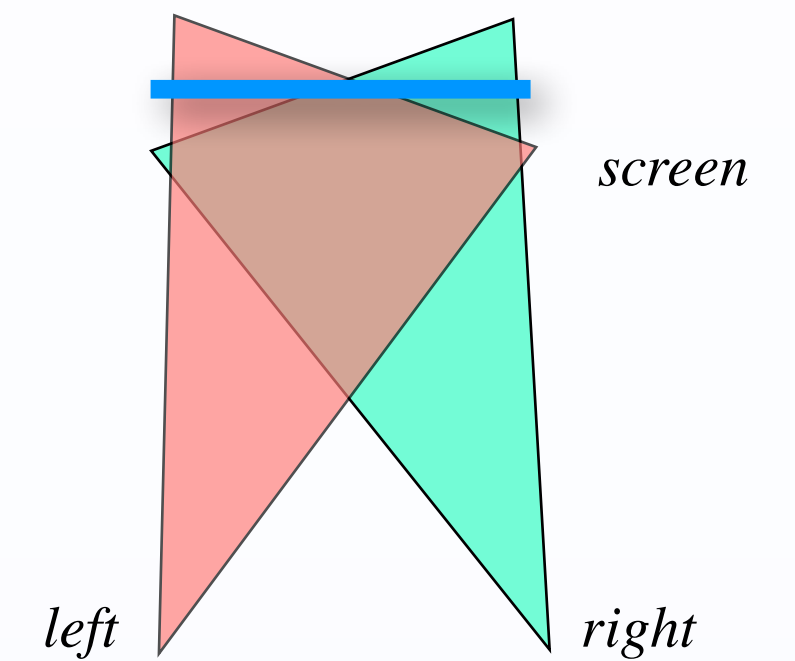
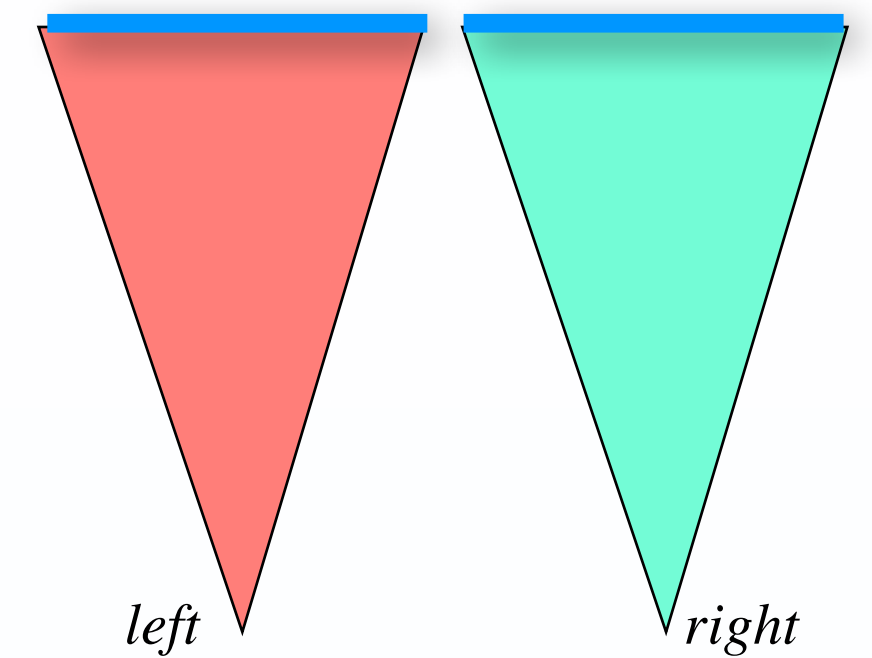


$$M = \begin{pmatrix} \frac{2n}{r-l} & 0 & \frac{l+r}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{b+t}{t-b} & 0 \\ 0 & 0 & -\frac{n+f}{f-n} & -\frac{2n \cdot f}{f-n} \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

$$q = M_{Ortho} \cdot M_{Persp} \cdot M_{R \sim L} \cdot V \cdot T \cdot \dots \cdot S \cdot R \cdot p$$

$$M$$

- Stereoskopische Filme
- Zwei Kameras mit einem Abstand aufstellen
- Jedes Frame für das rechte und linke Auge rendern
- Frage ist: Wie erzeugen wir ein Bild daraus?
- Korrespondierende Pixel (Rechts / Links)
- Rotation der Kamera ist eine schlechte Lösung
- Lösung: Scherung des Sichtvolumens !
- Achtung: geht meistens nicht in den üblichen 3D Programmen
- Sichtvolumen muss selber erzeugt werden



Sei $q_x = \frac{l+r}{2}$ Genau in der Mitte

Und wird damit auf die z-Achse abgebildet

Und p ein beliebiger Punkt

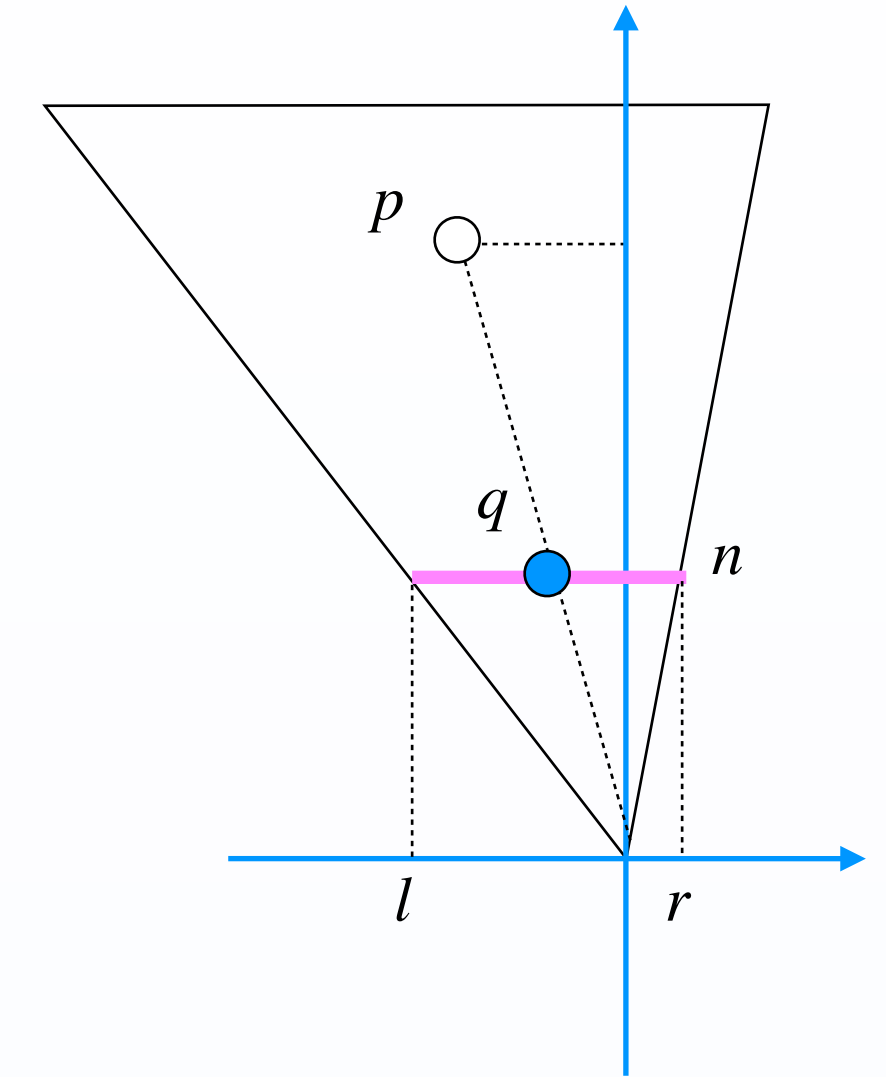
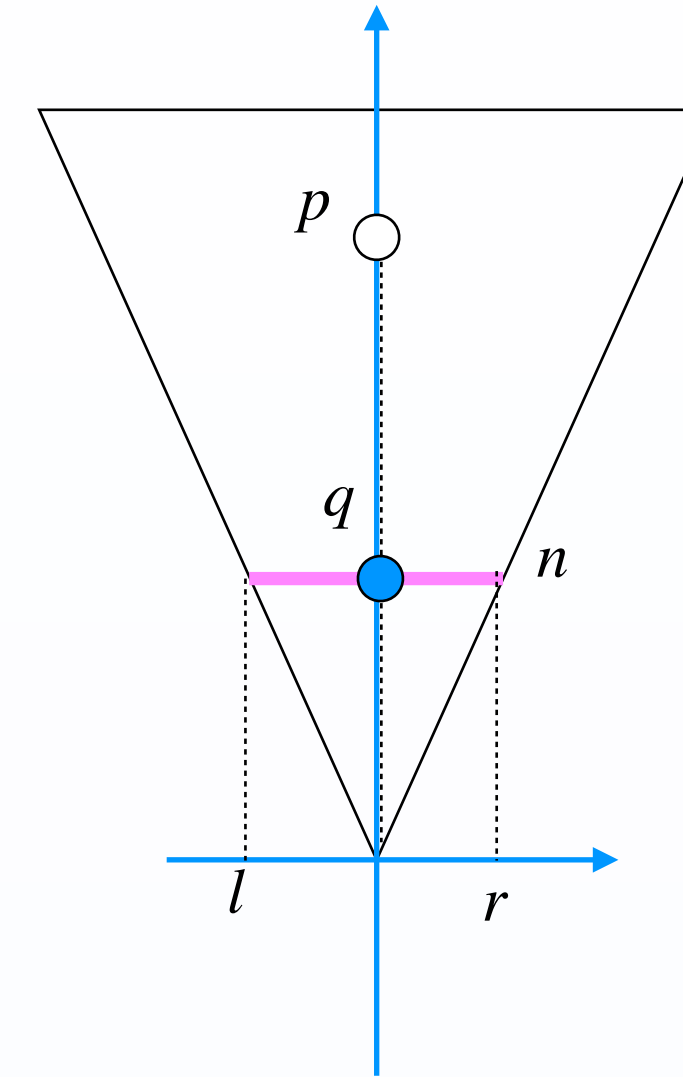
$$\frac{q_x}{p_x} = \frac{n}{p_z}$$

$$p_x = \frac{q_x}{n} \cdot p_z$$

$$p_x = \frac{l+r}{2n} \cdot p_z \quad \text{Benötigte Translation um } x$$

$$p_y = \frac{t+b}{2n} \cdot p_z \quad \text{Benötigte Translation um } y$$

$$M_{SH} = \begin{pmatrix} 1 & 0 & -\frac{r+l}{2n} & 0 \\ 0 & 1 & -\frac{t+b}{2n} & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad \text{Scherungsmatrix}$$



- Lokales Koordinatensystem
- Vertex (x,y,z,w) / Normalen (x,y,z,w) / Farbe (r,g,b,a) / Material (amb. / diffuse / spec.) / Textur (u,v) /...
- Objekte in das globale Koordinatensystem transformieren
- Normalen mit der transponierten Inversen Matrix
- Kamera aufstellen (Position/ lookat/ up-vector)
- Transformation in das Kamerakoordinatensystem
- Auge im Ursprung / Blick entlang der negativen Z-Achse / Rechtssystem
- Beleuchtung, Illumination (Phong,...)
- Projektionstransformation
- Rechts ins Linkssystem Transformation vom Persp. Frustum in einem Quader
- Quader skalieren & zentrieren & auf -1/+1
- Culling / Clipping
- z-Werte für die Verdeckung benutzen (Z-Buffer)
- Projektion (Division durch w) (ab hier 2D)
- Viewport Transformation
- x,y Werte (-1...+1) werden auf das View Window (Pixel) abgebildet
- Rastering
- Shading
- Texturing

